Integrated Development Environment User's Guide

for Ezairo® 7100

M-20805-010
March 2016

© SCILLC, 2016
Previous Edition © 2015
"All Rights Reserved"

ON Semiconductor®

Integrated Development Environment User's Guide

ON Semiconductor and  are registered trademarks of Semiconductor Components Industries, LLC (SCILLC). Ezairo is either a trademark or registered trademark of SCILLC. SCILLC owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of SCILLC's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. SCILLC reserves the right to make changes without further notice to any products herein. SCILLC makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does SCILLC assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in SCILLC data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. SCILLC does not convey any license under its patent rights nor the rights of others. SCILLC products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SCILLC product could create a situation where personal injury or death may occur. Should Buyer purchase or use SCILLC products for any such unintended or unauthorized application, Buyer shall indemnify and hold SCILLC and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that SCILLC was negligent regarding the design or manufacture of the part. SCILLC is an Equal Opportunity/Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:
Literature Distribution Center for ON Semiconductor
P.O. Box 5163, Denver, Colorado 80217 USA
Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada
Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada
Email: orderlit@onsemi.com

N. American Technical Support: 800-282-9855 Toll Free USA/Canada
Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910
Japan Customer Focus Center: Phone: 81-3-5817-1050

ON Semiconductor Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative

Table of Contents

Page

1. Introduction 7
    1.1 Purpose 7
    1.2 The Integrated Development Environment 7
    1.3 Intended Audience 7
    1.4 Conventions 7
    1.5 Organization 8
    1.6 Further Reading 8
2. The Integrated Development Environment 9
    2.1 Introduction 9
        2.1.1 IDE Features 9
        2.1.2 The IDE Workbench 9
            2.1.2.1 Perspectives 10
            2.1.2.2 Views 10
            2.1.2.3 Editors 11
        2.1.3 Resources 11
            2.1.3.1 Workspaces and the File System 12
            2.1.3.2 Sample Code 12
        2.1.4 Online Help 12
    2.2 IDE Projects 12
        2.2.1 Launching the IDE 12
        2.2.2 Creating Empty Projects 15
        2.2.3 Adding Existing Files to Projects 17
            2.2.3.1 Copying or Importing Files or Folders 17
            2.2.3.2 Linking Files or Folders 19
        2.2.4 Creating New Projects from Sample Code Templates 21
        2.2.5 Browsing Sample Code 24
    2.3 The Source Editors 24
        2.3.1 Syntax Highlighting 25
        2.3.2 Breakpoint Manipulation 26
        2.3.3 Key Bindings 27
        2.3.4 Code Completion 29
        2.3.5 Context-Sensitive Dynamic Help 29
        2.3.6 Outline View 30
            2.3.6.1 CFX 30
            2.3.6.2 ARM Cortex-M3 Core 31
            2.3.6.3 Filter Engine 32
            2.3.6.4 HEAR 32
    2.4 Managed Build System 33
        2.4.1 Build Stages 33
        2.4.2 Build Settings 33
        2.4.3 Build Variables 34
        2.4.4 Managing and Editing Configurations 35

2.4.5 Parallel Building . . . 36
2.4.6 Auto-build . . . 37
2.4.7 Inter-Project Dependencies . . . 37
2.4.8 Viewing the Executable Program . . . 38
2.4.8.1 CFE File Viewer . . . 38
2.4.8.1.1 Absolute Data Tab . . . 38
2.4.8.1.2 Content View Tab . . . 39
2.4.8.1.3 CFE File Viewer Free Memory Tool . . . 40
2.4.8.2 ELF File Viewer . . . 40
2.5 Setting Preferences . . . 41
2.5.1 Accessing Preferences . . . 41
2.5.2 Organization and Categories of Preferences . . . 42
2.5.3 A Partial List of Preferences . . . 42
2.6 Advanced Features . . . 43
2.6.1 Launching External Tools . . . 43
2.6.2 Launching an External Builder . . . 44
2.6.3 Integrating Other Eclipse Plug-Ins . . . 46
2.6.4 Integrating Other Source Code Control Systems . . . 46
2.6.5 Update Site and Update Notifications . . . 46
3. Debugging in the Integrated Development Environment . . . 47
3.1 Introduction . . . 47
3.1.1 Default Layout . . . 47
3.1.2 Views . . . 48
3.1.2.1 Debug View . . . 48
3.1.2.2 Source Editors . . . 49
3.1.2.3 Disassembly View . . . 49
3.1.2.3.1 CFX . . . 49
3.1.2.3.2 ARM Cortex-M3 Core . . . 50
3.1.2.4 Registers View . . . 51
3.1.2.5 Memory View . . . 51
3.1.2.6 Expressions View . . . 52
3.1.2.7 Breakpoints View . . . 52
3.1.2.8 Debug Consoles . . . 53
3.1.2.8.1 CFX Debug Console . . . 53
3.1.2.8.2 ARM Cortex-M3 Core Debug Console . . . 53
3.2 Using the Debugger . . . 54
3.2.1 General Configuration . . . 54
3.2.1.1 CTK Configuration . . . 54
3.2.1.2 SK4 Initialize Tool . . . 54
3.2.1.3 ARM Debug Server . . . 55
3.2.2 Debugging CFX Programs . . . 55
3.2.2.1 Creating a CFX Debug Configuration and Launching a CFX Debug Session . . . 56
3.2.2.2 Running Without Debugging . . . 57
3.2.2.3 Connecting To and Debugging a Running Program . . . 58

3.2.3 Debugging Filter Engine Configurations . . . 59
3.2.4 Debugging ARM Cortex-M3 Core Programs . . . 60
3.2.4.1 Prerequisites for Launching an ARM Cortex-M3 Core Debug Session . . . 60
3.2.4.2 Creating an ARM Cortex-M3 Core Debug Configuration and Launching a Debug Session 60
3.2.4.3 Manually Creating an ARM Cortex-M3 Core Debug Configuration and Launching a Debug Session . . . 61
3.2.4.4 Manually Creating an ARM Cortex-M3 Core Launch Group . . . 62
3.2.4.5 Selecting a Default Launcher . . . 62
3.2.4.6 Viewing ARM Cortex-M3 Core Registers . . . 63
3.2.4.6.1 Installing Embedded System Register View . . . 63
3.2.4.6.2 Accessing the Peripheral Registers . . . 65
3.2.5 Debugging Both Cores . . . 65
3.2.5.1 Prerequisites for Coordinated Debugging . . . 66
3.2.5.2 Starting a Coordinated Debug Session . . . 66
3.2.5.3 Using Coordinated Resume . . . 67
3.2.5.4 Using Coordinated Suspend . . . 67
3.2.5.5 Coordinated Breakpoints . . . 68
3.2.5.5.1 Enabling Coordinated Breakpoints . . . 68
3.2.5.5.2 Procedure for Using Coordinated Breakpoints . . . 69
3.2.5.6 Troubleshooting Coordinated Debug . . . 69
3.3 Debugger Preferences . . . 69
3.3.1 CFX Debug Configurations . . . 70
3.3.1.1 Download Tab . . . 71
3.3.1.2 Import Symbols Tab . . . 72
3.3.1.3 Source Path Tab . . . 72
3.3.1.4 Connection Tab . . . 73
3.3.1.5 Scripting Tab . . . 75
3.3.1.6 Coordinated Debug Tab . . . 75
3.3.2 ARM Cortex-M3 Core Debug Configurations . . . 76
3.3.3 Debug Preferences . . . 77
3.3.3.1 Run/Debug . . . 78
3.3.3.2 Console . . . 78
3.3.3.3 Disassembly . . . 79
3.3.3.3.1 CFX Disassembly . . . 79
3.3.3.3.2 ARM Cortex-M3 Core Disassembly . . . 80
3.3.3.4 External Tools . . . 81
3.3.3.5 Launching . . . 82
3.3.3.6 Numeric Format . . . 83
3.3.3.7 Perspectives . . . 84
3.4 Jython Command Console . . . 85
3.4.1 An Introduction to the Interpreter . . . 85
3.4.2 Defining Functions . . . 87
3.4.3 Storing Functions in Modules . . . 88
3.4.4 Launching the Standalone Debugger . . . 88

3.4.5 Further Information . . . 89
3.5 ARM Cortex-M3 Core Command Console . . . 89
3.5.1 Interrupt Control on the ARM Cortex-M3 Core . . . 90
3.5.2 Creating ARM Cortex-M3 Core Hardware Breakpoints and Watchpoints . . . 90
3.5.2.1 Managing ARM Cortex-M3 Core Hardware Breakpoints and Watchpoints . . . 90
3.6 Expression Language . . . 91
3.6.1 Literals and Identifiers . . . 92
3.6.2 List of Expression Operators . . . 92
4. The Communications Toolkit (CTK) Configuration Manager . . . 95
4.1 Introduction . . . 95
4.2 Launching the CTK Configuration Manager . . . 95
4.2.1 From the Start Menu . . . 95
4.2.2 From the IDE . . . 95
4.3 Using the CTK Configuration Manager . . . 96
4.4 Configuration Examples . . . 97
4.4.1 Promira Serial Platform Examples . . . 97
4.4.2 Communication Accelerator Adaptor Examples . . . 98
5. Using the Documentation . . . 100
5.1 Introduction . . . 100
5.2 Using Online Help . . . 100
5.2.1 Full-Text Search with External Help . . . 101
5.2.1.1 Narrowing the Search Scope in External Help . . . 101
5.2.2 Full-Text Search with Integrated Help . . . 102
5.2.2.1 Narrowing or Expanding the Search Scope in Integrated Help . . . 104
5.2.3 Context-Sensitive Dynamic Help . . . 105
5.3 Printing the Documentation . . . 106
5.4 Stand-Alone Documentation . . . 106
A. DDB Command Reference . . . 107
A.1 Introduction . . . 107
A.2 Commander . . . 107
A.2.1 Methods . . . 112

CHAPTER 1
Introduction

1.1 Purpose

The Integrated Development Environment (IDE) provides a graphical user interface where you can edit, build and debug your applications. It is the main programming interface for the Evaluation and Development Kit (EDK). This user's guide provides information for developing software using the IDE. It also describes additional tools that the IDE provides to support application development tasks.

1.2 The Integrated Development Environment

The open-source Eclipse platform is used as the basis for the IDE. It provides a graphical user interface for developing applications that will ultimately run on embedded systems based on an ON Semiconductor programmable DSP. The IDE provides a single starting point for all development activities in the EDK and includes:

- A customizable framework of multiple views
- An editor, project manager, automated build system and integrated debuggers
- Links to the Communication Toolkit configuration manager
- Documentation, both as searchable online help and in printable (PDF) versions

The build process translates source code to machine code that can run directly on the chip. The software tools that the IDE uses to build code are also available from the EDK 1 Ezairo® 7100 Command Prompt. For more information, see the *CFX Toolchain Reference for Ezairo 7100*.

There are two integrated debuggers. The first, called DDB, is used to debug source code that runs on the CFX. It is a stop-mode debugger capable of controlling execution (running, stepping, setting breakpoints and tracing) as well as editing data memory, registers, and machine code, and displaying disassembly. A text-only version of DDB with the same functionality (but without the display capabilities) is available at the command prompt.

The second debugger, called GDB, is the standard debugger for the GNU software system and it is used to debug the code that runs on the ARM® Cortex™-M3 core. For more information, see the GNU website (*http://www.gnu.org/software/gdb*).

For more information about Eclipse, see the Eclipse *Workbench User Guide*.

1.3 Intended Audience

This guide is for people who would like a graphical user interface for designing, building and testing DSP applications. It assumes familiarity with software development activities.

1.4 Conventions

In general, numbers are presented in decimal notation. In cases where hexadecimal notation is more convenient, these numbers are identified by the prefix "0x". For example, the decimal number 123456 can also be represented as 0x1E240.

This guide uses the following special fonts to signify particular types of information:

`monospace`	Commands and their options, file and path names, error messages, code samples and code snippets.

`mono bold`	A placeholder for the specified information. For example, replace **`filename`** with the actual name of the file.
bold	IDE user interface labels, such as those for menus, menu items and buttons.
italics	File and path names, or any portion of them.

1.5 Organization

The *Integrated Development Environment User's Guide* contains the following chapters and appendices:

- **Chapter 1: Introduction** describes the purpose of this guide and how the book is organized, provides system requirements and gives a list of suggested reading for more information.
- **Chapter 2: The Integrated Development Environment** explains the components of the IDE and how to use them to write and build code, manage projects and get help.
- **Chapter 3: Debugging in the Integrated Development Environment** explains how to use the Dynamic Debugger in the IDE to debug your programs.
- **Chapter 4: The Communications Toolkit (CTK) Configuration Manager** describes how to set up communication between the Evaluation and Development Board and your PC.
- **Chapter 5: Using the Documentation** describes the documentation set and how to use the online help.
- **Appendix A: DDB Command Reference** lists and describes all the commands that can be used on the Jython console.

1.6 Further Reading

For more information, refer to the following documents:

- *SK4 ELF Toolchain Reference for Ezairo 7100*
- *Hardware Reference Manual for Ezairo 7100*
- *CFX DSP Architecture Manual*
- *Evaluation and Development Board Manual for Ezairo 7100*
- *Firmware Reference Manual for Ezairo 7100*
- *Introduction to Ezairo 7100 Programming*
- *Communication Toolkit Programmer's Guide*
- *Eclipse Workbench User Guide*
- Jython website: http://www.jython.org/
- Python website: http://python.org/
- Lutz, Mark, and Ascher, David. *Learning Python*. 2nd ed. (O'Reilly, 2003)
- Lutz, Mark. *Programming Python*. 2nd ed. (O'Reilly, 2001)
- Pedroni, Samuele, and Rappin, Noel. *Jython Essentials*. 1st ed. (O'Reilly, 2002)
- GNU project debugger (GDB) website: http://www.gnu.org/software/gdb
- *Promira Serial Platform Quick Start Guide*
  http://www.totalphase.com/support/articles/203908616-Promira-Serial-Platform-Quick-Start-Guide

CHAPTER 2
The Integrated Development Environment

2.1 INTRODUCTION

This chapter describes how to create new IDE projects. You must create at least one project before you can work in the IDE. These projects rely on makefiles that are auto-generated by the Eclipse CDT Managed Build System and the UNIX build utility, Make, to manage their build processes. To learn about the Managed Build System consult Section 2.4, “Managed Build System” on page 33.

Ezairo 7100 projects use `cs-make`, a Windows port of GNU Make included in the CodeSourcery ARM toolchain. To learn more about GNU Make, consult the GNU Make manual at http://www.gnu.org/software/make/manual/make.html.

2.1.1 IDE Features

The IDE includes the following features:

- An advanced editor with integrated help
- A project manager with support for team development (source code control integration)
- Compilation tools and an integrated build system
- Two integrated source code debuggers (described in detail in Chapter 3, “Debugging in the Integrated Development Environment” on page 47)
- An online documentation system (described in Chapter 5, “Using the Documentation” on page 100)

2.1.2 The IDE Workbench

The IDE is based on the open-source Eclipse platform. Much of the functionality in the IDE is common to that found in Eclipse, and the user documentation for Eclipse provides excellent tutorials on how to become an expert in that environment. The IDE is based on Eclipse 3.7.1, and the Eclipse *Workbench User Guide* is available as part of the online documentation in the IDE under **Help > Help Contents**.

The IDE’s main user interface is called the *Workbench*, and it organizes tools and projects to be accessible and to facilitate your workflow.

Workbench	A collection of *perspectives*. This is really another name for the IDE itself, and is commonly used in Eclipse documentation.
Perspective	A collection of *views* and *editors*. Only one perspective is visible at a time.
View	A view is a window in which the content is updated based on another view, or where changes are applied instantly (such as through a right-clicked context menu). For example, the Project Explorer view in the Development perspective lets you apply actions instantly to the files in your project.
Editor	An editor is a window that follows an Open-Save-Close update model. Different editors can be used for different file types.

You can rearrange editors and views within the current perspective shown by the Workbench. By clicking on the title bar of an editor or view, you can drag it to another area, or outside the IDE window to have a free-floating view window. As you drag the view around, the cursor will change to let you know whether you can dock the view, place it in that location, or whether the dropped location is not allowed.

After modifying the look of your Workbench, you may wish to return to an original arrangement. On the **Window menu, click Reset Perspective**, which restores the current perspective to its appearance when originally installed.

2.1.2.1 Perspectives

The main perspectives that are provided in the IDE are:

C/C++ perspective

This is a collection of views and editors designed for editing files, building code, navigating through projects, and interacting with the source code control system.

Debug perspectives

This is a collection of views and editors designed to help you debug code by allowing you to see the contents of memory and registers, look at a disassembly view, control execution of a program, and edit the code. The default Debug perspective is intended for Ezairo 7100 use, but there is also a legacy SignaKlara™ Debug perspective available. In general, we recommend using the default Debug perspective.

For more information about setting a perspective to be the default, see the **Perspectives** option when you follow the directions in Section 3.3.3, "Debug Preferences" on page 77.

2.1.2.2 Views

Each perspective presents a default set of views. You can add or remove views and rearrange them as you like (see the Eclipse *Workbench User Guide* for more information). See Section 2.5.2, "Organization and Categories of Preferences" on page 42 for information on customizing the perspectives you want to use.

Default C/C++ perspective

This perspective includes the following views:

Project Explorer

Shows all the projects in a workspace, and the folders and files in each project. For more information on project resources, see Section 2.1.3, "Resources" on page 11.

Outline

Shows key components of a source code or include file that is open in an editor.

Make Target

A list of build outputs for each project in the workspace.

Problems, Tasks, Properties

Standard views that are displayed by Eclipse. See the Eclipse *Workbench User Guide* for more information.

Console

Shows the results of building a project.

In addition to these views, this perspective also includes one or more *editors* that show the contents of files and allows you to change them.

Default Debug Perspective

This perspective has the following views:

Debug	Shows the execution status of the program that is being debugged.
Breakpoints	Shows the breakpoints and allows you to enable, disable or remove them.
Disassembly	Shows you memory as assembly instructions. Separate views are available for programs on the CFX and the ARM Cortex-M3 cores.
Registers	Allows you to view and modify registers.
Variables	Allows you to view and modify program variables (not applicable to assembly language programs).
Expressions	Allows you to compose expressions for evaluation whenever execution suspends.
Outline	Shows key components of a source code or include file open in a source editor.
Console, Tasks, Problems, Executables	Standard views that are displayed by Eclipse. See the Eclipse *Workbench User Guide* for more information.

In addition to the above views, this perspective also includes one or more *editors* that display the line in the source code where the program execution suspended.

2.1.2.3 Editors

The IDE provides editors for all four of the source code types used to program Ezairo 7100. Each of these editors allows you to view and modify source code. They offer syntax highlighting and advanced editing features, such as a Content Assist (code completion) pop-up window which suggests EDK keywords (macros, labels, assembler directives and mnemonics) that you can choose to complete the word you are typing. These editors incorporate not only context-sensitive help, but also dynamic help, which automatically searches the indexed online help system as you type, to provide information relevant to your current coding session. You can customize the appearance of source code in the editor. See Section 2.3.1, “Syntax Highlighting” on page 25 for more information.

When you are working with an EDK source code file or include file in the editor, the contents are linked to the Outline view, which summarizes the main components of the code, allowing you to find significant items faster. The Outline view dynamically changes as you edit the files or change to another editor.

When debugging, an editor shows you the current source file and line being debugged.

2.1.3 Resources

“Resources” is a collective term for the projects, folders and files that exist in the Workbench. *Projects* contain folders and files. Projects are used for builds, version management, sharing and resource organization. Like folders, projects map to directories in the file system. (When you create a project, you specify a location for it in the file system.)

A *workspace* is a collection of projects and preferences. As described in the Eclipse *Workbench User Guide*, projects are created in the current workspace, which is represented by a directory in the file system. Eclipse normally

works with one workspace at a time; you can switch between workspaces at any time by selecting **File > Switch Workspace...**, and you can open more than one instance of Eclipse to access two workspaces simultaneously.

Eclipse treats workspaces and projects as being distinct from the file system for the operating system (e.g., the NTFS file system for Microsoft® Windows®). A workspace has a predefined structure and data files containing project and environment information. To maintain this structure and data, a workspace is normally accessed only through Eclipse.

When you start the IDE for the first time, it creates a workspace in the location of your choosing. Most of the tools available in the IDE require you to create a project before proceeding further.

2.1.3.1 Workspaces and the File System

Eclipse allows you to export files from your workspace to the file system. It also allows you to import files from the file system into your workspace. When you import, you can make a copy of the original source or link to it directly. For more information about copying vs. linking, see Section 2.2.3, "Adding Existing Files to Projects" on page 17.

While a project is open, you can work on any of its files, and you will see its contents in the Project Explorer view. When a project is closed, it cannot be changed in the Workbench. The resources of a closed project will not appear in the Workbench, however, they still reside on the local file system.

NOTE: Closed projects require less memory. Since they are not examined during builds, closing a project can improve build time.

2.1.3.2 Sample Code

The EDK includes several programs and libraries, most of which are supplied in both compiled form (ready to download and run) and source code form.

You can browse the supplied source code by using the IDE to create a link in your workspace to a folder in the file system. For step-by-step instructions, see Section 2.2.5, "Browsing Sample Code" on page 24.

You can also create a project from the sample code using a wizard in the IDE. It copies the supplied sample code to your workspace and allows you to customize it from there. See Section 2.2.4, "Creating New Projects from Sample Code Templates" on page 21 for more information.

2.1.4 Online Help

The help system provided with the IDE includes the entire EDK documentation suite in online and printable (PDF) formats. It also includes the *Workbench User Guide* from Eclipse which offers help on using the general features of Eclipse. For more information about the organization of the documentation and accessing the help system, including the full-text search facility and context-sensitive dynamic help, see Chapter 5, "Using the Documentation" on page 100.

2.2 IDE Projects

This section describes how to create new IDE projects. You must create at least one project before you can work in the IDE.

2.2.1 Launching the IDE

To use the IDE for the first time, follow the steps below:

1. To start the IDE, go to the Windows Start menu, and select **ON Semiconductor** > **EDK 1 Ezairo 7100 IDE**.

2. When you open the IDE for the first time, you are prompted to select a workspace for the session, as shown in Figure 1. You can agree to use the suggested location, type in or browse to a different location.

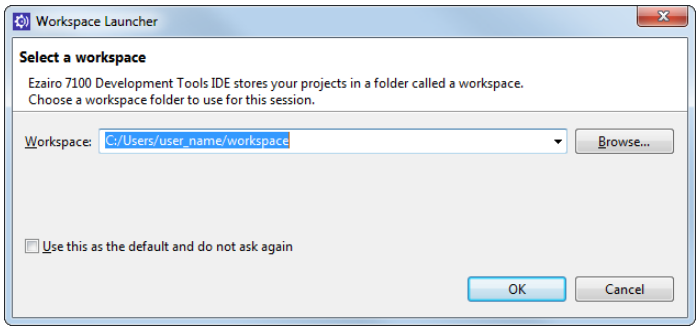


Figure 1. Workspace Launcher

The workspace is the work area for all your IDE projects. When you create a new project, it appears in a subfolder of this workspace folder (although you can select a different location when you create the project).

IMPORTANT: If you already had a workspace that you created for an ON Semiconductor DSP Systems device other than Ezairo 7100, create a new workspace for Ezairo 7100 development. Opening an existing workspace with the Ezairo 7100 IDE might cause the workspace to be modified in ways that are incompatible with the other product.

3. After selecting the workspace location, the IDE is launched with the Welcome screen (see Figure 2). The Welcome screen contains links to introductory information.

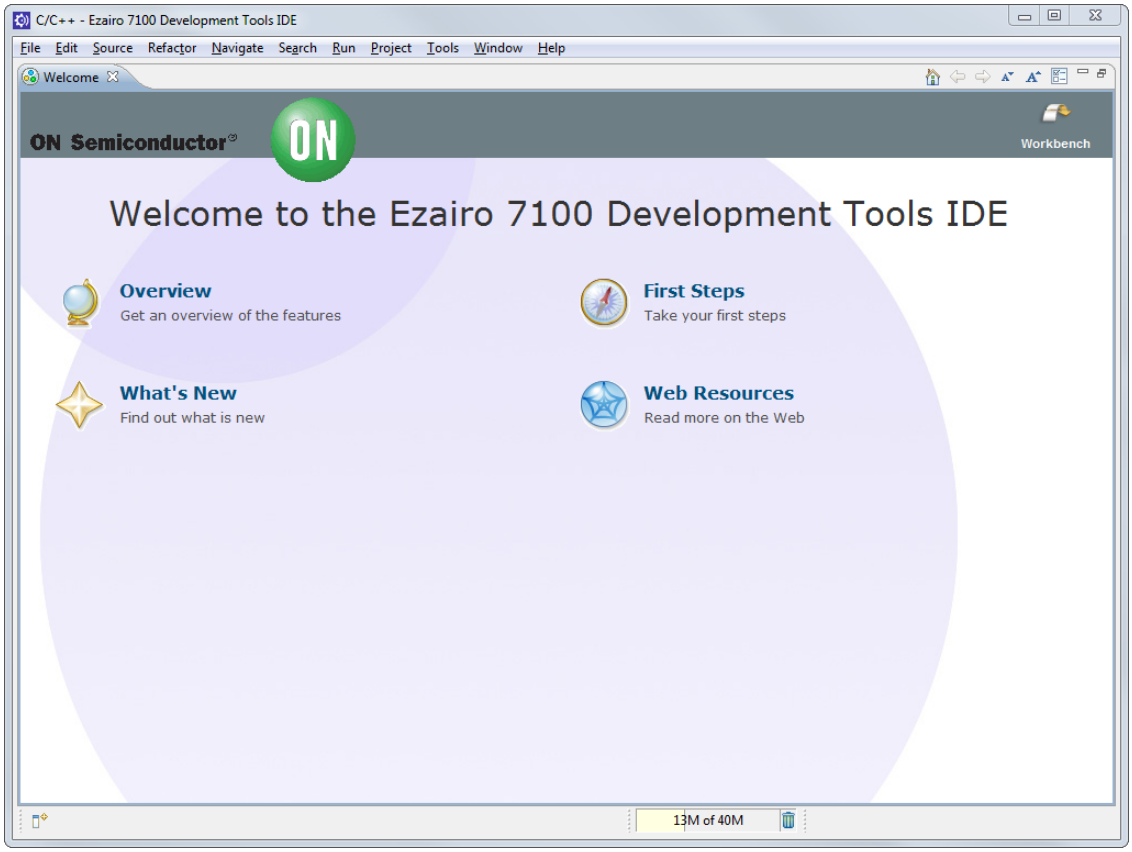


Figure 2. The Welcome Screen

4. When you are ready to start using the IDE, click the curved arrow that is labeled **Workbench** on the Welcome screen.

   Figure 3 shows the workbench opened with the default perspective, the C/C++ Perspective. This perspective has a Project Explorer on the left side, multiple stacked views at the bottom to manage your project, Outline view and Make Target view to the right, and the middle is reserved for an editor. You can also customize the default layout of the perspective to suit your preferences.

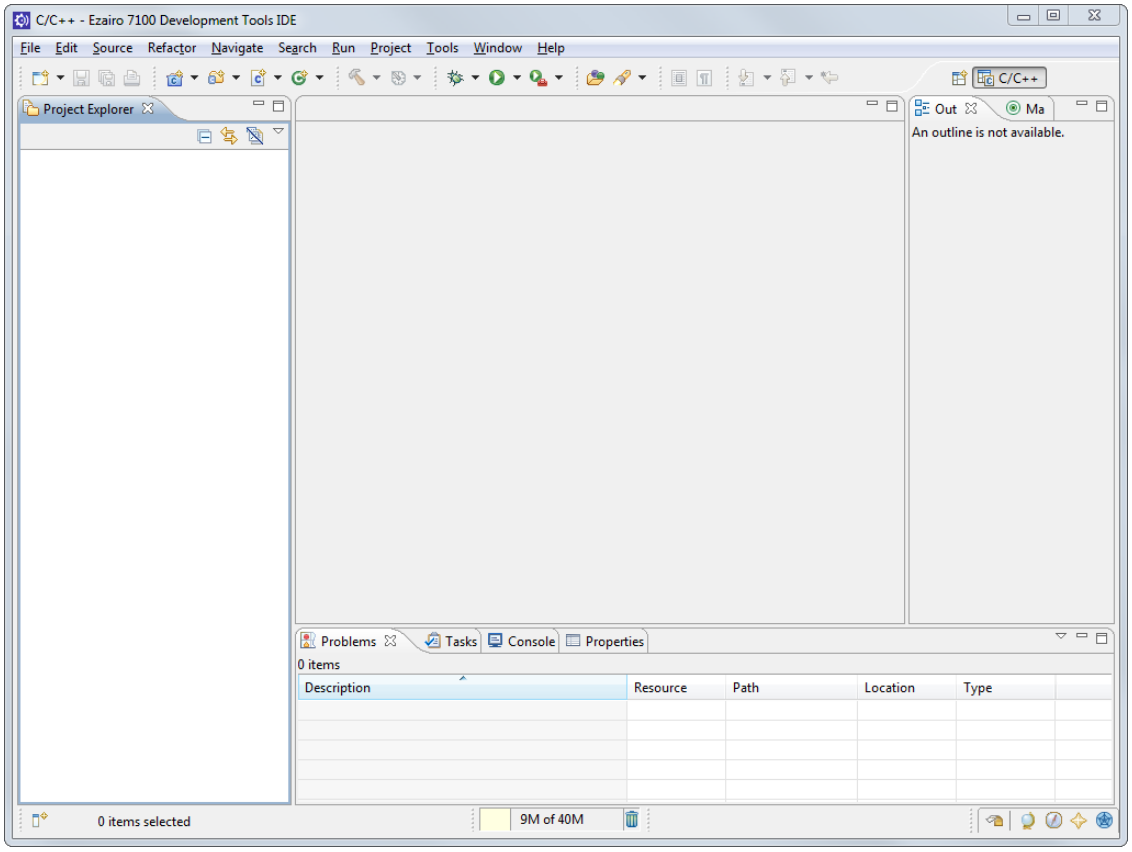


Figure 3. The Workbench

2.2.2 Creating Empty Projects

To create a new empty Ezairo 7100 project:

1. Click **File** > **New** > **Project**.
2. Expand the **SK4** category.
3. Select **SK4 Project** and click **Next**.
4. Type your project name in the text box.
5. In the **Project type** area, expand either the **SK4 Executable / EEPROM** or **SK4 Static Library** project type depending on the type of empty project you wish to create. For this example, expand the **SK4 Executable / EEPROM** project type.
6. Select **Empty SK4 Project**. Ensure that you do not select **Empty Project** as the **Empty SK4 Project** has required custom settings. On this dialog, you might notice project templates based on various types of sample code projects. If you are curious about them, each one comes with its own readme file, or you can get more explanation from the *Introduction to Ezairo 7100 Programming* manual.
7. At this point (see Figure 4 for an example) you can click **Finish** to create the project, or click **Next** to optionally change the project configuration. For this example, click **Finish** to avoid setting up any project configurations.

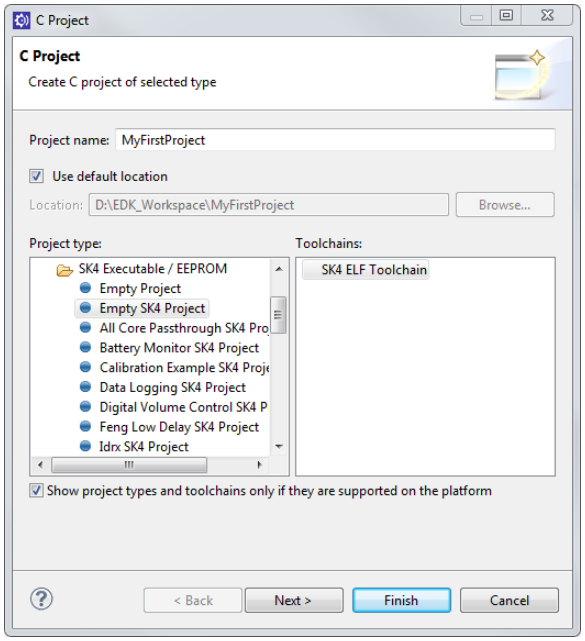


Figure 4. New SK4 Project Wizard Showing Empty Executable SK4 Project

The new project appears in the Project Explorer view (see Figure 5), and contains the following items:

1. *Includes:* Contains the IDE Environment includes. The IDE indexes and searches these folders when looking for files.
2. Makefile settings: The Managed Build System includes support for editable makefile settings files that are automatically included in the auto-generated makefile:
   - *makefile.defs*: This settings file is included after all the auto-generated sources, objects and subdirectory makefiles are included. If any C code exists in the project for the ARM Cortex-M3 core, files are added to the clean list and object files generated from the output of the ELF2CFS tool are added to the object list.
   - *makefile.init*: This settings file is included at the very top of the auto-generated makefile, and is used for additional initialization. By default, it ensures that Eclipse uses the Windows command shell to avoid confusion if another shell has been installed.
   - *makefile.targets*: This settings file is included after all of the generated targets, and allows you to specify additional targets. By default, it adds a pattern rule for the ELF2CFS tool which converts the ARM Cortex-M3 ELF file generated by the ARM GCC Linker to a CFX Assembly source file.

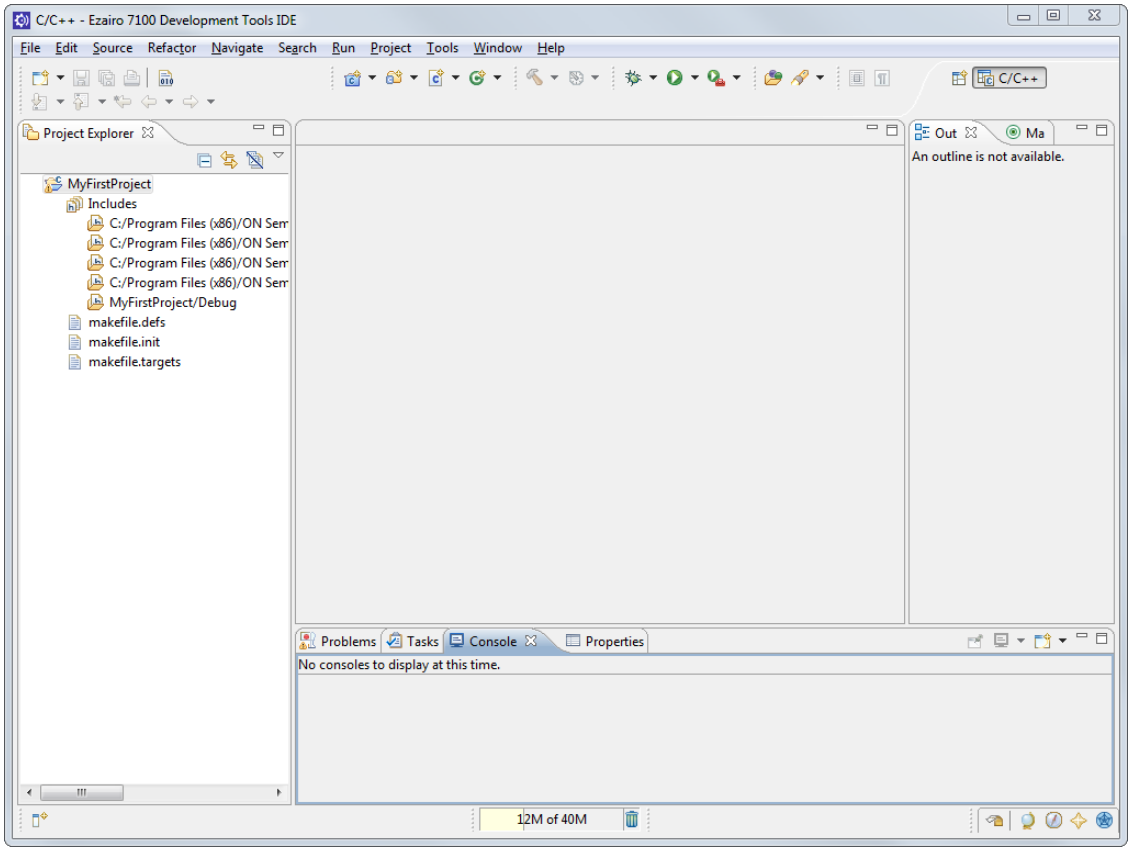


Figure 5. IDE Workbench Showing a New Empty Project in Project Explorer

2.2.3 Adding Existing Files to Projects

You can copy files into the project or link to them from the project. Copy the files to put them in your workspace and in your project directory. Link to the files to use them in their original location in the file system, but to allow them to be accessible as part of the project.

IMPORTANT: Eclipse does not prevent anyone from modifying files in a workspace from outside of Eclipse. However, if a resource is modified outside of Eclipse, the workspace falls out of sync with the filesystem and will need refreshing. Also, modifying a *.project* file or other Eclipse system file is generally discouraged because this might cause problems with your project or workspace.

2.2.3.1 Copying or Importing Files or Folders

To copy, either drag and drop files or folders from a file management window (e.g., Windows Explorer) to the project, or do the following:

1. In the Project Explorer view, right-click the project and click **Import...** You see a dialog like the one shown in Figure 6.

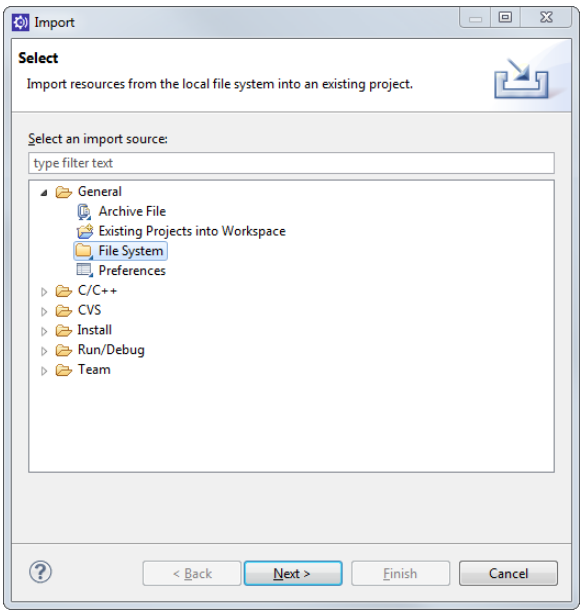


Figure 6. Importing Files and Folders to Your Project

2. Expand the General folder, select **File system** and click **Next**. An Import dialog appears like the one shown in Figure 7.

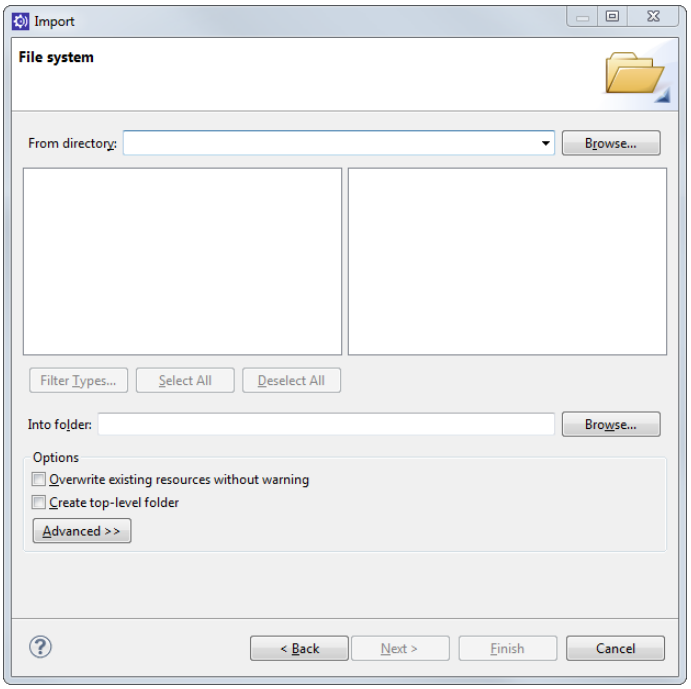


Figure 7. Selecting a Folder from the File System to Copy into a Project

3. Enter the directory path or click **Browse...** to navigate to a directory. Select all or part of its contents (files or folders). The list of resources you can choose from might look like Figure 8.

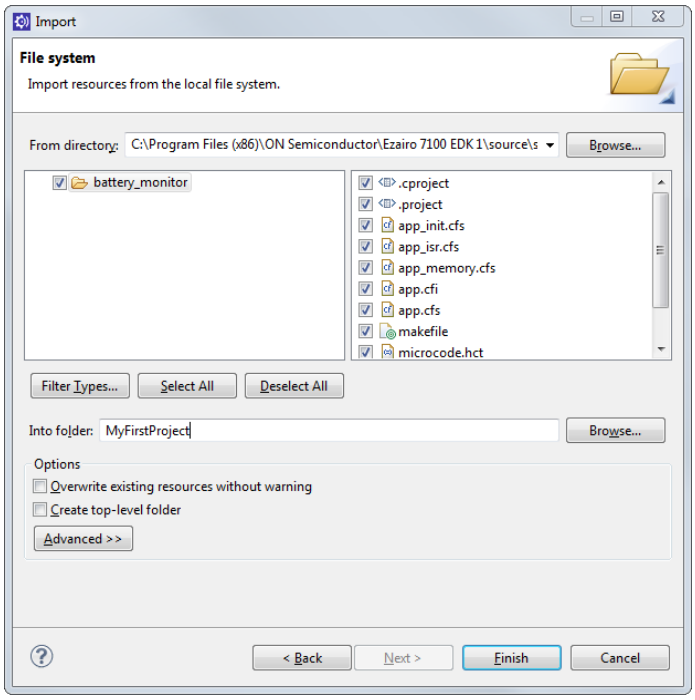


Figure 8. Choosing Resources to Include in your Project

4. Click **Finish** when you are done and the files appear under the project in the Project Explorer view.

2.2.3.2 Linking Files or Folders

To link files or folders to a project, do the following:

1. In the Project Explorer view, right-click the project in which you want to create the link and click **New > Folder** or **New > File**. The procedure is similar for both folders and files, so the following steps describe linking to a folder only.
2. In the New Folder dialog shown in Figure 9, verify the parent location where you want the new folder to reside, and give the folder a name in the **Folder Name** field.

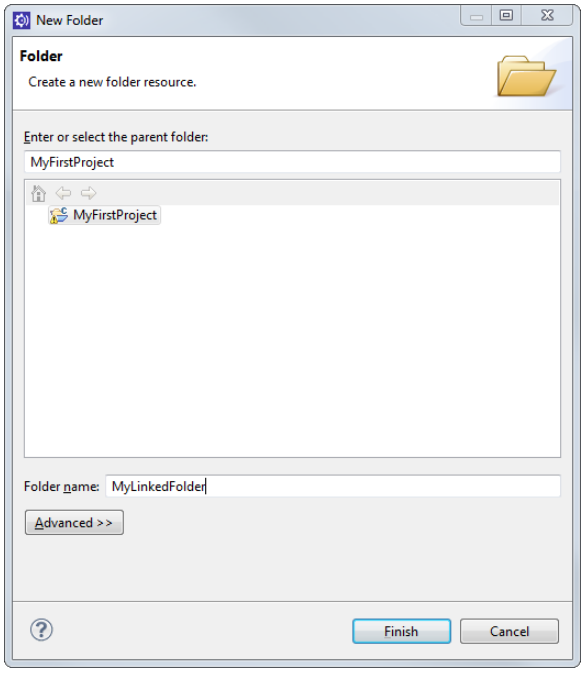


Figure 9. Defining a New Folder that Links to a Folder in the File System

3. Clicking the **Advanced**>> button displays the linking fields on the dialog. Click **Link to alternate location (Linked folder)**.
4. Click the **Browse...** button, navigate to the folder to which you want to link, and click **OK**.
5. Click **Finish**. The folder you created appears under the project in the Project Explorer view, and the folder icon has an arrow in a box to show it is a linked folder. If you expand that folder, you see the source code files underneath, as shown in Figure 10. Double-click any file to view it in the editor.

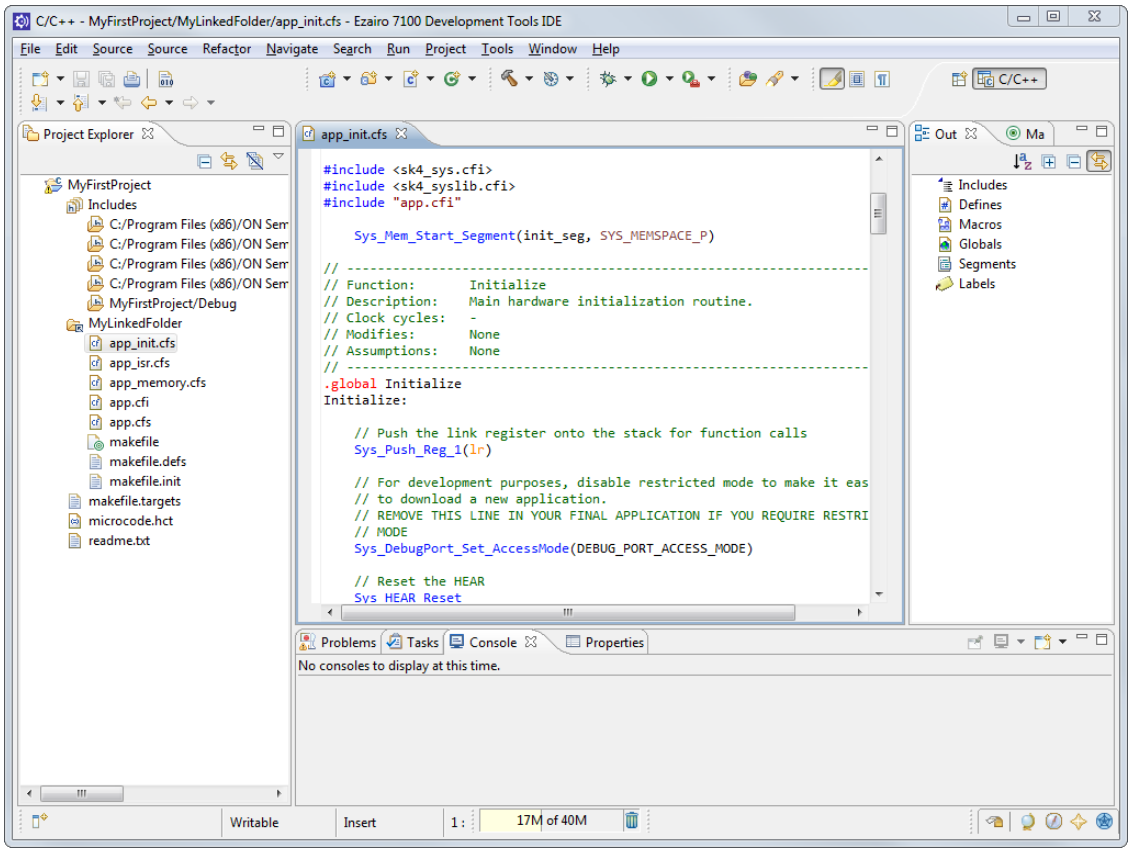


Figure 10. A Linked Folder Displaying the Source Code Files

NOTE: Not all IDE operations fully support linked resources. Use linked resources for read-only access to external locations that are not part of your project build.

2.2.4 Creating New Projects from Sample Code Templates

To create a new project from a sample code template:

1. Chose **File** > **New** > **Project**.
2. Expand the **SK4** category.
3. Select **SK4 Project** and click **Next**.
4. Enter your project name in the text box.

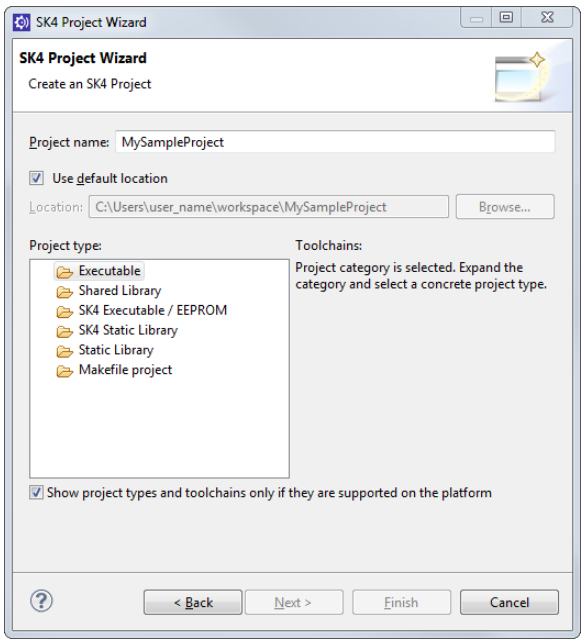


Figure 11. Naming your Project

5. In the **Project type** area, expand the **SK4 Executable / EEPROM** project type.
6. Select the sample project you wish to use.

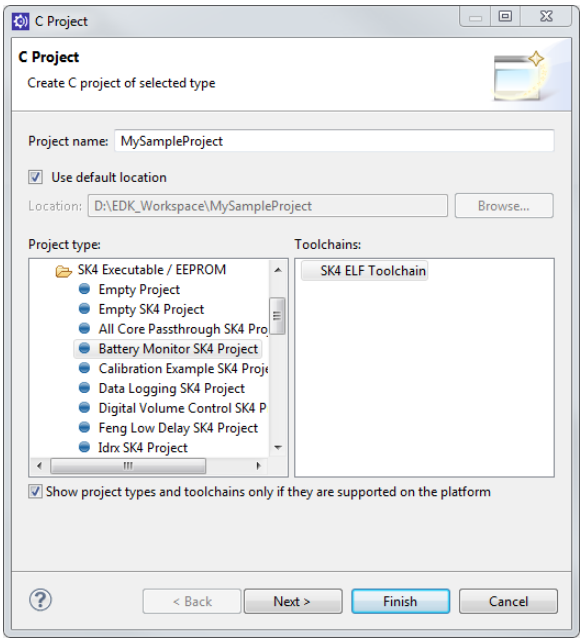


Figure 12. Selecting the Type of Project

7. At this point you can click **Finish** to create the project, or click **Next** to optionally change the project configuration. For this example, click **Finish** to avoid setting up any project configurations.

Your project files appear in the Project Explorer view at the left of the IDE's Workbench.

1. To build the project, do one of the following:
   - Right-mouse click the project name and select **Build Project**.
   - Select the project name and choose **Project > Build Project** from the menu bar.

   Build messages display in the Console view. For more information about configuring the build, see Section 2.4, "Managed Build System" on page 33. To automatically build projects whenever they change, see Section 2.4.6, "Auto-build" on page 37.

2. Expand the project. Notice the **Debug** folder, which is created when your project has been built. It contains the output executable file generated by this project, a *.cfe* file, which you can download to the chip's program memory, and use with the DDB integrated debugger.
3. To edit a source code file, double-click it. For a complete description of each file type, see Table 1.

Table 1. CFX ELF Toolchain File Extensions

File Type	ELF Toolchain
CFX assembly code	*.cfs*
CFX assembly header file	*.cfi*
preprocessed CFX assembly code	*.cft*
object file	*.o*
library	*.a*
CFX executable	*.cfe*
HEAR microcode	*.hct*
Filter Engine source code	*.fs*
ARM Cortex-M3 header file	*.h*
ARM Cortex-M3 source code	*.c*
ARM Cortex-M3 executable	*.elf*
EEPROM image	*.nvi*
loadmap file	*.map*
HEAR summary	*.smy*

The workbench will look similar to Figure 13.

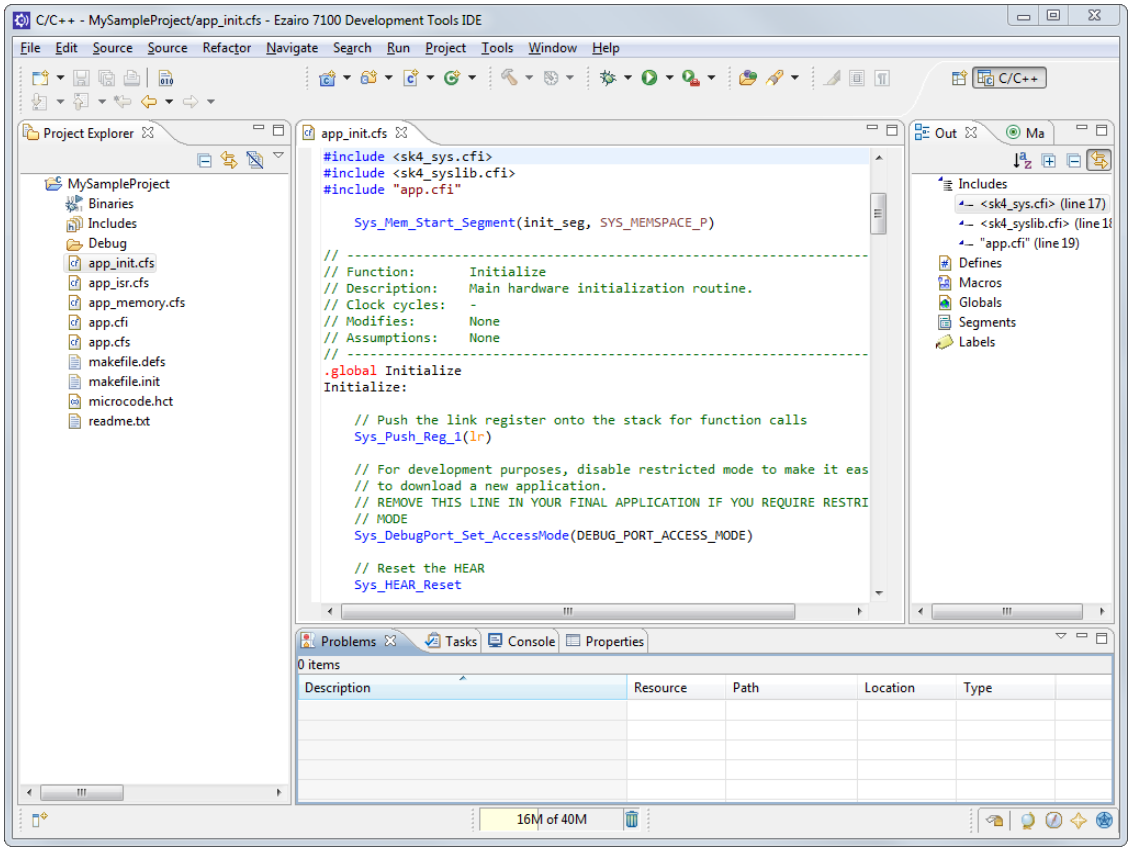


Figure 13. IDE with Sample Code Source Open in Editor

2.2.5 Browsing Sample Code

If you would like to look at the sample code without changing it, it is available in read-only files. From the IDE, you can create a link in your workspace to the sample code folder (or to any folder) in your file system:

1. You must have already created a project before you can create the link.
2. See Section 2.2.3.2, "Linking Files or Folders" on page 19 for the procedure on linking to folders or files. The locations of all the sample code are provided in the *Introduction to Ezairo 7100 Programming*.
3. These files are read-only, so if you attempt to save any changes, you are prompted to make the file read-write. We recommend that you do not save any changes to the sample code provided with the EDK; instead, you should edit a copy of the read-only versions provided.

2.3 The Source Editors

The source editors provided in the IDE are designed for the kind of work you would typically do in the EDK IDE. They feature:

- Syntax highlighting with customizable preferences
- Breakpoint manipulation (SK4 Source Code and C/C++ editors only)
- Key bindings that provide user-configurable keyboard shortcuts
- Content Assist, which provides code completion and user-customizable code templates

- Context-sensitive dynamic help
- An outline view

If you are editing file formats that the IDE does not support, you might prefer to customize your Workbench to use an external editor for those file types. Refer to the Eclipse *Workbench User Guide* for instructions on accessing an external editor and for standard editor features not described here.

NOTE: If you have automatic build turned on, the IDE immediately builds your project when you change a file in the editor and save it. For more information about the build process, see Section 2.4, “Managed Build System” on page 33.

2.3.1 Syntax Highlighting

The following syntax elements[1] can be highlighted in specific colors, bold, italic or any combination of these:

- Assembler directives
- Instruction mnemonics
- Instruction operands
- Assembler base conditions (CFX assembly only)
- Preprocessor directives
- Connecting symbol '|||' (CFX only)
- Connecting symbol '>>>' (CFX only)
- Connecting symbol '!!' (CFX only)
- Single-line comments
- Multi-line comments
- Define names
- Macro names
- Numeric values
- String literals
- Default text
- Operators (Filter Engine only)
- Program section labels (Filter Engine only)

To view the current settings and to change them:

1. Select **Window > Preferences**. This displays a Preferences dialog as shown in Figure 14 on page 26. For assembly language syntax highlighting, expand the **SignaKlara Development** item in the left panel to see the Editor and the Syntax Coloring preference categories. To configure C/C++ syntax highlighting preferences, expand the **C/C++** item in the left panel and click **Editor** > **Syntax Coloring**.

1. The IDE provides configurable syntax-highlighting editors for more than just assembly language. For the sake of brevity, not all editor syntax categories are discussed here.

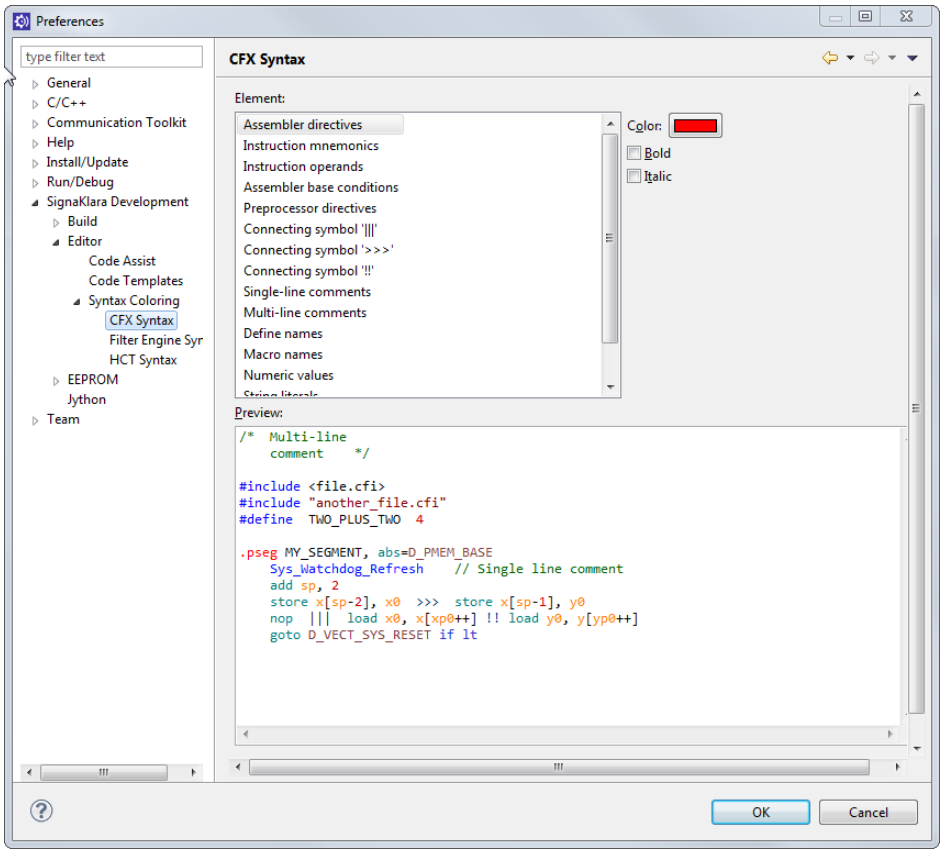


Figure 14. Viewing and Changing Editor Preferences

2. If you click **Editor > Syntax Coloring** and choose the editor you are interested in, you can change how the syntax is displayed. Choose a language element, then click **Color** or select **Bold** or **Italic** to change the highlighting for that element. The **Preview** box shows the effect your changes will have on the Editor.
3. If you click directly on **Editor**, you can configure more general settings such as whether to highlight matching brackets. You will also find a link to the global **Text Editors** preference page where you can configure tab widths, current line highlighting, foreground and background colors, as well as whether to insert spaces for tabs.
4. When you are finished, click **OK**.

2.3.2 Breakpoint Manipulation

In the SK4 Source Code editor and the C/C++ editor, you can:

- Toggle breakpoints on or off
- Skip all breakpoints
- Remove all breakpoints

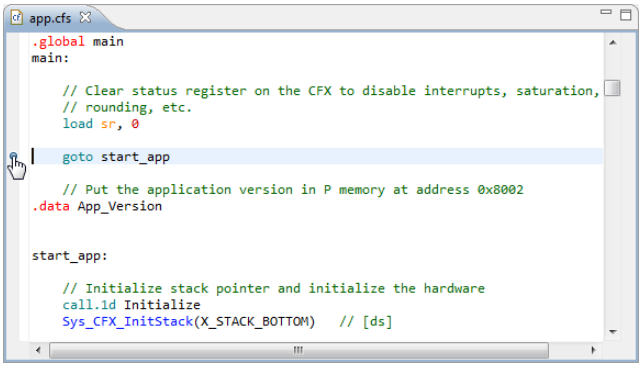


Figure 15. Setting a Line Breakpoint

To set a breakpoint, double-click in the left margin next to the line in the source code, and a blue dot appears, as shown in Figure 15 on page 27. You can toggle a breakpoint in one of several ways:

- Double-click on the blue dot and it will disappear.
- Right-click on the blue dot and select **Toggle Breakpoint**.
- With the breakpoint line selected, click **Run > Toggle Line Breakpoint**.

To skip all breakpoints, click **Run > Skip All Breakpoints**. This option leaves the breakpoints in place but disables them while the program is running. See how the blue dot in the margin changes in Figure 16 when breakpoints are being skipped.

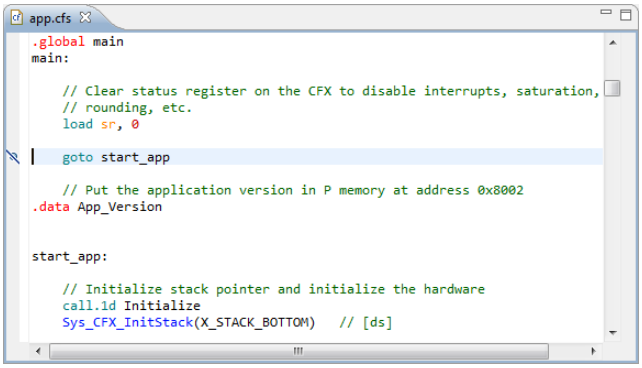


Figure 16. Icon Showing that Breakpoints Will Be Skipped

To remove all breakpoints, click **Run > Remove All Breakpoints**. You are asked to confirm this action before it is done because all the breakpoints in the code are deleted.

For more information about breakpoints, see Section 3.1.2.7, “Breakpoints View” on page 52.

2.3.3 Key Bindings

You can extensively customize the function of the keyboard in Eclipse. To view and change the key bindings (otherwise known as keyboard shortcuts), click **Window > Preferences > General > Keys**, and you will see a dialog similar to Figure 17.

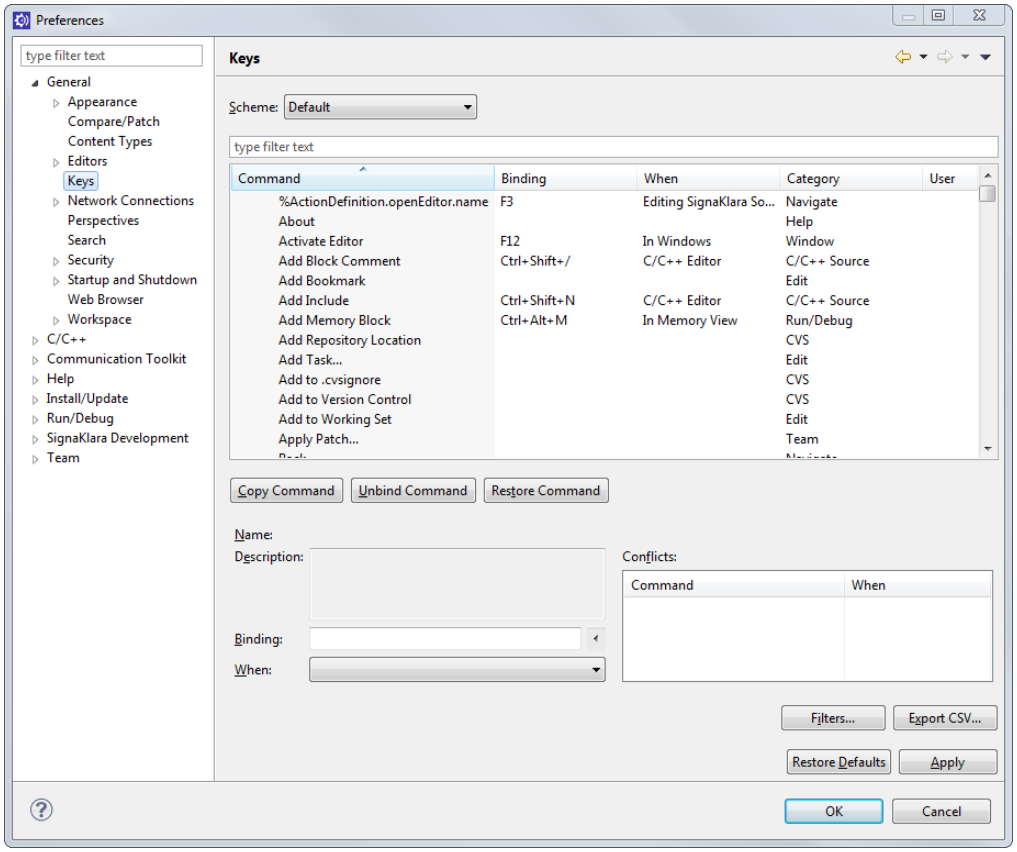


Figure 17. Dialog Box for Key Binding

The following key bindings are probably the most useful:

- Content Assist (**Ctrl+Space**), which is described in more detail in Section 2.3.4, "Code Completion"
- Build Clean (**Ctrl+Shift+C**)
- Toggle Breakpoint (**Ctrl+Shift+B**)
- Toggle Comment (**Ctrl+/**), which you can use on multiple lines
- Search (**Ctrl+H**)
- Build Selected Project (**Ctrl+Shift+F**)
- Build All Projects (**Ctrl+B**)
- Open Resource (**Ctrl+Shift+R**)
- Go To Line (**Ctrl+L**)

You can also use the standard cut (**Ctrl+X** or **Shift+Delete**), copy (**Ctrl+C** or **Ctrl+Insert**) and paste (**Ctrl+V** or **Shift+Insert**) key bindings.

To see which key bindings are available for use depending on where you are in the editor, select **Help > Key Assist...** (or **Ctrl+Shift+L**) and a pop-up window similar to Figure 18 appears.

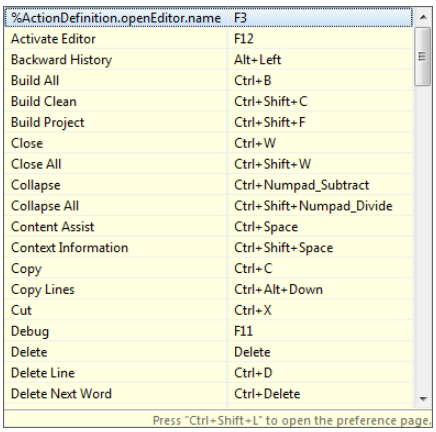


Figure 18. Key Bindings that are Available at the Current Place in the Editor

The pop-up window disappears when you click elsewhere on the Workbench.

2.3.4 Code Completion

The code completion feature is called Content Assist in Eclipse. When you are working in a source editor, you can use a key binding (**Ctrl+Space**) to display a pop-up window that has a list of possible commands, instructions or function calls that you might like to use based on what you have already typed. For example, if you had gotten as far as typing `ca`, and then pressed **Ctrl+Space**, you would get a pop-up as shown in Figure 19.

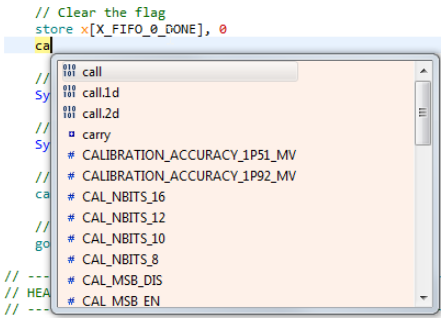


Figure 19. Content Assist Pop-up with Suggestions for Completing Your Code

Double-click your selection (or use the cursor keys and press Enter) and it automatically completes what you were typing. If you click anywhere else on the Workbench or press **Esc**, the pop-up window disappears.

2.3.5 Context-Sensitive Dynamic Help

As you enter code in the editor, or if you select any text in the editor, you can get context-sensitive dynamic help by pressing **F1** or selecting **Help > Dynamic Help**. This displays a help view that is updated based on the cursor location in the active view, and provides information about the current context of the active view. In the case of the source editors, the indexed online help system is searched, and information is provided at two levels:

- The top level provides help for the view.

- The second level provides help for the text typed, selected or near the cursor in the editor (in other words, dynamic help).

You can see an example of the Help view in Figure 20. As you select or type different text, the help in the dynamic portion of the Help view changes.

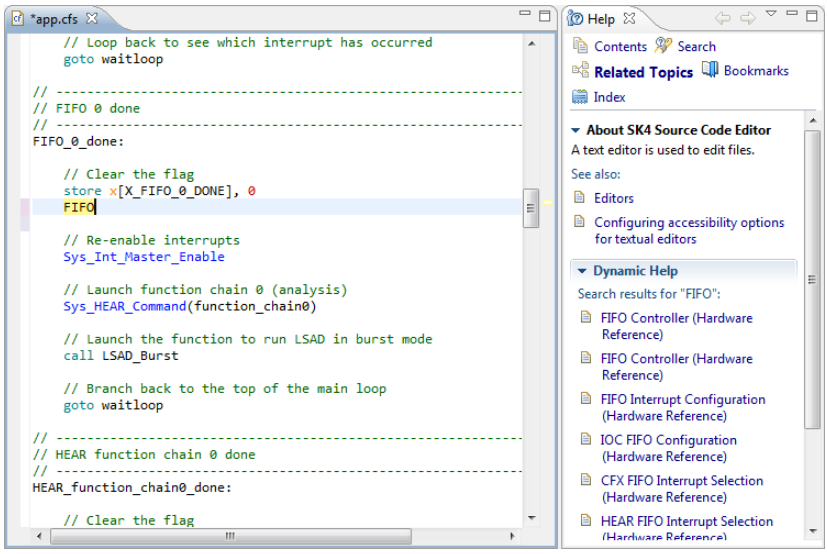


Figure 20. The Help View Displaying Context-Sensitive Dynamic Help

For more information about context-sensitive dynamic help, see Section 5.2, "Using Online Help" on page 100.

2.3.6 Outline View

When you are editing source files or include files, you can get an outline view of the code. The outline view shows the logical structure of the code so that you can browse through it to quickly find significant components. Depending on whether you are editing CFX, ARM Cortex-M3 core, Filter Engine or HEAR source files, these components are described below.

2.3.6.1 CFX

The components visible when editing CFX source are:

Includes — Header files that have been included with `#include` directive.

Defines — Constants that have been set with `#define`.

Macros — Macros that have been defined with `#macro`.

Globals — Any labels that can be referenced in other files. They are defined with the `.global` directive.

Segments — The labels for memory segments defined by the `.segment` directive (or the `.pseg`, `.xseg`, `.yseg`, and `.xyseg` aliases).

Labels — Named points in the code.

If any of these components are in the code you are currently editing, you can tell by glancing at the Outline view because of the plus sign (+) next to the component name. Expand that component, select an item, and the Editor view shows you that spot in the code, as illustrated in Figure 21.

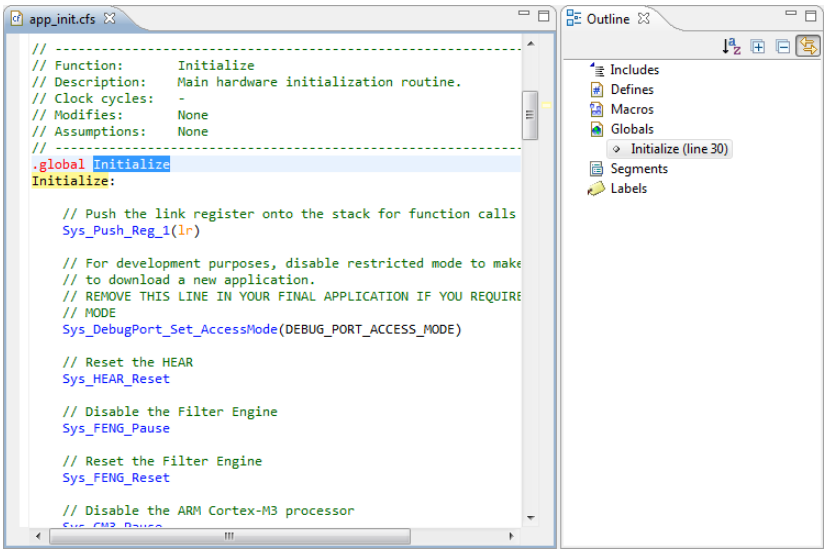


Figure 21. Selecting a Component in the CFX Outline View and Seeing it in the Editor

If you have more than one Editor view open, the Outline view changes as you change editors.

2.3.6.2 ARM Cortex-M3 Core

The components visible when editing ARM Cortex-M3 core source are:

Include Directives Header files that have been included with `#include` directive.

Functions Any functions defined in the source are visible here. Clicking on a function name will jump to its definition within the source.

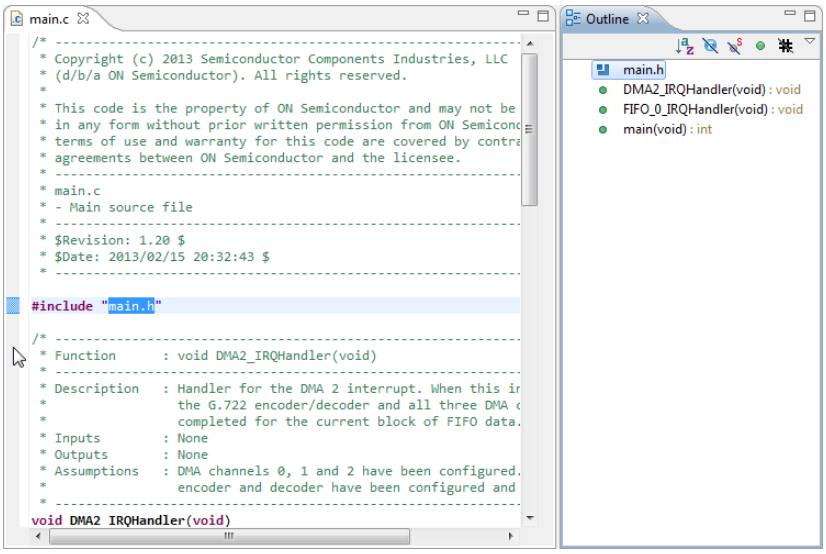


Figure 22. Selecting a Component in the ARM Cortex-M3 Core Outline View and Seeing it in the Editor

If you have more than one Editor view open, the Outline view changes as you change editors.

2.3.6.3 Filter Engine

The components visible when editing Filter Engine core source are:

Includes Header files that have been included with `#include` directive.

Macros Macros that have been defined with `#macro`.

Defines Constants that have been set with `#define`.

Labels Filter Engine source code "root" section and any other phase memory sections.

2.3.6.4 HEAR

The components visible when editing HEAR source code are:

Includes Header files that have been included with `#include` directive.

Macros Macros that have been defined with `#macro`.

Defines Constants that have been set with `#define` or `Define()`.

FIFOs Configured FIFO buffers. A FIFO is a remapping of a circular buffer such that the most recent samples are always located at the same fixed address in memory. See the *HEAR Configurable Accelerator Reference Manual* for more information.

Function Chains A sequence of microcode functions that are executed in response to a command from the CVF or an event from the FIFO controller.

Events An event from the FIFO controller that can be used to trigger the execution of a function chain.

2.4 MANAGED BUILD SYSTEM

The Managed Build System, based on the Eclipse CDT Managed Build System, is responsible for building output files from the source code in your project. The build system in the IDE automatically generates GNU Make build rules based on the contents of the project, and uses an integrated builder to generate appropriate makefiles. Depending on the type of project, this output will be an executable file or library file.

The build settings are configured through a special Build Settings Editor, and the build process can be extended with customized pre- and post-build steps. The IDE also has a feature that automatically builds projects whenever a file is saved.

2.4.1 Build Stages

The Ezairo 7100 system consists of four processors, each of which is programmable to some degree. Build tools for the CFX, HEAR, Filter Engine and ARM Cortex-M3 cores are integrated into the Managed Build System, under the SK4 ELF Toolchain. The automatic build process is divided into three stages: generating any assembly source files from Filter Engine source or ARM Cortex-M3 processor subprojects, compiling or assembling the source files into object files, and combining the object files to produce the final project output. The Managed Build System uses the toolchain driver and object librarian to create the output files by performing the following operations:

- Assembler source and header files are generated if necessary from the Filter Engine assembler, HEAR Configuration Tool, or possibly ELF2CFS tool if this project includes ARM Cortex-M3 source code.
- The Managed Build System uses the assembler stage of the toolchain driver to assemble source files into object files.
- For executable projects, the Managed Build System uses the linker stage of the toolchain driver to combine the object files into an executable. The object file created by the HEAR assembler is also linked at this point.
- For library projects, the Managed Build System uses a librarian tool to combine the object files into a library. The object file created by the HEAR Configuration Tool is also combined at this point.

2.4.2 Build Settings

You can access your project's build settings by performing the following operations:

1. In the Project Explorer view, right-click your project and select **Properties**.
2. Expand **C/C++ Build**.
3. Select **Settings**.

This editor page (shown in Figure 23) allows you to configure tool-specific build settings as well as create and edit configurations for your project build. Configurations contain instances of build settings. You can have as many configurations as you like. To create, rename or delete configurations, click **Manage Configurations...** beside the configuration selection drop-down.

Build settings control the behavior of the various stages of the project build. In addition to general categories and settings, each build tool has a set of categories containing build settings specific to that tool. To edit your build settings, click the category on the left and edit the settings on the right.

Using the **Restore Defaults** button you can reset all settings in the current build configuration to their default value.

The Build Settings Editor allows you to freely edit the build settings in a working copy of the project's build settings. When you are finished editing, save all the information to the project by clicking **Apply**.

For more information about the build settings for the CFX, or to build a project from a command line, see the *SK4 ELF Toolchain Reference for Ezairo 7100* .

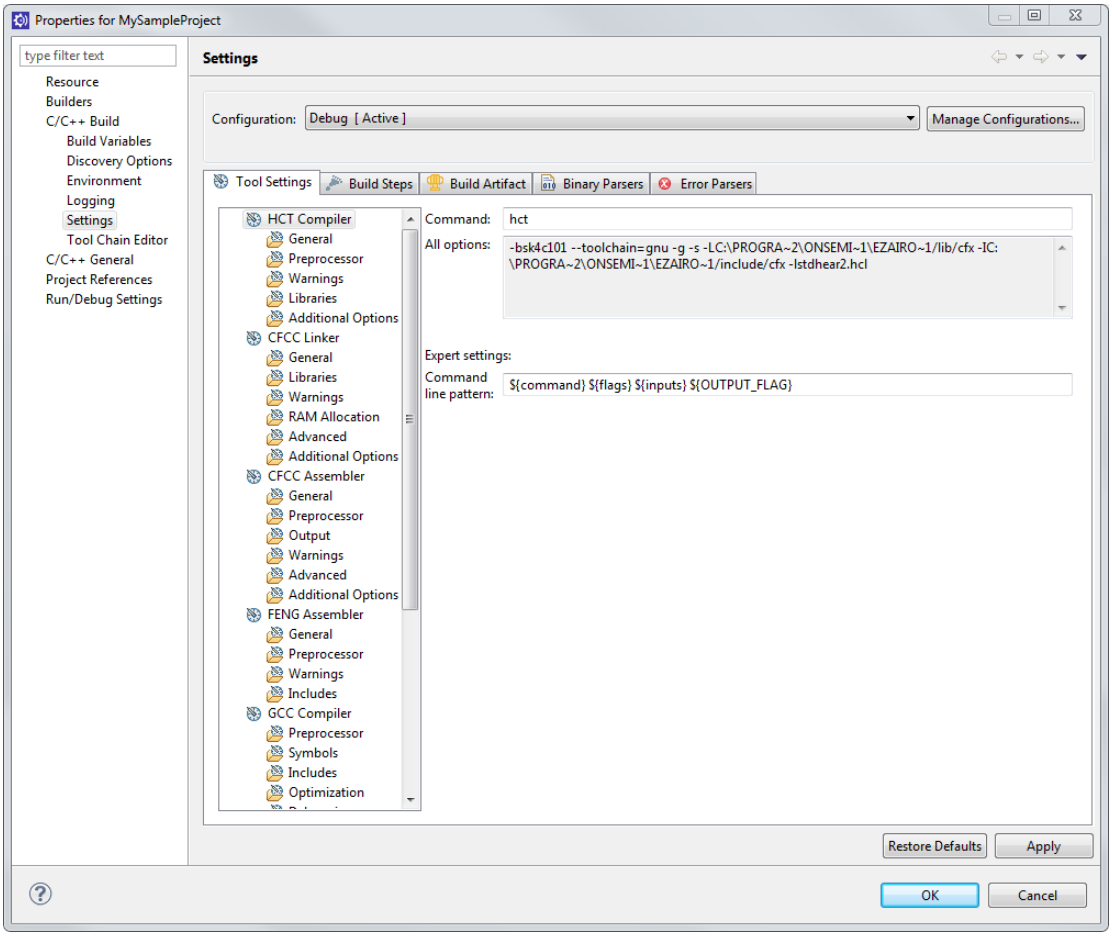


Figure 23. Build Settings Editor

2.4.3 Build Variables

The IDE replaces build variables with corresponding values before your project is built. They are of the format ${VariableName}. They can be used in your project's build settings to easily maintain common settings across various tools. The build system uses several default build variables.

To access build variables:

1. Right-click on your project and select **Properties**
2. Expand **C/C++ Build**
3. Select **Build Variables**
4. At this point you can add new variables. To view and modify default supplied variables, select the **Show system variables** check box.

2.4.4 Managing and Editing Configurations

The Managed Build System stores build settings within project configurations. Your project must have at least one configuration. You can create multiple configurations that allow you to build multiple versions of your project's output artifacts using different build settings.

To add, remove, rename or reorder build configurations:

1. Right-click the project name.
2. Click **Build Configurations > Manage...**. This presents the dialog shown in Figure 24.

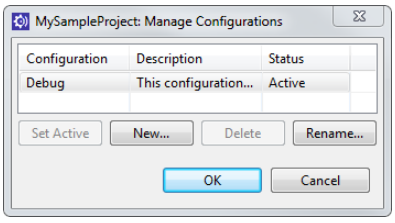


Figure 24. Manage Configurations Dialog

3. Click **New**. This presents the dialog shown in Figure 25.

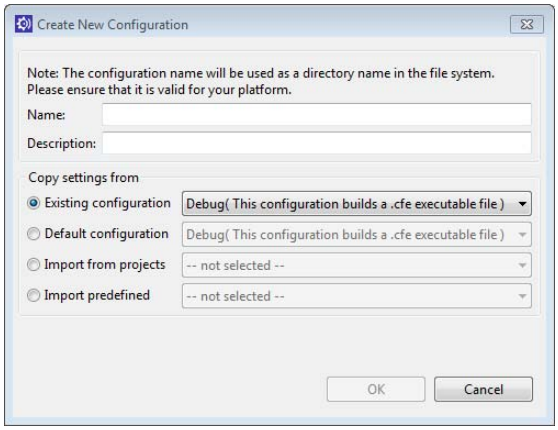


Figure 25. New Configuration Dialog

4. Give your configuration a name and a description. The build configuration name must be unique and must not overlap with an existing folder in your project. If certain build configurations must be built before others in your project, remember that build configurations are always built in the order in which they appear in the Manage Configurations dialog.
5. Leave **Copy settings from Existing configuration** selected as it is usually convenient to copy an existing configuration and then modify it.

The Managed Build System will let you configure your project to build either an executable plus an EEPROM image, or a library. This helps to avoid creating projects that attempt to accomplish too much. For example, if you are writing code that is intended to be a reusable library consumed by many applications, it makes sense to segregate this into a simple library project on its own, instead of including it as a single build configuration of a large project containing many unrelated pieces.

2.4.5 Parallel Building

You can use the parallel build feature to reduce project build time. Parallel build distributes and executes the build jobs for each project across multiple cores on your processor. The reduction in build time depends on the number of processor cores, processor speed, and the load from other processes on your system. Larger projects often see the best improvements in build speed.

You can turn on parallel build for a project by performing the following operations. To help you to navigate through the Project Explorer view, refer to Figure 26.

1. In the Project Explorer view, right-click your project and select **Properties**.
2. Click **C/C++ Build**.
3. Open the **Behaviour** tab.
4. Select the **Use parallel build** checkbox.

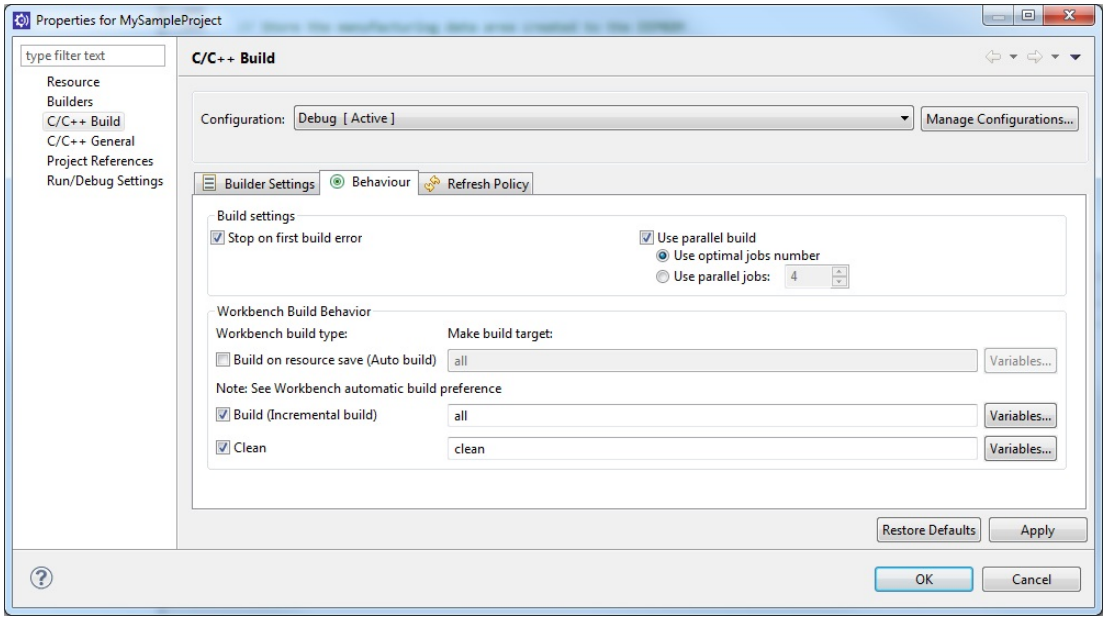


Figure 26. Build Behaviour Tab for Parallel Building

From this point, you can choose one of the following ways to set the number of cores used for the build:

- You could choose **Use optimal jobs number**, which allows the system to determine the number of cores used. Note that Eclipse dynamically chooses the number of parallel jobs based on both the number of cores in your computer and the content of the project. For example, there is no advantage to parallel building a project consisting of serial processes; in this case, the optimal number of jobs is one.
- You could select the **Use parallel jobs** radio button and manually set the maximum number of cores to use for this project. This option is best used when you would like to perform a parallel build, but not use all of your cores due to constraints from other processes.

Sometimes parallel build is not advantageous. Parallel build compiles multiple files in parallel, not serially compiled one after each other. Consequently, any compilation errors are not returned in order, and appear disorganized.

We suggest turning off parallel build when fixing many errors in a project. Parallel build is off by default to avoid this confusion.

2.4.6 Auto-build

If auto-build is turned on, the IDE Workbench automatically rebuilds a project when one of the following occurs:

- A file included in the build (or one of its dependencies) is changed or deleted
- A valid input file is created or added to the project
- The project is cleaned by clicking **Project** > **Clean...**
- A dependent project is built

As part of this project rebuild, the build system performs dependency calculations in an attempt to calculate exactly what needs to be rebuilt, and what is already up to date. This allows the build system to rebuild only those files that have changed since the last build. You can continue to edit your source code while the build is running. Any build errors are displayed in the editors and the Project Explorer, Console and Problems views.

It is sometimes helpful to turn off this feature, especially when making a large number of changes and you know that the source code will not build correctly until you are done.

To enable or disable the auto-build feature, click **Project** > **Build Automatically**. Furthermore, you have to set auto-building for each project. Do the following:

1. Right-click the project name in the Project Explorer view, and click **Properties**.
2. Select **C/C++ Build** from the list on the left.
3. Click the **Behavior** tab.
4. Select **Build on resource save (Auto build)** and click **OK**.

If **Project** > **Build Automatically** is already enabled, the project builds immediately.

2.4.7 Inter-Project Dependencies

When auto-build is turned on, the build system automatically rebuilds a project whenever the state of that project changes. Normally, the build system will only rebuild a project when any of its member resources change. You can, however, set up references to other projects to inform the build system that changes in these referenced projects should also be taken into consideration when determining whether to rebuild a project.

To set up a reference to a dependent project in your workspace:

1. Select the project in which to declare the reference.
2. Click **Project** > **Properties**.
3. In the project properties dialog, select the **Project References** category on the left.
4. From the list shown on the right hand side of the dialog, select the project upon which this one depends.

In Figure 27, the MyProject project is set up to reference the library project named MyLibrary. This is because MyProject is actually configured to link with a library that is an output of a build configuration in the project MyLibrary. Once this project reference is defined, every time a change occurs in the project MyLibrary, the build system will not only rebuild the project MyLibrary, but it will also attempt to rebuild the project MyProject afterwards, ensuring any changes are propagated down the project dependency chain.

NOTE: This automatic rebuild of both projects only occurs when auto-build is enabled. If auto-build were turned off and the project MyLibrary rebuilt manually, the automatic rebuild of MyProject afterwards would not occur.

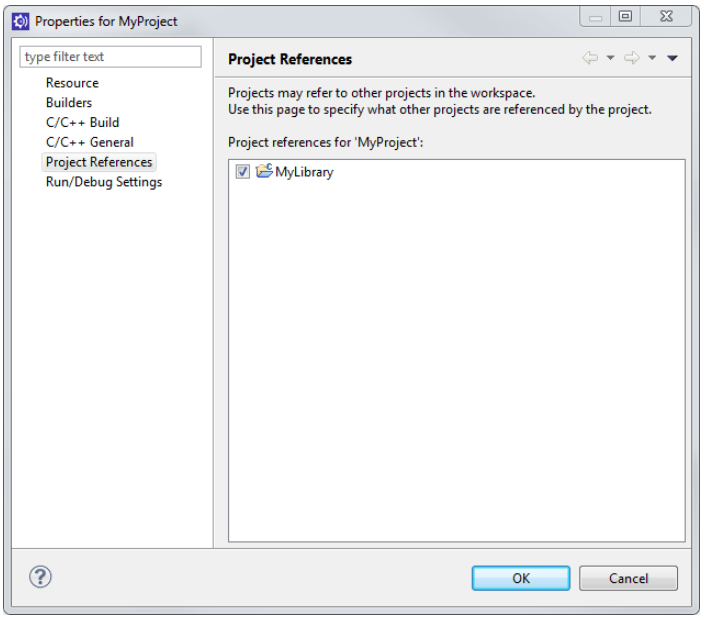


Figure 27. Project References

Note that this functionality is also provided for your convenience on the last page of the **New SK4 Project** wizard (click **Advanced...**), allowing you to create project references at the same time you create your project.

2.4.8 Viewing the Executable Program

The two outputs from building an executable project are files with the extension *.cfe* and *.elf*. The CFE File Viewer is used to view the contents of the .cfe file, and the ELF File Viewer is used to view the contents of the .elf file.

2.4.8.1 CFE File Viewer

To start the CFE File Viewer:

1. Navigate to a *.cfe* file. In the Project Explorer view, look under `your_project_name` > **Binaries**.
2. Double-click the executable file name.

The CFE File Viewer provides an **Absolute Data** tab, as well as a **Content View** tab.

2.4.8.1.1 Absolute Data Tab

The **Absolute Data** tab shows the raw data in the *.cfe* file. See the *SK4 ELF Toolchain Reference for Ezairo 7100* for an explanation of this file format.

2.4.8.1.2 Content View Tab

The **Content View** tab provides a graphical layout of all memories (on the right side) and a free memory tool (at the bottom left) as seen in Figure 28. The memory layouts show unavailable blocks, used blocks, free memory, interleaved HEAR memory areas and aliased HEAR usage in those areas.

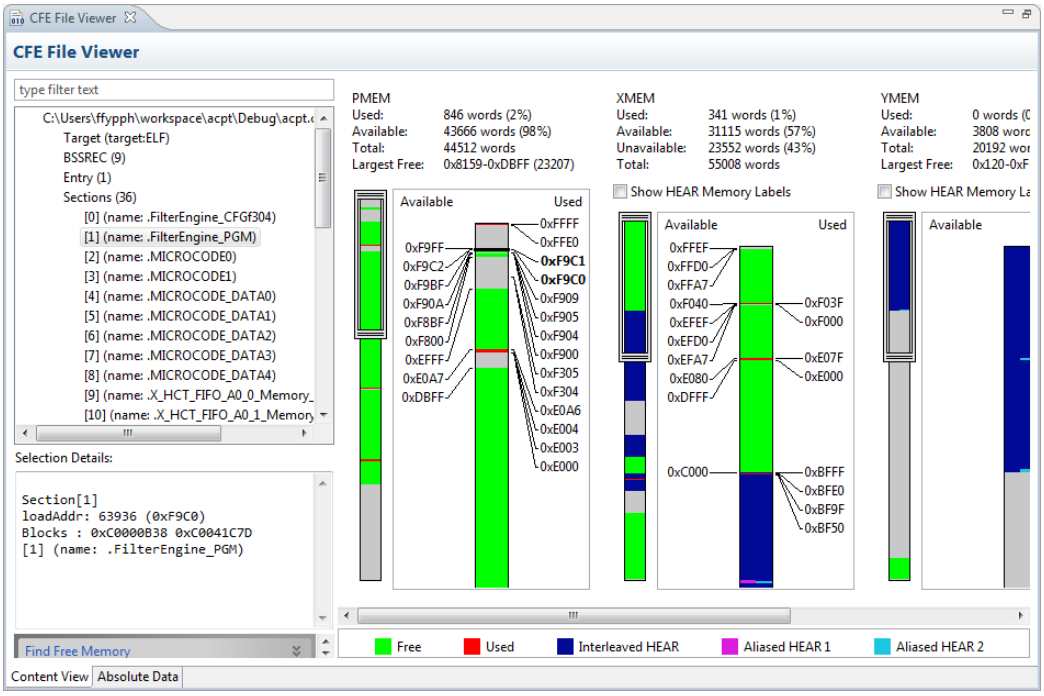


Figure 28. CFE File Viewer Content View

The contents of memory are displayed in a tree viewer (top left view) and a corresponding graphical representation (right side CFE File Viewer). In the graphical representation, each memory type is shown in two ways:

- The left column shows the entire memory with some part of it enclosed in a magnification slider. You can slide or resize the magnifier.
- The right column shows an expanded view of the memory in the magnification slider, zoomed in to the size of the magnification slider.

Hovering over a used block, aliased block or address range shows the section name and size of that block. Hovering over a free memory block (green) shows the size of that contiguous unused memory range. Clicking an address label of a used block expands the tree view to the section associated with that used block.

Clicking the **Show HEAR memory** check box provided for X and Y memory displays all HEAR memory names on the memory map and removes all other address labels.

Aliased HEAR 1 is shown to the left in the memory display in interleaved memory and corresponds to First Memory as is described in Table 17 of the Hardware Reference Manual. Aliased HEAR 2 is shown to the right and corresponds to the Second Memory.

Hovering over a used section or address label in HEAR memory space highlights the associated aliased areas on the zoomed-in display in interleaved HEAR memory space. Similarly, clicking an aliased memory block in interleaved HEAR memory space scrolls the zoomed-in display of the associated memory and highlights the used memory block in HEAR memory space.

NOTE: The recommending memory layout settings are assumed to be as described in section 7.4 of SK4 ELF Toolchain Reference for Ezairo 7100 . Specifically, XYMEM is assumed to lie in the range 0x0020 through 0x011F.

2.4.8.1.3 CFE File Viewer Free Memory Tool

This tool searches free memory for the contiguous length specified within the memory segments specified. If you search "All" of XMEM or YMEM, then the tool searches all free memory including HEAR memory, but excluding all interleaved HEAR memory space such as H01. However, you may search a specific interleaved space. If searching an interleaved HEAR memory space, the tool reports contiguous gaps that have both interleaved double words free which can accommodate "length" words.

To find free memory:

1. Expand the Find Free Memory view in the lower left corner.
2. Enter the minimum length (in decimal) of contiguous memory that you want to find.
3. Select the type of memory to search. If you select `XMEM` or `YMEM`, you can also specify which HEAR memory segment to search.
4. Click **Find**.

The start address, end address and length of the memory segment are displayed in the result. See Figure 29 for an example.

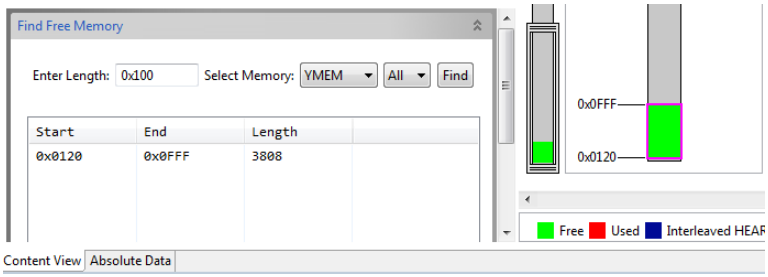


Figure 29. Finding Free Memory

2.4.8.2 ELF File Viewer

To start the ELF File Viewer:

1. Navigate to an *.elf* file. In the Project Explorer view, look under **`your_project_name`** > **Binaries**.
2. Double-click the executable file name.

The ELF File Viewer provides a tree view of available sections stored in the file, as seen in Figure 30 (similar to the CFE Content View). Symbols contained within specific sections are listed when the tree is expanded. You can search for a section, symbol or a specific memory address in the filter text box.

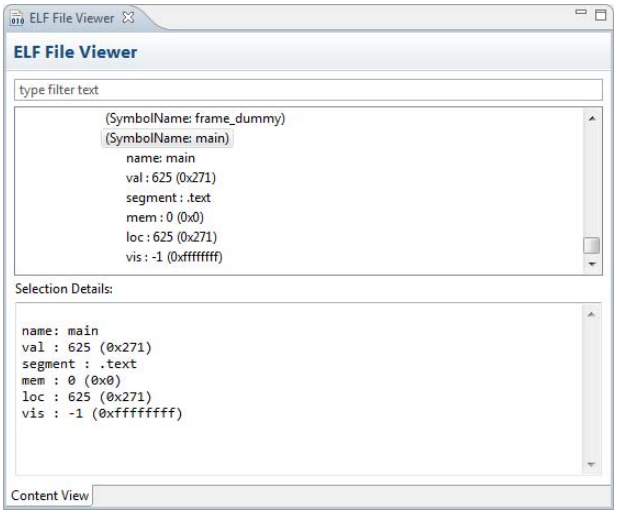


Figure 30. ELF File Viewer Content View

2.5 SETTING PREFERENCES

Many aspects of the appearance and behavior of the IDE Workbench can be modified through the **Preferences** dialog.

2.5.1 Accessing Preferences

Select **Window** > **Preferences...**to access the **Preferences** dialog shown in Figure 31 on page 41.

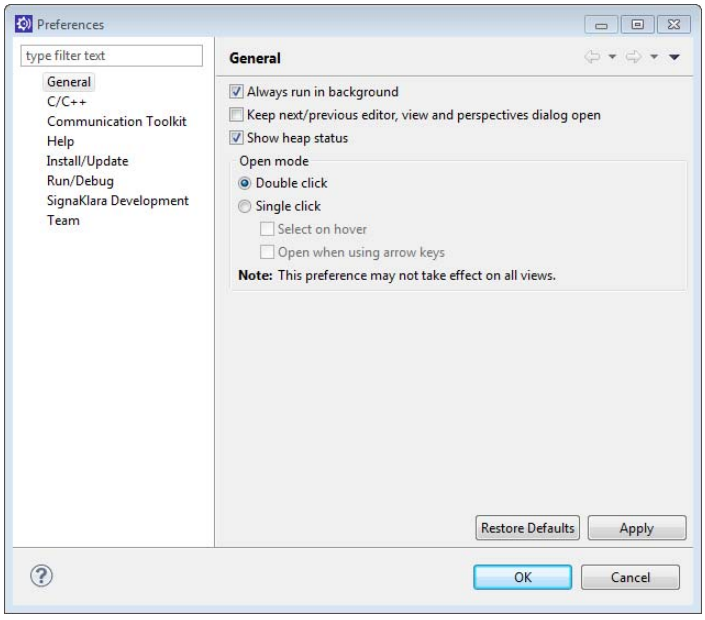


Figure 31. The Preferences Dialog

2.5.2 Organization and Categories of Preferences

The preferences settings are organized into pages which are further organized into a hierarchy that is displayed on the left side of the dialog. Select a page from this hierarchy to display the settings on that page. Some entries in the hierarchy can be expanded to show more pages of settings.

The following preferences categories are available:

General
: Allows you to change the colors for the user interface, key bindings, perspectives, startup and shutdown behavior, web browser settings and workspace settings.

C/C++
: General settings for C/C++ development, including the ARM Cortex-M3 source editor.

Communication Toolkit
: Preference settings specific to the Communication Toolkit (CTK).

Help
: Allows you to change the behavior of the Help client and server.

Install/Update
: Allows you to change the rules for software updates and enable automatic software updates.

Run/Debug
: Allows you to change the appearance and behavior of the debug perspectives. This is discussed more in Section 3.3, "Debugger Preferences" on page 69.

SignaKlara Development
: Contains settings specific to the SignaKlara Development perspective, including the appearance and behavior of the build console, Jython console and the CFX, HEAR and Filter Engine source editors.

Team
: Allows you to change the behavior of the Team Synchronization features of the IDE Workbench.

If you cannot find a setting, try entering all or part of the setting name in the **type filter text** box at the top of the **Preferences** dialog. The hierarchy will highlight the pages containing related settings.

Some of settings can be configured on a project-specific basis, rather than affecting the entire workspace. To explore and set project properties, right-click the project and click **Properties**. Several items under **C/C++ General** have options or check boxes for making them apply only to that project.

2.5.3 A Partial List of Preferences

Here are a few settings that you might want to change:

Automatic software updates
: On the **Install/Update** > **Automatic Updates** page, check **Automatically find new updates and notify me**. You can configure how frequently the IDE Workbench searches for updates.

Source editor preferences

In the pages under **SignaKlara Development** > **Editor** you can change the behavior of the CFX, HEAR and Filter Engine source editors including configuring Content Assist, editing and creating user-customizable code templates as well as configuring syntax highlighting of keywords, numbers and other elements in SK4 source code.

The same elements can be configured for the ARM Cortex-M3 source editor, in the pages under **C/C++** > **Editor**.

For more information, see Section 2.3.1, “Syntax Highlighting” on page 25.

Using an external web browser for help

First, select the external web browser to use on the **General** > **Web Browser** page. Then, still in the Preferences window, in the list on the left, select **Help**. On that page, in the **Open Help contents** list, click **In an external browser**.

2.6 Advanced Features

This section describes several advanced features of the IDE Workbench. You do not need to use these features to develop software. However, they support external tools, build systems and debuggers that you use in your development environment.

2.6.1 Launching External Tools

External tools are software applications that are not part of the IDE. You can quickly launch external tools from within the IDE Workbench, which allows you to use them without leaving the development environment. Select **Run** > **External Tools** to see a list of configured external tools.

You can add more external tools:

1. Click **Run** > **External Tools** > **External Tools Configurations...**. Select Program from the list of available tool launch configuration types, then press the New icon on the toolbar as shown in Figure 33 on page 45.
2. You will see the External Tools dialog shown in Figure 32.

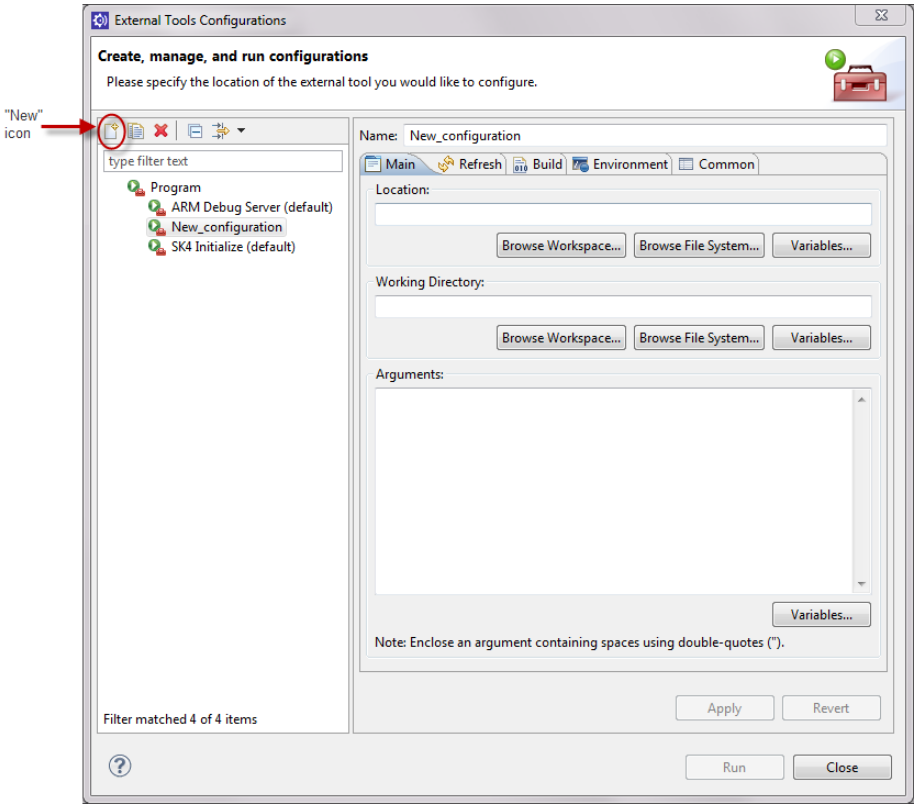


Figure 32. The External Tools Dialog

3. You should change the name of the tool to something meaningful, and you must specify the location of the tool directly or by browsing the workspace or file system.
4. Once you have created a new external tool and you have run it once, it will appear on the **External Tools** menu. You can also force an external tool to appear on the **External Tools** menu by adding it to your favorites: select **Run** > **External Tools** > **Organize Favorites...**.

User-defined external tool configurations are stored within your workspace. Therefore, if you change to a different workspace, you have to redefine any custom external tools you may have defined. You can also save the external tool definition as part of your project. In the external tool configuration dialog, click the **Common** tab > **Save as > Shared file**, and browse to a project folder in which to store the launch configuration.

2.6.2 Launching an External Builder

You can launch an external tool such as `make` to perform all or part of the build process.

1. In the Project Explorer view, select a project that you want to build with an external tool.
2. Access the project properties by selecting **Project** > **Properties**.
3. Select the **Builders** page.
4. To add a new external builder, select **New...**.
5. In the **Choose configuration type** dialog that appears, select **Program** and press **OK**. The dialog shown in Figure 33 appears.

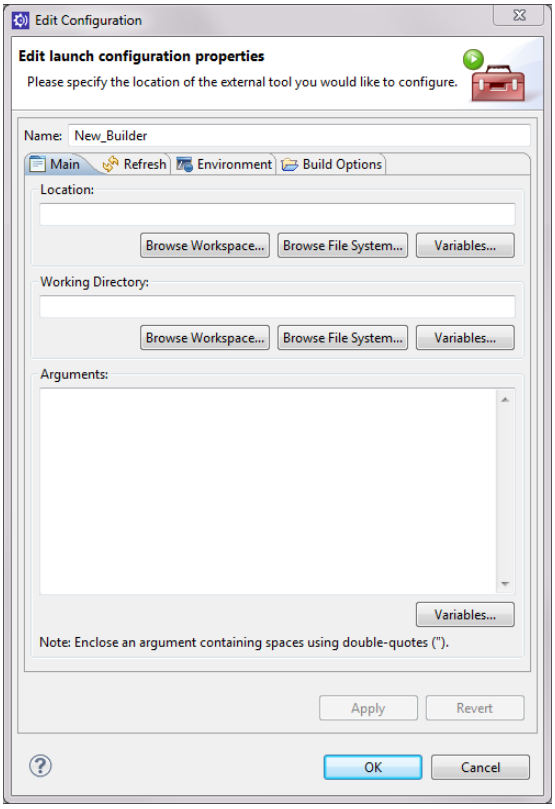


Figure 33. The External Builder Properties Dialog

6. Change the name of the builder to a meaningful value.
7. You can enter the name of the program and the working directory either directly or by browsing the workspace or file system.
8. You can specify program arguments, environment variables, and build and refresh options in the remaining pages of the dialog.
9. When you click **OK** and return to the **Builders** page, you can adjust the execution order of your external builders and the standard **Project Builder** using the **Up** and **Down** buttons. You can also enable or disable the external builders or the standard builder using the check boxes in the builder list.

For example, to use *DMAKE.EXE*[2] and a *makefile* to build your project instead of the standard builder, you would:

1. Create a new builder called “dmake”.
2. In the **Location** field, browse to the *DMAKE.EXE* executable.
3. In the **Working Directory** field, browse to the location of the *makefile*.
4. Save this builder configuration, and disable the standard builders by clearing the box beside each of them.

2. DMAKE is not included in this EDK, but is an example of an external build tool.

2.6.3 Integrating Other Eclipse Plug-Ins

There are hundreds of other Eclipse plug-ins available from various sources. For an online catalogue of available third-party plug-ins see http://marketplace.eclipse.org/. The quality and compatibility of all of these plug-ins are neither assessed nor guaranteed by ON Semiconductor. Direct any support questions for these plug-ins to their vendors.

Follow the installation instructions that come with the plug-ins. We recommend that third-party plug-ins *not* be installed in the *eclipse* folder located in the product install directory (normally *C:\Program Files\ON Semiconductor\Ezairo 7100 EDK 1*), but rather in an external *extension location*. For more information, refer to the following URLs:

- http://www-128.ibm.com/developerworks/library/os-ecl-manage/
- http://wiki.eclipse.org/FAQ_Can_I_install_plug-ins_outside_the_main_install_directory%3F

If you ignore this advice and do decide to install third-party plug-ins in the *eclipse* folder within the IDE's product installation location, be sure to keep a copy of these plug-ins as the contents of the *eclipse* directory, including any installed third-party plug-ins, will be deleted if the EDK is uninstalled.

2.6.4 Integrating Other Source Code Control Systems

To use other source code control systems, you must install a plug-in that provides support for that system. Many source code control system vendors provide Eclipse plug-ins that should work with the EDK. As with all plug-ins, their quality and compatibility with the IDE are neither assessed nor guaranteed by ON Semiconductor. Direct any support questions for these plug-ins to their vendors.

2.6.5 Update Site and Update Notifications

The Update Site is a web site that only our customers can access, and where updates to the EDK software are posted. You can either check periodically for updates or set your preferences to notify you when an update is available (as described in Section 2.5.3, "A Partial List of Preferences" on page 42).

1. Ensure that you have a working Internet connection and that you have Administrator privileges.
2. Start the IDE by going to the Windows **Start** menu and clicking **ON Semiconductor > EDK 1 Ezairo 7100 IDE**. If you are working on Windows Vista or Windows 7, right-click on **EDK 1 Ezairo 7100 IDE** and click **Run as administrator**.
3. Choose **Help > Check for Updates** menu option.
4. You are prompted for a user name and password. These have been provided by ON Semiconductor for your customer installation. Enter them and you see a list of new features to install.
5. If there are no new updates, you will see a message informing you of this situation. If there are updates for the EDK, select the features that you would like and click through the update wizard to confirm and install the updates.

CHAPTER 3
Debugging in the Integrated Development Environment

3.1 INTRODUCTION

This chapter is about using the IDE's integrated debuggers to run your program. If your program does not work correctly, the debuggers have functions that allow you to inspect it by stepping through instructions, and other functions to find and eliminate the bugs.

When a debug session is launched, the IDE switches to the Debug Perspective, a collection of views that are useful for debugging. These views display memory, registers, breakpoints and source code, among other things. They also control whether the device being debugged is running, suspended or stepping.

See Section 3.2, "Using the Debugger" on page 54 for instructions on launching a debug session.

3.1.1 Default Layout

When you first open the Debug Perspective, you see a set of views similar to those shown in Figure 34.

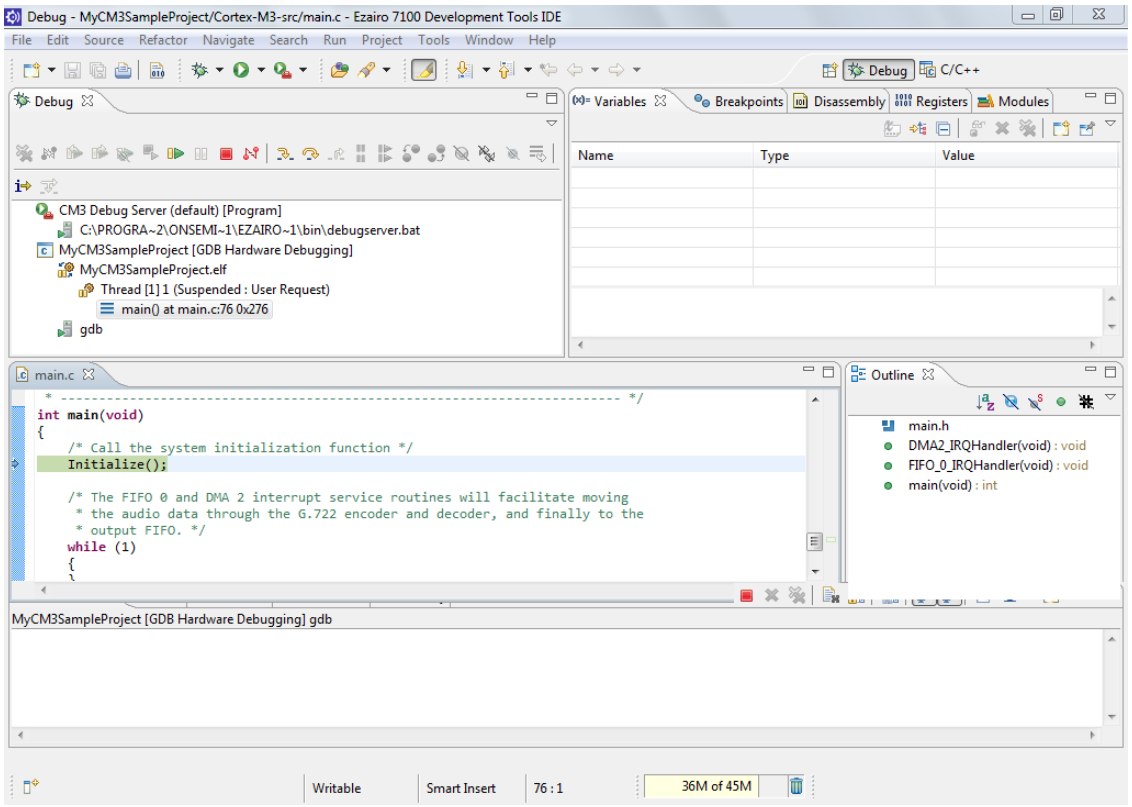


Figure 34. Default Debug Perspective

3.1.2 Views

The Debug Perspective consists of several views (and an editor) that are useful for debugging. These include:

Debug View	Run, suspend or step the target processor
Source Editor	Follow execution of program and toggle breakpoints
Disassembly View	View memory as assembly instructions
Registers View	View and modify registers
Memory View	View and modify memory
Expressions View	Evaluate expressions
Breakpoints View	Enable, disable or remove breakpoints, and configure external breakpoints
Debug Console View	Interact directly with the debugger process

To open a view manually, use **Window > Show View** and select the view. Once a view is opened, it remains in the perspective unless you manually close it.

3.1.2.1 Debug View

The Debug view allows you to manage the debugging or running of a program in the Workbench. It displays each debugging session as a hierarchy of five debug elements, all of which can be seen in Figure 35:

Launch	Identifies the debug session with the name of the launch that is being debugged.
Target	The device being debugged.
Main Thread	The single thread of execution. As shown in the figure below, this element is always called **Main thread**.
Stack Frame	The debug context. The figure below shows an example of this element as the value of the program counter **0x8000{pmem}**, the path to the source code file **..\app.cfs** relative to the *.cfe* file, and the line number. This is only visible when the program is suspended. There is only one stack frame element for assembly programs.
Debugger Process	The command interpreter that is available through the console view. In the figure below, it is called **Debugger (DDB)**. The rest of the information is about the communication connection.

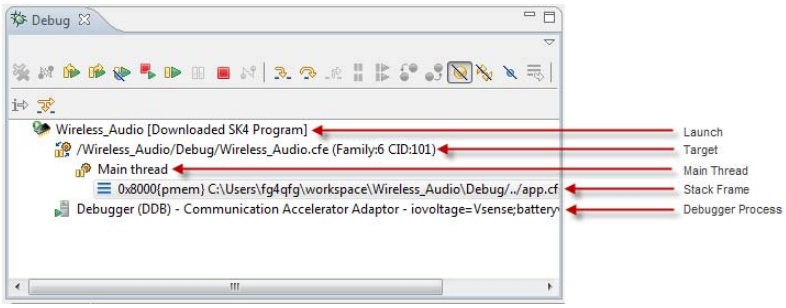


Figure 35. Debug View

3.1.2.2 Source Editors

In the Debug Perspective, the same source editor (either the SK4 Source Code Editor or the C/C++ Editor) that you used in the C/C++ perspective becomes a window into the execution of your program, showing the current source line as you step through it. The source code line corresponding to the current value of the program counter is shown by both a blue arrow in the left-hand gray column, and highlighted source code, as you can see in Figure 36.

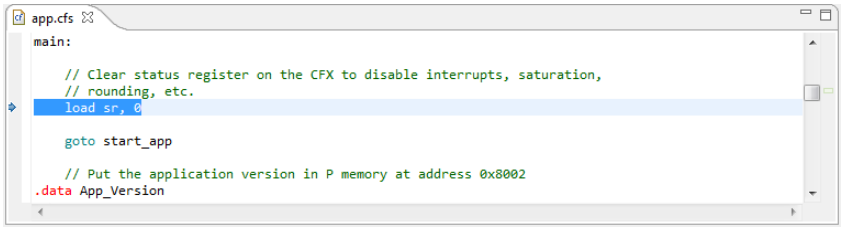


Figure 36. Source Editor

Breakpoints can be placed in your program by double clicking on the margin on the left hand side of the editor window. For more information about using breakpoints in the editor, see Section 2.3.2, “Breakpoint Manipulation” on page 26.

3.1.2.3 Disassembly View

In addition to the source editor, you can also follow the execution of your program in the Disassembly view, which provides a scrollable view of all of program memory as assembly instructions.

The layout of the Disassembly view differs when debugging a CFX or ARM Cortex-M3 core application. The Disassembly views for the two cores also have different icons, as an aid to distinguish them.

3.1.2.3.1 CFX

The CFX Disassembly view is shown in Figure 37. The right-most column displays the number of cycles each disassembled instruction would take to execute. You might have to adjust the width of this view to see this column.

The icon for the CFX Disassembly view is a representation of binary digits.

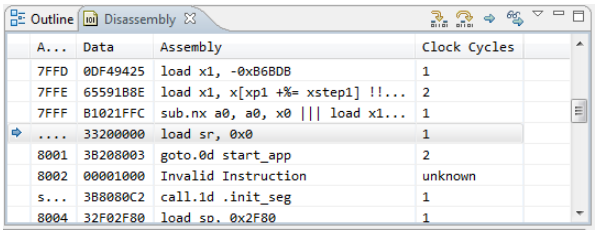


Figure 37. CFX Disassembly View

If the program counter indicator has scrolled out of view, you can make it appear by right-clicking in the window, and clicking **Show Current**, or clicking the blue arrow on the tool bar. You can also jump to any address with the **View the disassembly at the specified address** toolbar item.

The toolbar of the Disassembly view also contains buttons (with curved yellow arrows) that allow you to **step one instruction** or **step over one instruction**.

As in the source editor, breakpoints can be placed on a particular address by double-clicking on the left-most column.

3.1.2.3.2 ARM Cortex-M3 Core

The ARM Cortex-M3 core Disassembly view, shown in Figure 38, contains two columns. The instruction address is on the left, and the disassembled instruction is on the right.

To make the ARM Cortex-M3 core Disassembly view appear, click **Window > Show View > Disassembly**. The icon for the ARM Cortex-M3 core Disassembly view is a group of three horizontal lines, solid on the bottom with the higher ones more broken up.

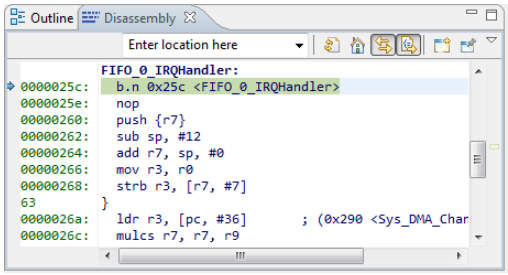


Figure 38. ARM Cortex-M3 Core Disassembly View

If the program counter indicator has scrolled out of view, you can make it appear by clicking **Go to Current Program Counter (Home)** on the tool bar. You can also jump to any address by typing it in the **Enter location here** text box in the toolbar.

Clicking **Show Source** in the toolbar jumps to the ARM Cortex-M3 core source for the currently disassembled instruction.

3.1.2.4 Registers View

The values of registers can be viewed and modified through the Registers view, shown in Figure 39. DSP core registers, digital control and configuration registers, and analog control registers are supported. The bottom half of the view shows the selected register value in the active numeric format: decimal, hex, octal, binary or fractional. A register can be added to the Expressions view by right-clicking it and clicking **Watch**. If a register value changes, it is highlighted in yellow. If a register contains bit fields, you can expand the register to show each field.

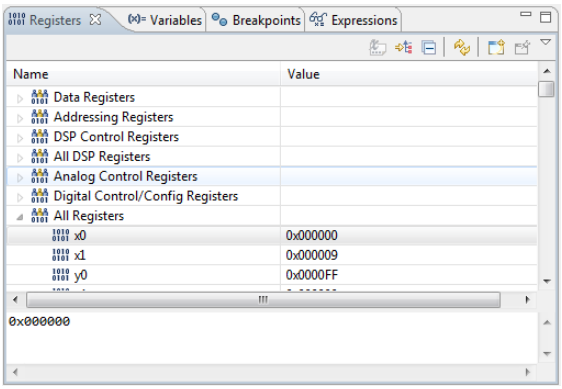


Figure 39. Registers View

For the ARM Cortex-M3 core, the Registers view provides the values of accumulators and data registers.

For convenience, different views of the registers are available for the CFX core:

Addressing Registers
: The memory pointer registers

DSP Control Registers
: Registers that control program flow on the DSP

All DSP Registers
: The DSP core registers, both data and program control

Analog Control Registers
: Only the analog block control registers

Digital Control/Config Registers
: Memory-mapped digital control and configuration registers

All Registers
: All of the registers

3.1.2.5 Memory View

Memory views (like the one shown in Figure 40) can focus on any address. Note that a memory view is associated with CFX debugging, and a separate memory view is associated with ARM Cortex-M3 core debugging. If you are debugging the two cores in the same debug session, you can open the memory view associated with the currently highlighted debugging thread.

You can use symbolic names, register names or other expressions, as well as fixed address values. You can view and modify values in memory with the memory view, which can render values in hexadecimal, unsigned integer, signed integer, ASCII and signed fractional. With the CFX core, you can also specify any memory space.

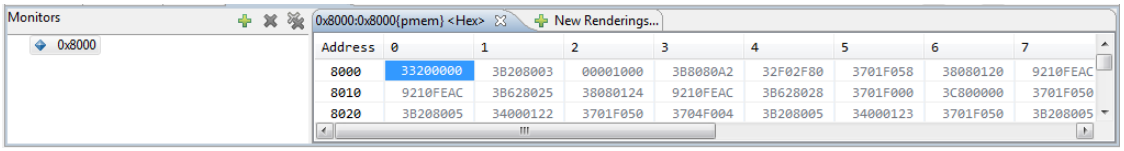


Figure 40. Memory View

3.1.2.6 Expressions View

In the Expressions view, you can compose expressions for evaluation whenever execution suspends. For example, suppose you are interested in the value pointed to by a register called xp1. Although you could watch the pointer register value in the registers view and look up the corresponding memory address in the memory view, it would be more work and take more time than using the Expressions view, where all you have to do is add `*xp1` to the expression list as shown in Figure 41.

To add an expression:

1. Click the icon to create a new watch expression as shown in Figure 41.
2. Type an expression using the language described in Section 3.6, "Expression Language" on page 91.
3. Click **OK**. (Pressing Enter inserts a new line.)

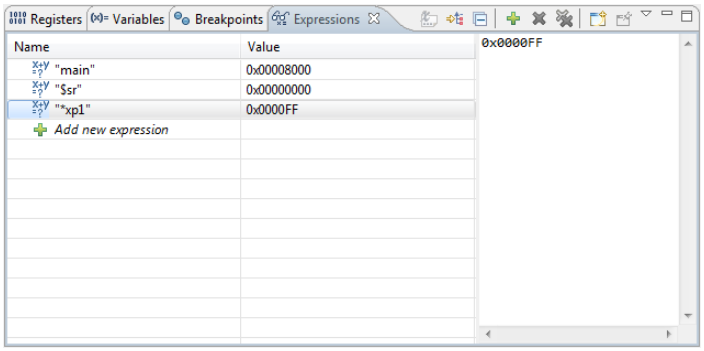


Figure 41. Expressions View

If you add an expression that evaluates a register that contains bit fields, you can expand the result to show each field.

3.1.2.7 Breakpoints View

Breakpoints make it possible to suspend the execution of a program at a specific address or line of source code. Breakpoints are set in the source editor or Disassembly view. External breakpoints make it possible to suspend the execution of a program when a particular event occurs that can be detected by the external breakpoint feature of the DSP. Both breakpoints and external breakpoints are controlled from the Breakpoints view as shown in Figure 42.

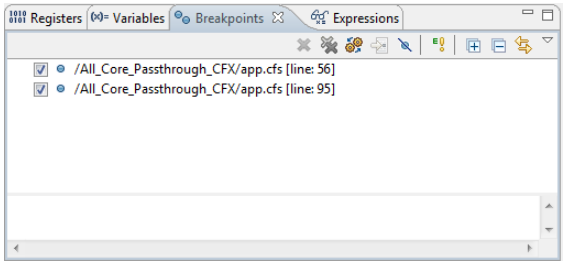


Figure 42. Breakpoints View

The Breakpoints view lists all breakpoints set in the workspace:

- Source-line breakpoints list the project, resource and line number associated with the breakpoint.
- Disassembly breakpoints list the address.
- External breakpoints list the external breakpoint number and name of the debug configuration associated with the breakpoint.

Breakpoints with a check are enabled; click the check box to toggle them from enabled to disabled. From the Breakpoints view, you can disable or enable particular breakpoints, remove some (or all) breakpoints, skip all breakpoints, or configure external breakpoints.

3.1.2.8 Debug Consoles

In addition to the many interactive graphical components provided in the IDE, interactive command interpreters can be used to issue commands directly to the underlying debugger processes. The interpreters are accessed through console windows that appear when debug sessions are launched (see Figure 43 for an example).

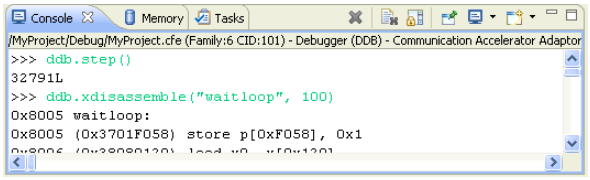


Figure 43. Debug Console

3.1.2.8.1 CFX Debug Console

Everything that you can do from the IDE—stepping, running, setting breakpoints, etc.—can also be done through the CFX debug console. Moreover, the command interpreter is not limited to issuing single commands.

The command interpreter is actually a Jython interpreter. With Jython, a Java-based implementation of the popular Python programming language, you can compose complex scripts that automate any regular debugging task that you encounter.

The command interpreter is discussed in more detail in Section 3.4, “Jython Command Console” on page 85.

3.1.2.8.2 ARM Cortex-M3 Core Debug Console

In addition to the commands available in the standard GDB console, the ARM Cortex-M3 core debug console supports some commands specific to the ARM Cortex-M3 core. Interrupt control, hardware breakpoints, and

watchpoints for the ARM Cortex-M3 core can be managed using commands entered directly in the debug console. Information about these can be obtained with query commands in the GDB console as well.

For more information, see Section 3.5, "ARM Cortex-M3 Core Command Console" on page 89.

3.2 Using the Debugger

The CFX and ARM Cortex-M3 cores on the Ezairo 7100 system support remote debugging from a Windows workstation. When you debug an application, it is downloaded to the device, resumes running, and remains connected. Alternatively, you can simply run an application on the device, which means that it is downloaded, resumes running, and disconnects.

This section describes how to configure and launch debugging sessions.

3.2.1 General Configuration

3.2.1.1 CTK Configuration

You need two CTK configurations: one for the CFX and one for the ARM Cortex-M3 core. Two configurations, called CFX_CFG and CM3_CFG, are typically installed with your EDK software and are designed to work with the cheat sheets and default external tools.

If you are re-installing the EDK, the configuration file already exists and is not replaced. To restore the CFX_CFG and CM3_CFG configurations, we recommend renaming the *config.ini* file where the configurations are stored (for example, *C:/Program Files (x86)/Common Files/SignaKlara/CTK* on Windows 7), and installing the CTK again using **`EDK_HOME`***/ctk/redistribution/ctk/CTKRuntime.msi*, where **`EDK_HOME`** is the folder where you installed the Ezairo 7100 EDK.

If the CFX_CFG and CM3_CFG configurations are somehow deleted, here is how to recreate them:

1. Open the CTK Configuration Manager from the Windows **Start** menu > **ON Semiconductor**.
2. Click **Add** to create a new CTK configuration.
3. Enter a name for the new configuration and click **OK**.
4. Select **Communication Accelerator Adaptor** as the Interface.
5. Select **I2C** as the Protocol.
6. Change the **iovoltage** to `Vsense`.
7. Change the **batteryvoltage** to `1.8V`.

For CFX_CFG:

1. Make sure the address under **Protocol Settings** is `96`.
2. Leave the rest of the options at their default settings.

For CM3_CFG:

1. Change the address under **Protocol Settings** to `64`.
2. Leave the rest of the options at their default settings.

3.2.1.2 SK4 Initialize Tool

Unless the ARM Cortex-M3 processor has already been started (by a CFX program, for example), you must run the SK4 Initialize tool before starting an ARM Cortex-M3 processor debug session. The SK4 Initialize tool loads a small

program onto the CFX to prevent it from continuously resetting the board. It also writes `CM3_RESUME` (`0x10`) into the `SYSTEM_CTRL_2` register, allowing the ARM Cortex-M3 processor to run. The easiest way to run the SK4 Initialize tool is to set up an external tools configuration in the IDE. This configuration is included by default in the IDE, under the name SK4 Initialize, however instructions on how to recreate it manually are:

1. Open **External Tools Configurations**:
   a. Click **Run > External Tools > External Tools Configurations**.
   b. Create a new program launch configuration.
   c. Enter `SK4 Initialize` in the **Name** field.

2. In the **Main** tab:
   a. Enter `${env_var:SK4_TOOLS}\bin\np5init.exe` for the **Location**.
   b. Add this argument: `--cfx-config=`**`CFX_CFG`**

3. In the **Build** tab:
   • Clear the **Build before launch** check box.

3.2.1.3 ARM Debug Server

The ARM Cortex-M3 Core Debug Server (henceforth referred to as the ARM Debug Server) can be run via the command line, or in the IDE using an external tools configuration. You must specify the name of a CTK configuration to use for communicating with the ARM Cortex-M3 core, and a TCP port number to use. A default configuration named **ARM Debug Server (Default)** is included when you install the IDE. This default configuration assumes the name of the CTK configuration to be `CM3_CFG` and the port number to be `5000`. You may modify the configuration or create new ones. (See Section 2.6.1, “Launching External Tools” on page 43.) Below are instructions on how to create an ARM Debug Server configuration manually:

1. Open **External Tools Configurations**:
   a. Click **Run > External Tools > External Tools Configurations**.
   b. Create a new debug configuration.
   c. Enter a name such as `ARM Debug Server` in the **Name** field.

2. In the **Main** tab:
   a. Enter `${env_var:SK4_TOOLS}\bin\debugserver.bat` for the **Location**.
   b. Add these arguments: `--core=arm --config=CM3_CFG 5000`

3. In the **Build** tab:
   • Clear the **Build before launch** check box.

3.2.2 Debugging CFX Programs

Before you can begin debugging a CFX program, you must create a debug configuration and launch a debug session. This section shows you how.

IMPORTANT: Before attempting to launch a debug session, ensure that a development board is connected to your PC and powered on (refer to the appropriate Evaluation and Development Board manual for more information).

When you build a project, it generates an executable file with the extension *.cfe*. This file contains the machine code and debug information for the program. By default, it is created in the project Debug folder and is named after the project. If you need more information about building a project, see Section 2.4, “Managed Build System” on page 33.

3.2.2.1 Creating a CFX Debug Configuration and Launching a CFX Debug Session

To create a debug configuration for a new program:

1. Locate the executable file (*Debug* folder, **.cfe*) in the Project Explorer view (in the C/C++ Development perspective).
2. Right-click the executable file to display the context menu. Click **Debug As** > **Downloaded SK4 Program**.
3. Since this is the first time you are debugging this executable, you are prompted for a communication configuration. You should already have configurations called CFX_CFG and CM3_CFG for the CFX and ARM Cortex-M3 core respectively. (If you do not, see Section 3.2.1.1, "CTK Configuration" on page 54.) Select a configuration and click **OK**.

Alternatively, you can manually create a debug configuration. Select the executable file and click the Debug arrow on the main toolbar (see the insect icon in Figure 44). Click **Debug Configurations...**. and in that dialog box, click **Downloaded SK4 Program** from the list of configuration types on the left.

1. Click the "New debug configuration" icon, above the list of configuration types on the left.
2. In the **Name** field, replace the default text with a more meaningful name for the debug configuration.
3. On the configuration **Download** tab, add the executable files to download when launching a debug session by clicking **Add...**.
4. Click **Apply**.
5. On the **Connection** tab, be sure to select the correct connection configuration, and click **Apply**. If you want to create a different communication configuration, add it now (refer to Chapter 4, "The Communications Toolkit (CTK) Configuration Manager" on page 95 for details about creating a communication configuration).
6. Now you can launch a debug session by clicking **Debug**.

With either of the above procedures, the IDE automatically switches to the Debug Perspective, showing the newly created debug session in the Debug view. The program has been downloaded to the processor and is running.

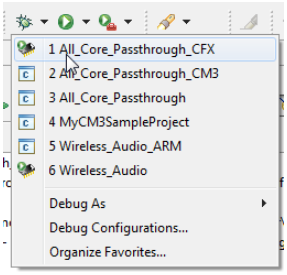


Figure 44. Menu for Downloading and Running an Application Using the Integrated Debugger

After the session has been launched once, it will show up in the Debug shortcut menu and can be launched from there. If you used the quick method of creating a debug configuration, the configuration name is the same as the executable file name by default. A debug session can also be launched by clicking **Debug As > Downloaded SK4 Program**.

A debug configuration contains all of the settings necessary to relaunch a debug session exactly as it was the first time. More information about debug configurations is in Section 3.3.1, "CFX Debug Configurations" on page 70.

3.2.2.2 Running Without Debugging

The IDE can download and run a program without launching a debug session, leaving the communication lines free for other PC programs that communicate with the target processor.

The process is similar to launching a debug session (see Section 3.2.2.1, “Creating a CFX Debug Configuration and Launching a CFX Debug Session” on page 56), but instead of using the **Debug As** > **Debug Configurations...** context menu item after right-clicking the *.cfe* file, you use the **Run As > Run Configurations...** menu item, as shown in Figure 45.

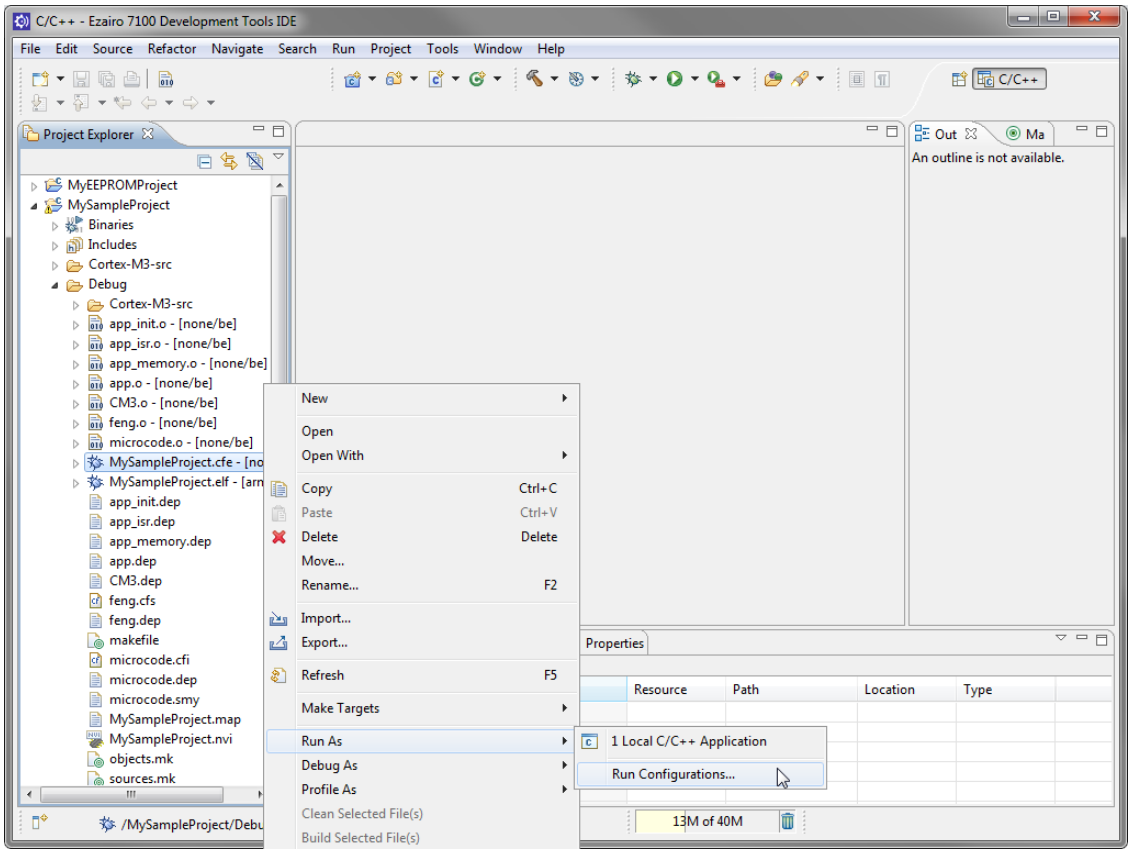


Figure 45. Menu for Downloading and Running an Application Using the Integrated Debugger

Similarly, to run the program a second time, look up the launch configuration under the “Run” icon instead of the “Debug” one as shown in Figure 46.

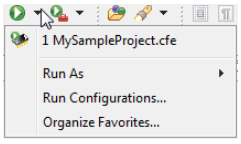


Figure 46. Launch Configuration List for Running

You can use the debug configurations and run configurations interchangeably after they are defined. After the first time you run them, they appear in both shortcut drop-down lists with the "Debug" icon and the "Run" icon.

3.2.2.3 Connecting To and Debugging a Running Program

You can create a debug session that connects to a program that is already running on the processor; that is, the debug session does not load a program when it is launched. Do the following:

1. Open the launch configuration dialog by selecting the **Run** > **Debug Configurations...** menu item in the main menu. The dialog is shown in Figure 47.

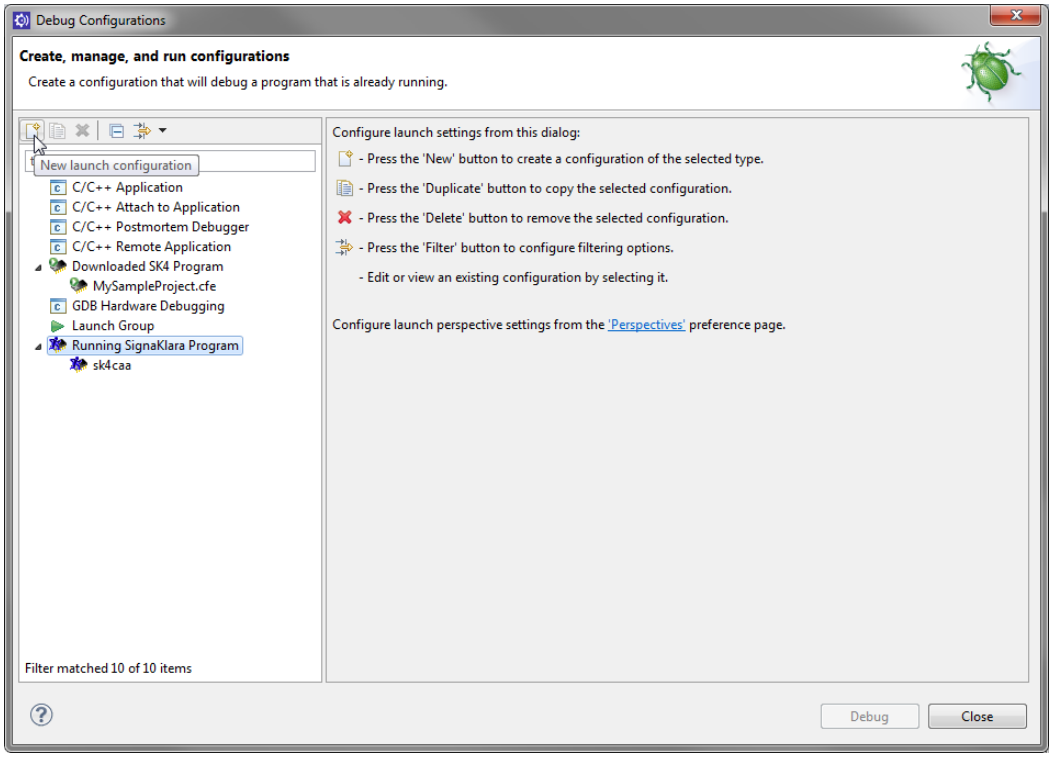


Figure 47. Launch Configuration Dialog

2. Select the **Running SignaKlara Program** item, then:
   a. Click the **New** button, as shown in Figure 47. This creates a new launch configuration.
   b. Give it a meaningful name.
   c. Click **Apply**.

3. On the **Connection** tab, select or define a communication configuration to use with the chip. For details about creating a communication configuration, refer to Chapter 4, "The Communications Toolkit (CTK) Configuration Manager" on page 95. If you click **Debug** at this point, the debug session skips loading any symbolic information, allowing you to debug without associated symbols or source code. If you want to load symbolic information and link the debug session to the source code, continue to the next step.
4. If you know what program is already loaded on the processor, you can specify its *.cfe* file on the **Import Symbols** tab. This imports the debug information in the *.cfe* file into the debug session so that label names and other symbols can be used in expressions. The program data, however, is not loaded to the processor. Furthermore, source-level debugging is available if the source code associated with the *.cfe* file is present. Note

that in order for source-level debugging and symbol resolution to work properly, the *.cfe* file specified must match what is running on the device exactly. To specify the *.cfe* file:

a. Add the executable files to download when launching a debug session by clicking **Add...** and following the prompts.
b. Click **Apply**.
c. Launch a debug session by clicking **Debug**. The IDE automatically switches to the Debug Perspective, showing the newly created debug session in the Debug view.

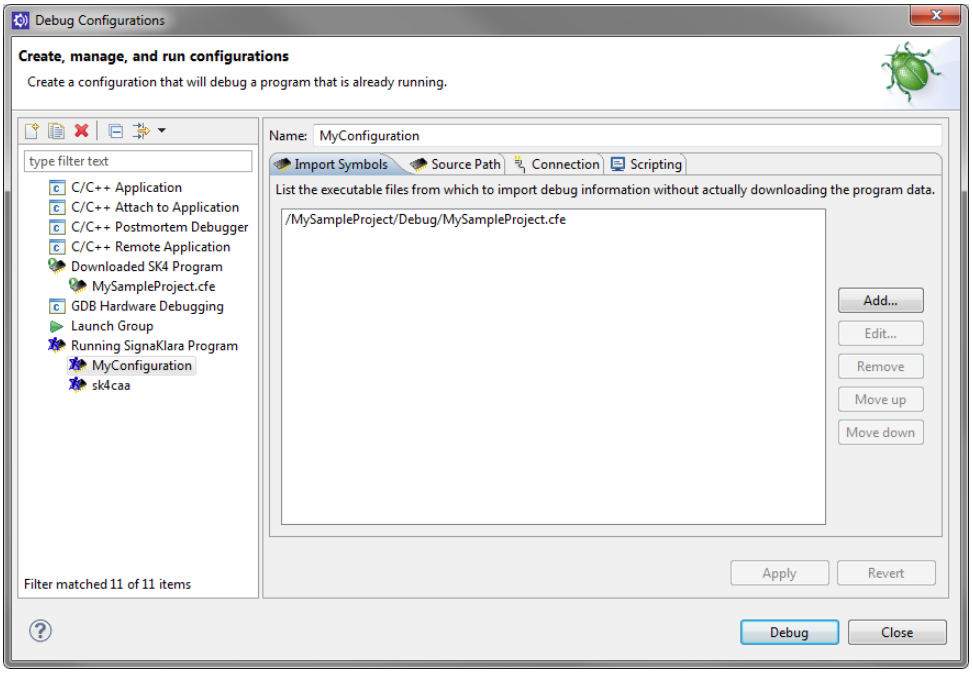


Figure 48. Launch Configuration Details

Refer to Section 3.3.1, “CFX Debug Configurations” on page 70 for information on the additional parameters that can be set in the launch configuration.

3.2.3 Debugging Filter Engine Configurations

The Filter Engine core on the Ezairo 7100 system does not have its own debug port.

Still, single-step and pause features are available through the `D_FENG_CTRL` register. For further information about Filter Engine control, see the *Hardware Reference Manual for Ezairo 7100*.

To debug a Filter Engine configuration via shared memory (mapped to 0xF800-0xF9FF in P memory) in a CFX debug session:

1. Ensure that the Filter Engine is disabled by modifying the `D_FENG_CTRL` register. The CFX debugger does not disable the Filter Engine when it halts the CFX; you must disable it manually. Failing to disable the Filter Engine before viewing its memory will likely cause a memory fault in the Filter Engine.
2. View and modify the Filter Engine memory in the Memory View in the IDE.

3.2.4 Debugging ARM Cortex-M3 Core Programs

The ARM Cortex-M3 core debugger is the GNU Debugger (GDB). It is launched and controlled in the IDE via components provided by the Eclipse CDT project. It relies on a separate server program, the ARM Debug Server, to communicate with a target device, which you must start before launching a debug session. This section gives instructions for creating a new debug configuration for GDB, and using it to initiate a debug session.

For information about the ARM Debug Server external tool, refer to Section 3.2.1.3, "ARM Debug Server" on page 55.

IMPORTANT: Before attempting to launch a debug session, ensure that a development board is connected to your PC and powered on (refer to the appropriate Evaluation and Development Board manual for more information).

3.2.4.1 Prerequisites for Launching an ARM Cortex-M3 Core Debug Session

The GDB requires a debug server program to coordinate communication with the target Ezairo 7100 device. You must run the debug server program before you launch a debug session.

While you are debugging the ARM Cortex-M3 core, you must ensure that:

1. The ARM Cortex-M3 core is running (also referred to as *started* or *resumed*) so it can properly respond to requests from the debugger.
2. The CFX core does not reset the Ezairo 7100 system, either directly or through a reset of the Ezairo 7100 system, as this causes the ARM Cortex-M3 core to switch back to the stopped (or *paused*) state.

The simplest way to satisfy these requirements is to run the Ezairo 7100 Initialization Utility, SK4 Initialize, which resumes the ARM Cortex-M3 processor and loads a small program to the CFX that regularly refreshes the watchdog timer (see Section 3.2.1.2, "SK4 Initialize Tool" on page 54 for details).

Another way is to first load a working program of your own to the CFX via the CFX debugger (see Section 3.2.2, "Debugging CFX Programs" on page 55 for details). You must wait until the CFX program has started or resumed the ARM Cortex-M3 core before loading your ARM Cortex-M3 core program.

If you are debugging an application where the CFX and the ARM Cortex-M3 core programs are tightly coupled and communicate regularly via system interrupts, we recommend that you:

1. Build the ARM Cortex-M3 core program image into your CFX program, to ensure that it is loaded to ARM Cortex-M3 core memory before the CFX program enables the ARM Cortex-M3 core.
2. Halt the CFX while you launch your ARM Cortex-M3 core debug session, ensuring that no interrupts are triggered while the ARM Cortex-M3 core program loads.

3.2.4.2 Creating an ARM Cortex-M3 Core Debug Configuration and Launching a Debug Session

Create a debug configuration for a new program as follows:

1. Locate the executable file (*Binaries* folder, **.elf*) in the Project Explorer view (in the C/C++ Development perspective).
2. Right-click the executable file to display the context menu. Click **Debug As** > **Downloaded ARM Cortex-M3 Core Program (GDB Hardware Debugging)**.

3. You are asked if a launch group should be created. Click **Yes** unless this project does not contain a CFX executable program.
4. You are asked if the existing GDB server instance should be terminated. Click **Yes**.

NOTE: If this is the first time you are launching an ARM Cortex-M3 core debug session in this workspace, you are prompted to select a default launcher. See Section 3.2.4.5, “Selecting a Default Launcher” on page 62 for more information. You will not have to repeat that procedure for this workspace again.

The IDE automatically creates a launch group for this executable (its name is the same as the name of the executable file), and switches to the Debug Perspective, showing the newly created debug session in the Debug view. The program has been downloaded to the processor and is running.

3.2.4.3 Manually Creating an ARM Cortex-M3 Core Debug Configuration and Launching a Debug Session

This procedure is the manual version of the one described in Section 3.2.4.2, “Creating an ARM Cortex-M3 Core Debug Configuration and Launching a Debug Session” on page 60.

To create a new debug configuration:

1. Right-click the project that you would like to debug in the **Project Explorer** view.
2. Click **Debug As > Debug Configurations...**
3. In the **Debug Configurations** dialog, create a new **GDB Hardware Debugging** debug configuration by clicking the “New launch configuration” icon.
4. Optionally, change the default name for the debug configuration in the **Name** field.

In the **Main** tab:

1. If your project name is not already there, choose it by using the **Browse...** button.
2. In the **C/C++ Application** field, select the project *.elf* file using the **Search Project...** button if it is not already there.

In the **Debugger** tab:

1. Change **GDB command** from `gdb` to `arm-none-eabi-gdb`.
2. Ensure that the **Port number** is the TCP port number to use for communication. It must be the same as the port number in the ARM Debug Server external tool configuration, which is 5000 by default.

In the **Startup** tab:

1. Clear the first two check boxes: **Reset and Delay** and **Halt**.
2. Leave the following selected: **Load image**, **Use project binary**, and **Load symbols**.
3. Under **Runtime Options**, select the **Resume** check box. This runs the program when you launch the debug session. (By default, when a debug session starts, the core is stopped and the program counter is set to the program’s starting point.)
4. Click **Apply** and **Close**.

To launch a debug session:

1. Ensure that you have a non-resetting program loaded to the CFX core, as described in Section 3.2.4.1, “Prerequisites for Launching an ARM Cortex-M3 Core Debug Session” on page 60.

2. Launch the debug server by clicking your ARM Debug Server configuration in **Run > External Tools**.
3. Open the **Debug Configurations** dialog by clicking **Run > Debug Configurations...**.
4. To launch the debug session, select your debug configuration in the tree on the left hand side and click **Debug**.

3.2.4.4 Manually Creating an ARM Cortex-M3 Core Launch Group

Instead of getting an automatically created Launch Group as described in Section 3.2.4.2, "Creating an ARM Cortex-M3 Core Debug Configuration and Launching a Debug Session" on page 60, you can create a Launch Group to automatically run SK4 Initialize, the ARM Debug Server, and the ARM Cortex-M3 core debug configuration in one step. The steps to create this launch group are:

1. In the **Project Explorer** view, right-click the project that you would like to debug.
2. Click **Debug As > Debug Configurations...**
3. Create a new **Launch Group** configuration.
4. Specify a name for the Launch Group in the **Name** field.
5. In the **Launches** tab, click **Add...**. The **Add Launch Configuration** window appears.
6. In the **Launch Mode** drop-down box select **run**. Then select **Program > SK4 Initialize**. Click **OK**.
7. In the **Launches tab**, click **Add...** again.
8. In the **Launch Mode** drop-down box, select **run**. Then select **Program > ARM Debug Server**.
9. In the **Post launch action** drop-down box, select **Delay** and enter 3 seconds. This value can be adjusted based on the host system. Click **OK**.
10. In the **Launches tab**, click **Add...** again.
11. In the **Launch Mode** drop-down box, select **debug**. Then select the ARM Cortex-M3 core debug configuration for the project you wish to debug (under GDB Hardware Debugging). Click **OK**.
12. Click **Apply**.

With this Launch Group, you do not have to meet the prerequisite of running the GDB debug server program as described in Section 3.2.4.1, "Prerequisites for Launching an ARM Cortex-M3 Core Debug Session" on page 60.

3.2.4.5 Selecting a Default Launcher

If you are debugging an ARM Cortex-M3 core executable for the first time in a new workspace, you have to specify a default launcher. After the procedure in Section 3.2.4.2, "Creating an ARM Cortex-M3 Core Debug Configuration and Launching a Debug Session" on page 60, you are prompted to select a default launcher as shown in Figure 49.

1. Click **Change Workspace Settings**. The dialog shown in Figure 50 appears.
2. Under **Launch Type/Mode**, select **[Debug]** under **GDB Hardware Debugging**.
3. Under **Preferred Launcher**, select **GDB (DSF) Hardware Debugging Launcher**.
4. Click **Apply**, then **OK** to the two dialogs to complete the setting.

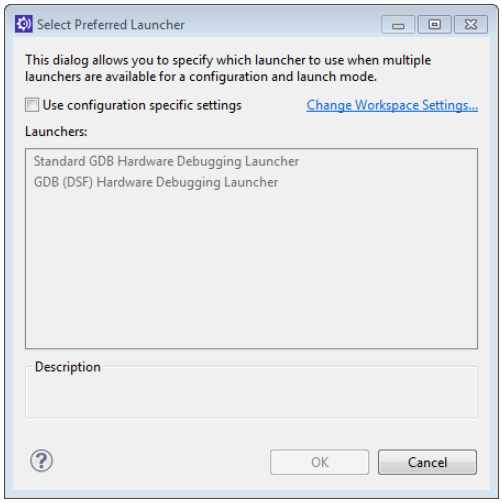


Figure 49. Prompt to Select a Default Launcher

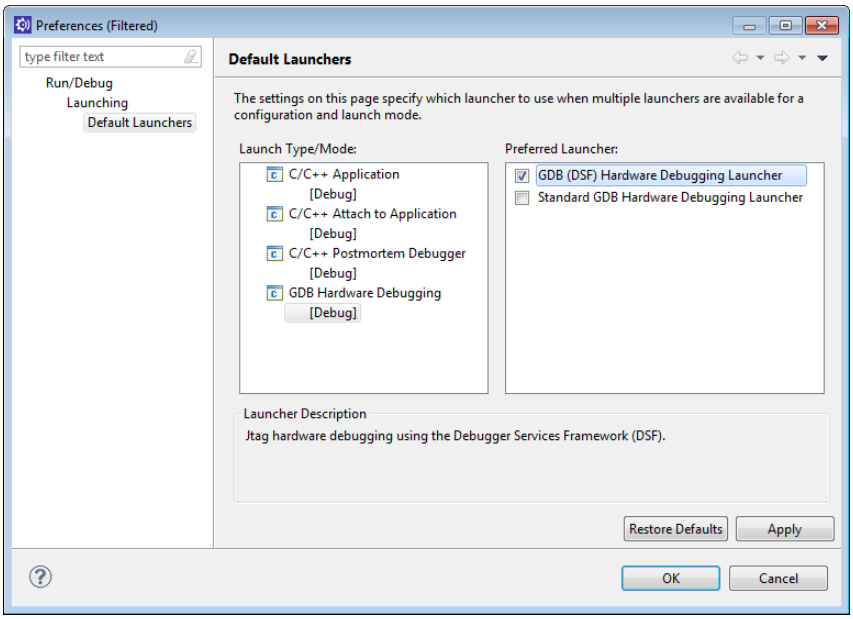


Figure 50. Setting Preferences for Default Launchers

3.2.4.6 Viewing ARM Cortex-M3 Core Registers

To view these registers in the Debug perspective, you have to install a third-party open source plug-in for Eclipse called the Embedded System Register View, which displays the registers for the ARM peripherals, and configure an XML file for the ARM Cortex-M3 core, which allows you to view the general registers.

3.2.4.6.1 Installing Embedded System Register View

To install the Embedded System Register View plug-in:

1. Choose **Help > Install New Software**.
2. Click the **Add** button next to the **Work with** field. It displays an Add Repository dialog box.
3. In the **Name** field, enter `EmbSysRegView`.
4. In the **Location** field, enter `http://embsysregview.sourceforge.net/update`.
5. Click **OK** and follow the rest of the prompts to complete the installation.
6. If you receive an authentication failure for the update site, click **OK** and continue because it will not affect the installation.
7. If you get a warning message about installing unsigned content, click **OK** to proceed.
8. The installation will take several minutes to complete, and you must restart the IDE at the end.

To configure the Embedded System Register View plug-in for use with Ezairo 7100:

1. Outside of the IDE, navigate to the following folder: **<EDK_HOME>***eclipse**plugins**org.eclipse.cdt.embsysregview_***<version_number>****data**cortex-m3*. **<EDK_HOME>** is the folder where the EDK is installed. See the *Ezairo 7100 Installation Guide* for more information about the default installation location. **<version_number>** is the version number for the Embedded System Register View plug-in, such as 0.2.3.
2. Create a folder called *OnSemiconductor*.
3. Copy the following file to the *OnSemiconductor* folder that you just created: *C:\Program Files (x86)\Common Files\SignaKlara\CTK\machines\Ezairo7100.xml*
4. In the IDE, choose **Window > Preferences**.
5. Navigate in the left panel to select **C/C++ > Debug > EmbSys Register View**.
6. In the drop-down menus for the following fields, make these selections:
   - **Architecture**: `cortex-m3`
   - **Vendor**: `OnSemiconductor`
   - **Chip**: `Ezairo7100`

   The Preferences dialog filled in for the EmbSys Registers view is shown in Figure 51.

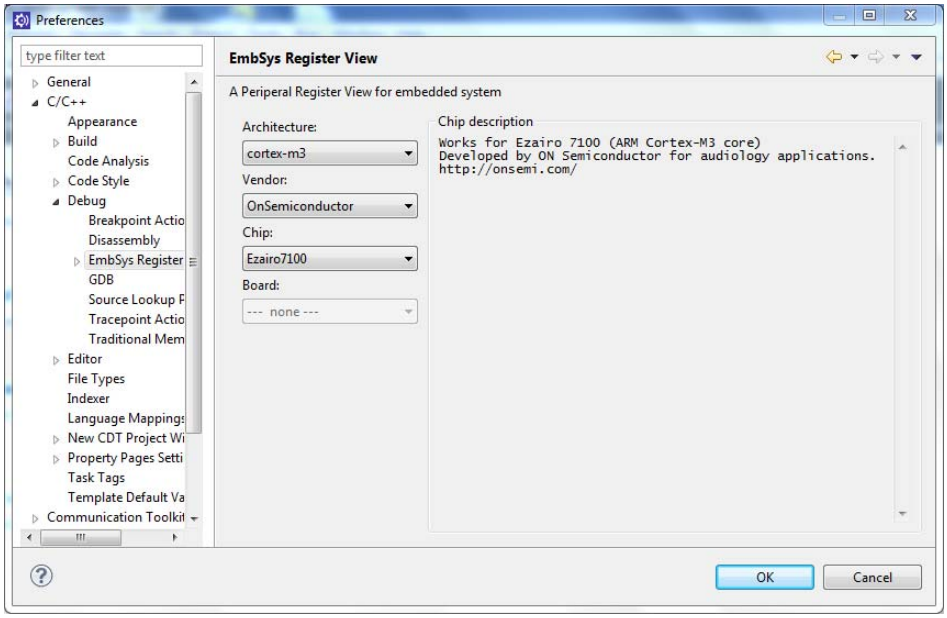


Figure 51. Preferences Dialog Configured for the EmbSys Registers View Plug-in

7. Click **OK**.

To use this plug-in, see the next section.

3.2.4.6.2 Accessing the Peripheral Registers

Once you have the EmbSys Register View plug-in, you can see the peripheral and control registers on the ARM Cortex-M3 core. To access the registers, and ARM debug session must be active (see Section 3.2.4.1, “Prerequisites for Launching an ARM Cortex-M3 Core Debug Session” on page 60 for more information):

1. Suspend the ARM core by selecting the running thread and clicking the Suspend button.
2. Choose **Window > Show View > Other...**.
3. In the Show View dialog, select **Debug > EmbSys Registers** and click **OK**.

The new EmbSys Registers view is shown in Figure 52.

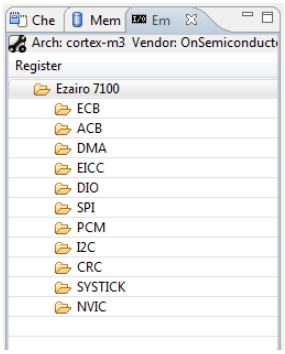


Figure 52. EmbSys Registers View Displaying Peripheral Registers for the ARM Cortex-M3 Core

3.2.5 Debugging Both Cores

The Ezairo 7100 EDK provides features to enhance debugging on both the CFX and the ARM Cortex-M3 cores when operating together. This is called the *coordinated debug* feature.

Toolbar buttons for coordinated debugging are shown in Figure 53.

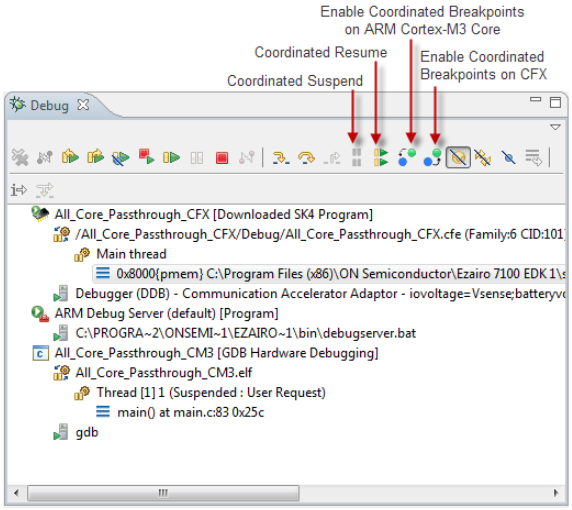


Figure 53. Toolbar Buttons for Coordinated Debugging

NOTE: The coordinated debug toolbar buttons are enabled only when a CFX process is selected in the Debug view.

3.2.5.1 Prerequisites for Coordinated Debugging

Before starting coordinated debugging, ensure that you have prepared the following:

- Create a project that uses both CFX and ARM Cortex-M3 processor code. Sample applications that do this include the All-Core Passthrough, the Wireless Audio Passthrough, and the Digital Volume Control. Refer to Section 2.2, "IDE Projects" on page 12.
- Create debug configurations for both the CFX and the ARM Cortex-M3 processor sides of this project. Refer to Section 3.2.2, "Debugging CFX Programs" on page 55 and Section 3.2.4, "Debugging ARM Cortex-M3 Core Programs" on page 60.
- Have appropriately named communication configurations for both the CFX and ARM Cortex-M3 processor parts of the Evaluation and Development Board. The port number must match in the Coordinated Debug tab, the debug server configuration, and the ARM debug configuration. Refer to Section 3.2.1.1, "CTK Configuration" on page 54.

3.2.5.2 Starting a Coordinated Debug Session

To use the coordinated debug feature, we recommend starting the cores and the debug sessions in a specific order to prevent start-up issues with initialization:

1. Launch the **SK4 Initialize** external tool.
2. Launch a CFX debug session.
3. If you have not checked **Suspend on entry point on launch (debug mode only)** in your CFX debug configuration, click **Suspend** and proceed to the next step.

   If you have, and assuming you are using a sample code project, then allow the `Initialize` function to complete by clicking **Resume**, then click **Suspend**. The `Initialize` function must complete before starting the ARM Debug Server because it resets the ARM Cortex-M3 core, which affects the Debug port for the ARM Cortex-M3 core. Ensure that the core is suspended at some point after the function completes.

4. Start the ARM Debug Server external tool.
5. Launch an ARM debug session. If the ARM Cortex-M3 core debug configuration is not set up to suspend on entry, use the **Suspend** button.

With two active sessions in the Debug view and the cores suspended, the buttons related to coordinated breakpoints, as well as the coordinated resume button, are enabled. The coordinated suspend button is enabled only when both cores are running. All buttons are enabled only when the CFX thread is selected. The buttons for each individual debug session are still functional.

3.2.5.3 Using Coordinated Resume

To use the coordinated resume feature, the following procedure assumes that both cores are suspended, as shown in Figure 54:

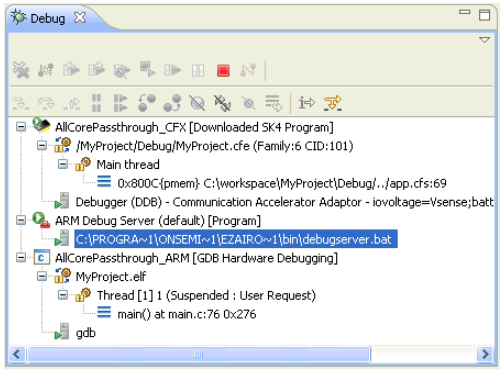


Figure 54. Both Cores in Suspended State

1. Select the CFX thread.
2. Click **Enable Coordinated Resume** (see Figure 53 on page 66 to identify this button).
3. Select the ARM Cortex-M3 core thread.
4. Click **Resume**.

This starts the CFX core first, followed by the ARM Cortex-M3 core within 26 SYSCLK cycles.

3.2.5.4 Using Coordinated Suspend

To use the coordinated suspend feature, the following procedure assumes that both cores are running, as shown in Figure 55:

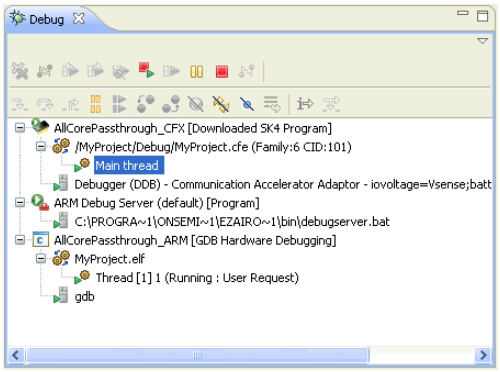


Figure 55. Both Cores in Resumed State

1. Select the CFX thread.
2. Click **Suspend CFX and ARM Cortex-M3 cores** (see Figure 53 on page 66 to identify this button)

This sends a command to the CFX that suspends both the CFX and the ARM Cortex-M3 cores simultaneously.

3.2.5.5 Coordinated Breakpoints

The EDK can enable coordinated breakpoints on the CFX core, the ARM Cortex-M3 core or both cores. Coordinated breakpoints provide the ability to have a breakpoint on one core suspend the other core. This allows you to examine what each core is doing at the moment when the breakpoint is hit.

When a breakpoint is hit, both cores are placed in a suspended state as shown in the Debug view. The editor shows where the program has stopped, as it would when a breakpoint is hit within a single-core debugging session. The core that has stopped because of the coordinated breakpoint (the one without the active breakpoint) is suspended as if you had clicked on the **Suspend** button.

IMPORTANT: When CFX coordinated breakpoints are enabled (i.e., enable suspending the ARM Cortex-M3 core when the CFX hits a breakpoint), and a coordinated resume is used, the CFX can potentially hit a breakpoint before the ARM Cortex-M3 core starts. This can happen because it can take up to 26 clock cycles before the ARM Cortex-M3 core starts running after the CFX executes the coordinated resume. If this happens, the user interface likely cannot correctly indicate whether the ARM Cortex-M3 core is running or suspended, nor the instruction that the program counter is on. To avoid this, ensure that at least 26 clock cycles worth of instructions are between the instruction where the CFX resumes and the first breakpoint. If necessary, you can insert NOP instructions in the code.

3.2.5.5.1 Enabling Coordinated Breakpoints

To enable coordinated breakpoints, two toolbar buttons are available (see Figure 53 on page 66 to identify them). They are enabled when the CFX process is selected in the debug view.

Enable Coordinated Breakpoints on ARM Cortex-M3 core: The ARM Cortex-M3 core enters a suspended state when any breakpoint set on the CFX is hit.

Enable Coordinated Breakpoints on CFX: The CFX enters a suspended state when any breakpoint set on the ARM Cortex-M3 core is hit.

NOTE: The coordinated breakpoint buttons are toggle buttons. If coordinated breakpoints are enabled, clicking the button again disables the coordinated breakpoints for that core.

When both coordinated breakpoint enable buttons are set, the core that hits a breakpoint first suspends the other core.

3.2.5.5.2 Procedure for Using Coordinated Breakpoints

To use coordinated breakpoints:

1. Suspend both cores.
2. Select the main thread for the core on which you want to set a breakpoint.
3. Set a breakpoint on that core.
4. Select the CFX thread and enable coordinated breakpoints for the other core (the core without the breakpoint).
5. Start running the core that does not have a breakpoint by selecting its thread and clicking **Resume**.
6. Start the core that has a breakpoint by selecting its thread and clicking **Resume**, or by using the coordinated resume procedure (Section 3.2.5.3, “Using Coordinated Resume” on page 67).

3.2.5.6 Troubleshooting Coordinated Debug

If you see one of the following, the IDE is no longer communicating properly with the ARM Cortex-M3 core:

- An error such as E_SEND_FAILURE or E_INVALID_STATE appears in the ARM debug server console or in a dialog in the IDE.
- The debug view shows the ARM Cortex-M3 core as running or suspended when it should be in the opposite state.

Several things can cause this to happen:

- The ARM Cortex-M3 core debug register is no longer enabled.
- The CFX has reset the ARM Cortex-M3 core debug port.
- Coordinated resume was used while CFX breakpoints were enabled, and the CFX hit a breakpoint before the ARM Cortex-M3 core had enough time to start running.

In these cases, restore communication by:

1. Terminating the ARM Cortex-M3 core debug session
2. Shutting down the ARM debug server process by pressing `Enter` in the ARM debug server console
3. Restarting the ARM debug server and ARM Cortex-M3 core debug session

You can leave the CFX debug session running. Do not rerun the **SK4 Initialize** external tool unless you also restart the CFX debug session.

3.3 Debugger Preferences

Debugging can be customized in several ways:

Debug configurations
: These store various settings that affect how debug sessions are launched.

Communication configurations

These determine how the debugger communicates with the target processor. The Communication Toolkit (CTK) Configuration Manager handles this and is described in more detail in Chapter 4, "The Communications Toolkit (CTK) Configuration Manager" on page 95.

Debug preferences

These control behavior for all debug sessions.

3.3.1 CFX Debug Configurations

CFX debug configurations determine the context of debug sessions. They specify which program to load and run, and which communication configuration to use. They can also perform simple scripting tasks as well as define custom scripting functions for use in the debug console. Everything pertinent to a debug session is stored in its debug configuration.

You can view and edit debug configurations through the **Debug Configurations** dialog (shown in Figure 56), which can be displayed by selecting the **Debug Configurations...** menu item in one of the following ways:

- Look under **Run** in the main menu
- Right-click anywhere in the Project Explorer view and click **Debug As...**.
- In the toolbar of the Debug perspective, click the drop-down arrow for the bug icon

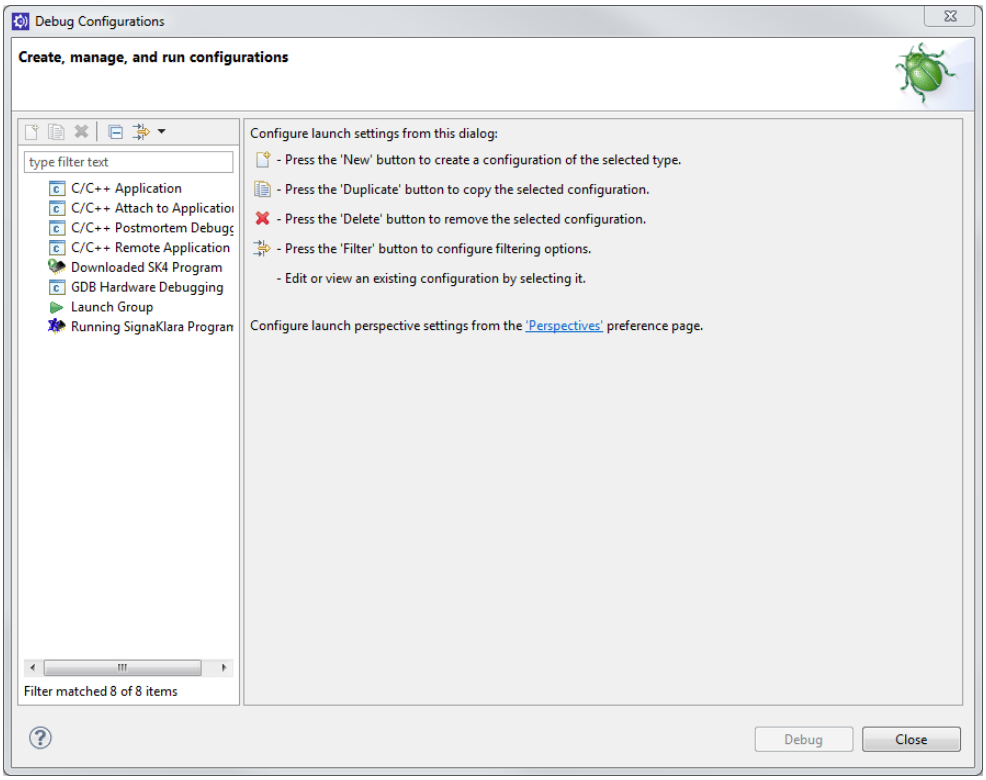


Figure 56. Debug Configurations Dialog

The pane on the left side displays all the existing configurations grouped by type. Configurations can be added and removed by clicking on the **New** and **Delete** buttons above it.

The right pane consists of a number of tabbed pages. The number of tabs depends on the type of configuration (**Downloaded SK4 Program** for example), but is always a subset of these:

Download This tab identifies:

- The executable files to debug
- Whether status registers are cleared when the session is launched
- Whether the debugger halts on the entry point after the program is loaded
- Whether to enable cfeinfo for DWARF information lookup (this option enables single-stepping and setting breakpoints at the end of P memory segments)

Import Symbols The tab lists the executable files from which to read symbolic information without downloading any program data.

Source Path A search path for locating source files

Connection The communication configuration to use for the debug session.

Scripting Definitions and actions to run in the Jython console when starting up the debug session.

Coordinated Debug ARM Debug Server settings.

3.3.1.1 Download Tab

The **Download** tab, shown in Figure 57, specifies parameters related to loading programs.

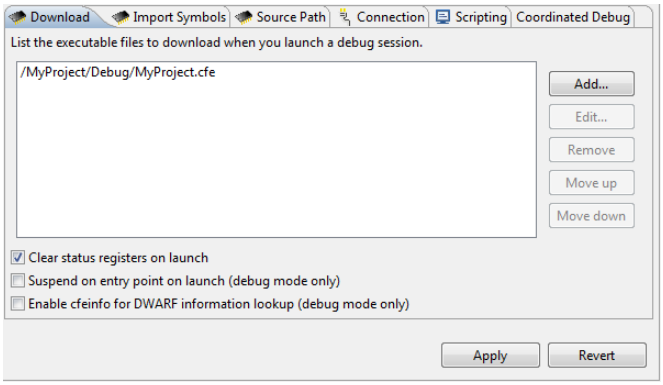


Figure 57. Debug Configurations Dialog - Download Tab

The **Download** tab allows you to specify several executable *.cfe* files to load when a debug session is launched.

The programs are loaded in the order that they are specified in the list. If two programs in the list have program data that overlaps in memory, the data in the second file data will overwrite that of the first. Execution will begin at the entry point of the first program.

The order of the files is also important for retrieving symbolic information. The debugger searches the executable files in the order that they are listed. Once it finds the information that it is looking for—be it the location of a label, the current source line, or some other debug info—the debugger stops searching.

If the configuration type is **Running SignaKlara Program**, this tab is not shown. If you need to download a program when you launch a debug session, you should use a **Downloaded SK4 Program** configuration.

The check box beneath the executable file list indicates whether or not status registers are cleared when the debug session is launched. Beneath that check box is another that indicates whether the debugger runs the program or suspends on the entry point after it has been loaded. A third check box specifies whether to enable cfeinfo for debugging data lookups (DWARF information).

3.3.1.2 Import Symbols Tab

The **Import Symbols** tab, shown in Figure 58, allows you to specify several executable *.cfe* files from which to import symbolic debugging information.

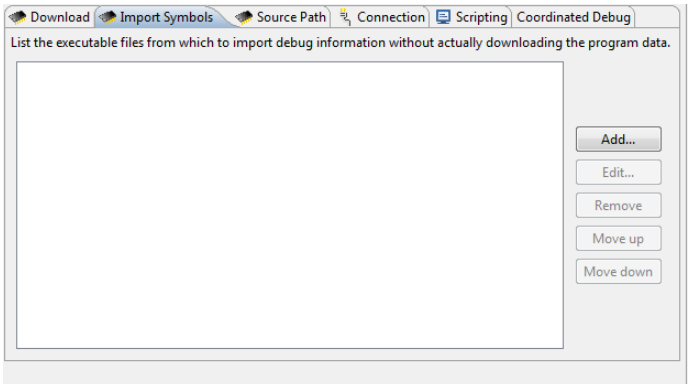


Figure 58. Debug Configurations Dialog - Import Symbols Tab

Files specified in the **Import Symbols** tab are read solely for their debugging information. Any program data that they might contain is not loaded.

As with the **Download** tab, the order in which the files appear in the list determines the order in which they are searched for debug symbols. For example, if two executable files each contain a symbol with the same name, the debugger will use the one from the file that appears first in the list.

NOTE: The list specified on the **Download** tab is searched before the list specified on the **Import** tab.

The **Import Symbols** tab is hidden when a debug configuration is being edited for running (versus debugging), as no symbolic information is needed to download and run a program.

3.3.1.3 Source Path Tab

The **Source Path** tab, shown in Figure 59, solves a problem that can occur with libraries. Source code for libraries is often compiled in a different folder from where it is eventually linked into a final executable, resulting in symbolic information that contains unrecognizable paths.

When the assembler compiles a source file to an object file, it stores file names in the symbolic information of the object file as relative to the folder in which it is run. When you run the object librarian to generate a library, the relative file names in the object files are copied, as is, into the resulting library. Similarly, the linker copies the relative paths in symbolic information from libraries and object files into the symbolic information for the resulting executable file. It does not adjust any file names in the symbolic information regardless of where a library or object file came from. When you link your source code with a library or object file that was compiled in a different folder, all of the file names imported from that library or object file's symbolic information are relative to where it was compiled, not to the one in which you are linking.

Without some help from you, the debugger might not be able to determine where the source code for a library or object file is located. It automatically uses the folder containing the executable file (*.cfe*) as a base for relative file names, but if your program contains object code from libraries or object files that were compiled in another directory, then you need to tell the debugger which other folders to use as a base for relative file names in your program's symbolic information.

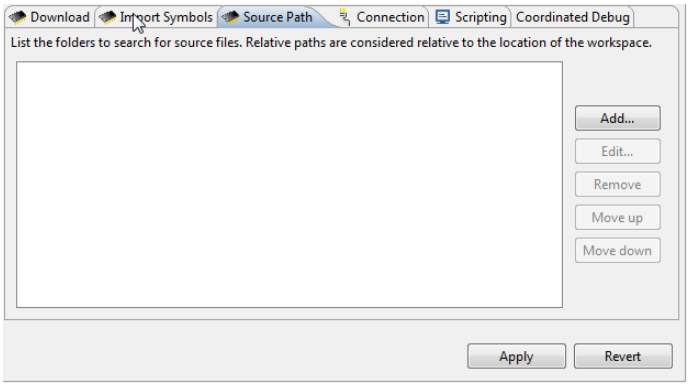


Figure 59. Debug Configurations Dialog - Source Path Tab

The **Source Path** tab allows you to specify these other folders. The debugger uses the folders in the list to find source files for your program. Implicitly starting with the folder that contains the program file, it tries each folder in the list as the base for any relative file names in the symbolic information, stopping when one resolves to an existing file.

Each folder name in the list can be absolute or relative. If a relative path name is specified, it is considered relative to the root folder of the workspace.

3.3.1.4 Connection Tab

The IDE is capable of communicating with a variety of development boards through a variety of communication devices. You can specify which communication device is used for a debug session in the Connection tab, shown in Figure 60.

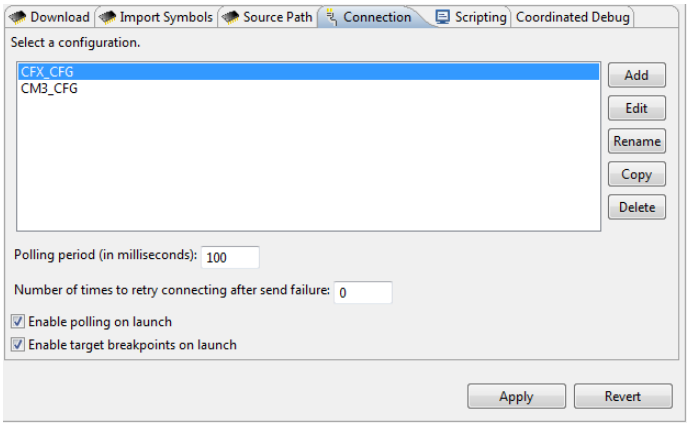


Figure 60. Debug Configurations Dialog - Connection Tab

The **Connection** tab presents a list from which you can select a communication configuration for use by the debug session. You can also add, edit and remove communication configurations from the list with buttons on the right side. More details on managing communication configurations is given in Chapter 4, "The Communications Toolkit (CTK) Configuration Manager" on page 95.

IMPORTANT: Communication configurations are global settings. Changing or removing communication configurations affects all of the debug configurations that rely on them.

The Connection tab also offers a way to control various aspects of how the debugger communicates with the target processor when your program is running. The debugger periodically sends status requests to the target processor to detect whether it has been suspended by a breakpoint (a process called *polling*). The Connection tab gives you control over whether and how often the debugger polls and how many times it should retry sending a status request in the event of a failure.

The **Polling period** field controls how long the debugger waits between each attempt to ascertain the status of the target processor. The time is specified in milliseconds and indicates a minimal wait. The actual time between status requests could be greater than what you specify, depending on how your host operating system and the Java Virtual Machine schedule the I/O operation.

If you are debugging a target processor with a communication protocol that can be shared with other data and devices, such as I^2C, a single status request might time out as it waits for the protocol to become available. You can adjust the **Number of times to retry connecting after send failure** field to force the debugger to retry sending a status request a certain number of times before it gives up and waits another polling period.

Conversely, it is possible that polling from the debugger will interfere with other communication on a shared protocol, such as I^2C. You can suppress polling in the debugger from the moment that you launch a session by clearing the **Enable polling on launch** check box. However, this also disables breakpoint detection.

A check box labeled **Enable target breakpoints on launch** controls whether breakpoint hardware is enabled on the core when the debug session is initiated. If you choose to have breakpoints disabled when your session launches, you can re-enable them later using the **Toggle Target Breakpoints** button in the Debug view toolbar. See Section 2.3.2, "Breakpoint Manipulation" on page 26 for more details.

3.3.1.5 Scripting Tab

The IDE comes with an integrated Jython interpreter that you can use to automate repetitive or tedious tasks while debugging. It is accessible from the Scripting tab, as shown in Figure 61. Note that the Jython interpreter is only available when working with CFX applications.

The debug console (described in Section 3.4, “Jython Command Console” on page 85) provides a way to interact with the interpreter by typing commands at a prompt, which is ideal for short commands, but inconvenient for defining longer functions and procedures. For example, once a line has been entered into the interpreter, it cannot be edited. Furthermore, all functions defined in the debug console last only as long as the debug session is open. Once the session is terminated, definitions are lost, and must be entered again when a new debug session is launched.

The **Scripting** tab of the **Debug Configurations** dialog overcomes these difficulties by providing a place to capture function definitions and initialization code that are used every time a debug session is launched.

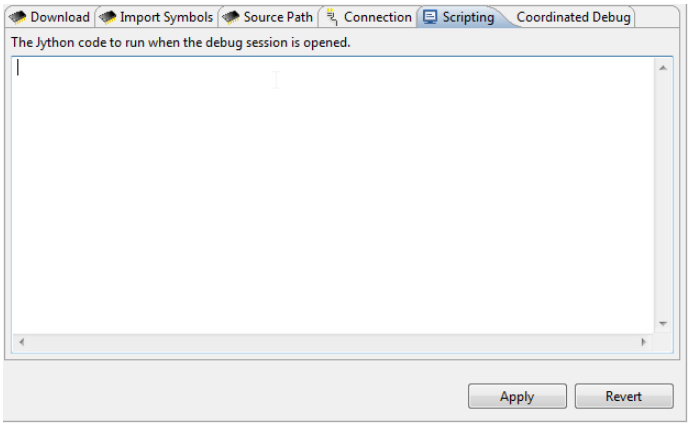


Figure 61. Debug Configurations Dialog - Scripting Tab

The tab itself is straightforward: simply enter the definitions and initialization code that you want to run when the debug session is launched.

Depending on the type of the debug configuration, whether it is a **Downloaded SK4 Program** or a **Running SignaKlara Program**, the target processor might be suspended or running when the script is executed. If your script contains operations that communicate with the target processor, such as reading or writing registers or memory, and the debug configuration is a **Running SignaKlara Program**, it is your responsibility to suspend the processor beforehand. If the debug configuration is a **Downloaded SK4 Program**, you may assume that the processor is suspended when the script is executed.

3.3.1.6 Coordinated Debug Tab

When running a coordinated debug session, the environment needs the hostname and port number for communicating with the ARM Debug Server. This tab specifies those values (as shown in Figure 62).

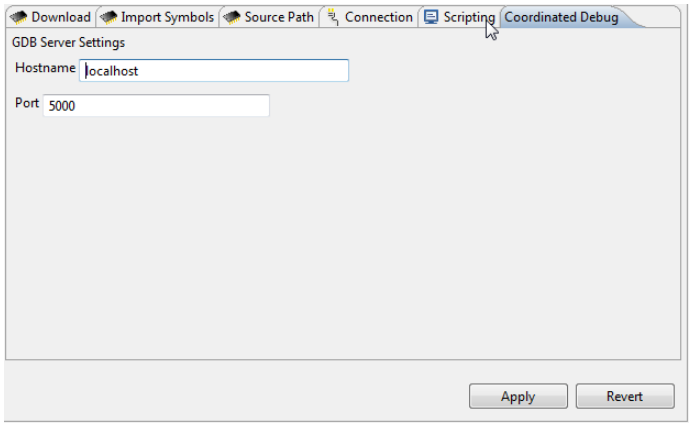


Figure 62. Debug Configurations Dialog - Coordinated Debug Tab

3.3.2 ARM Cortex-M3 Core Debug Configurations

As with the CFX, debug configurations for the ARM Cortex-M3 core determine the context of debug sessions. They specify which program to load and run, and which communication configuration to use. Everything pertinent to a debug session is stored in its debug configuration.

You can view and edit ARM Cortex-M3 core debug configurations through the **Debug Configurations** dialog (shown in Figure 63). The dialog can be displayed by selecting the **Debug Configurations...** menu item and then navigating to the **GDB Hardware Debugging** section in the tree on the left hand side. Refer to Section 3.3.1, "CFX Debug Configurations" on page 70 for the several ways of opening the **Debug Configurations** dialog.

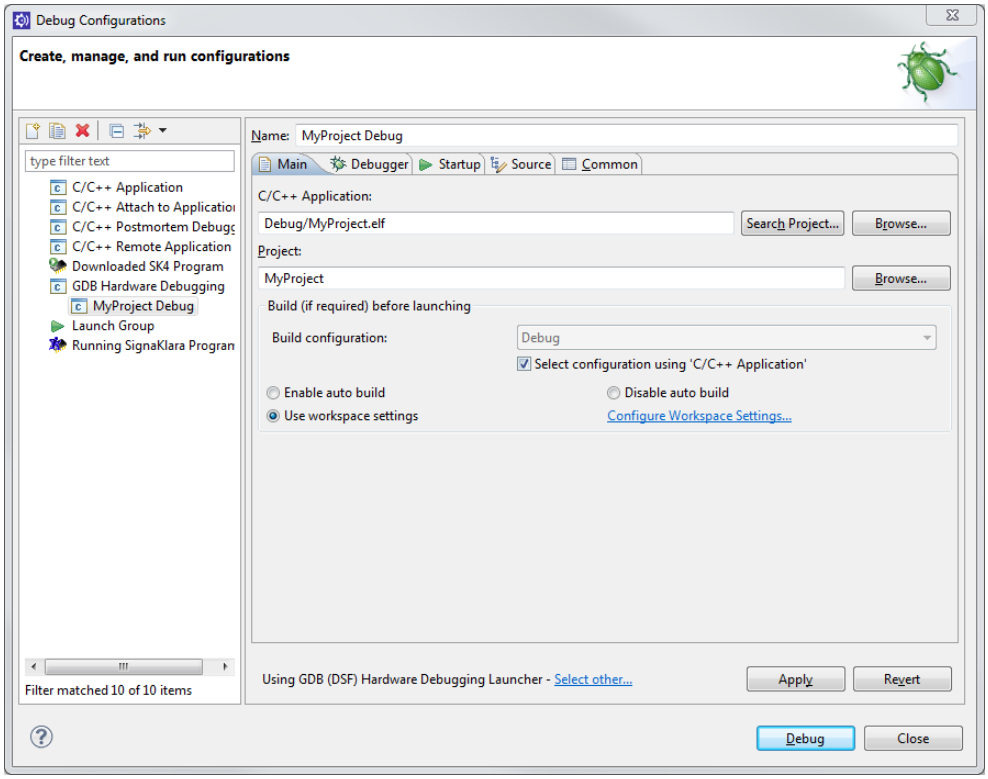


Figure 63. ARM Cortex-M3 Core Debug Configurations Dialog

The pane on the left side displays all the existing launches grouped by type. Configurations can be added and removed by clicking on the **New** and **Delete** buttons above it.

This right pane consists of a number of tabbed pages. The following tabbed pages are visible when working with a **GDB Hardware Debugging** debug configuration:

Main This tab identifies the executable files to debug and build-before-launch settings.

Debugger This tab contains settings required to run the GDB and connect to the ARM Debug Server.

Startup This tab configures what tasks are performed when starting a debug session.

Source This tab is used to specify the location of folders containing the project source code.

Common This tab contains general debug configuration settings.

3.3.3 Debug Preferences

You might want to change preferences to customize the working environment. You can access all IDE preferences with the **Window** > **Preferences...** menu item. General information about IDE preferences is provided in Section 2.5, “Setting Preferences” on page 41. This section highlights some of the preferences that apply to debugging found under **Run/Debug**.

3.3.3.1 Run/Debug

The **Run/Debug** preference tab provides several general debug settings as shown in Figure 64.

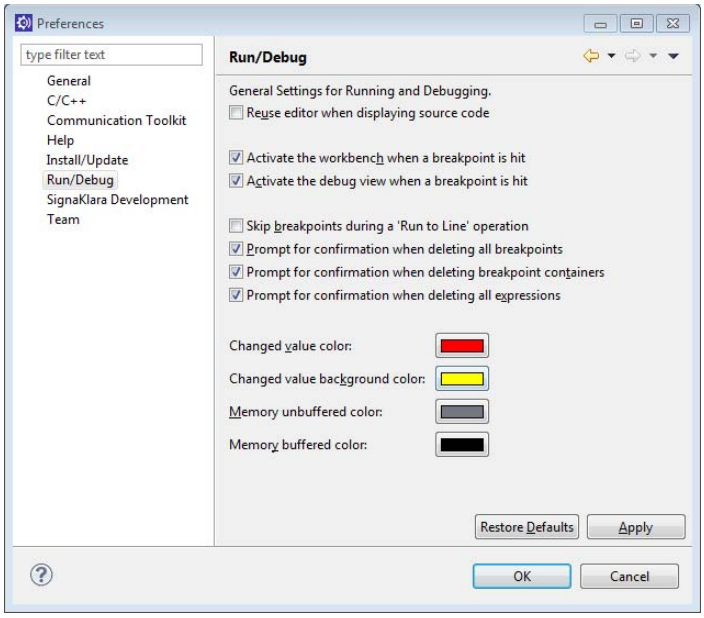


Figure 64. Preferences - Run/Debug Tab

You can choose to have the IDE reuse a single editor window when stepping through code, reducing the number of editors you have open at any one time. If the check box is unchecked, the IDE will open a new editor whenever execution suspends in a file that has not yet been opened.

You can control which windows are activated when a breakpoint is hit: the entire Workbench window, or the Debug view. When a window is activated, it is brought to the top.

3.3.3.2 Console

The CFX debug console has several preferences associated with it, as shown in Figure 65. To view and change these preferences, click **Window > Preferences > SignaKlara Development > Jython**.

The ARM Cortex-M3 core debug console preferences are accessible by clicking **Window** > **Preferences** > **Run/Debug** > **Console**.

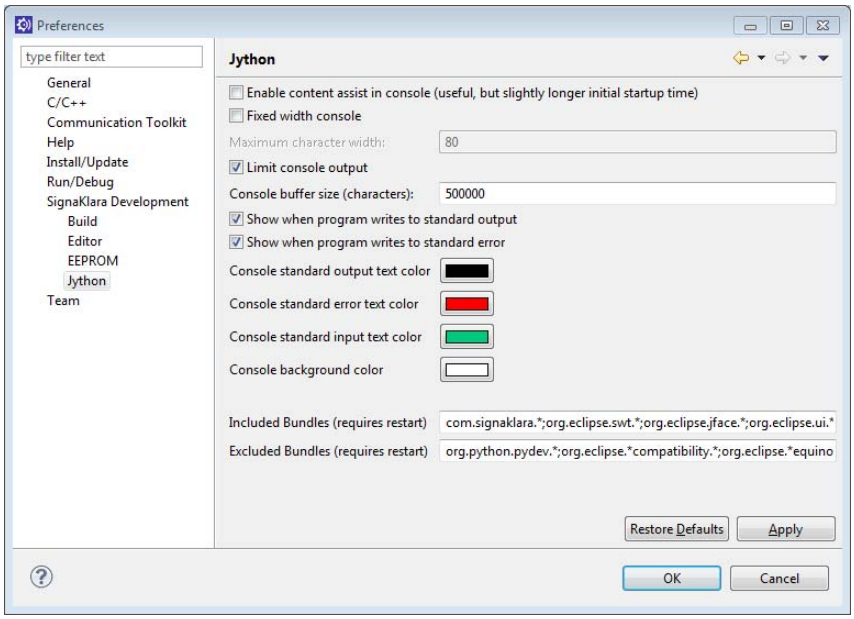


Figure 65. Preferences - Jython Tab

You can control whether the console wraps lines at a fixed width (and what that width is), limit the buffer size of the console to a fixed number of characters, ignore data printed to standard output or standard error, and choose the colors in which text is displayed.

3.3.3.3 Disassembly

The IDE offers some control over how disassembled instructions appear in the disassembly view.

3.3.3.3.1 CFX Disassembly

You can control how numeric values are displayed in the CFX disassembler as shown in Figure 66.

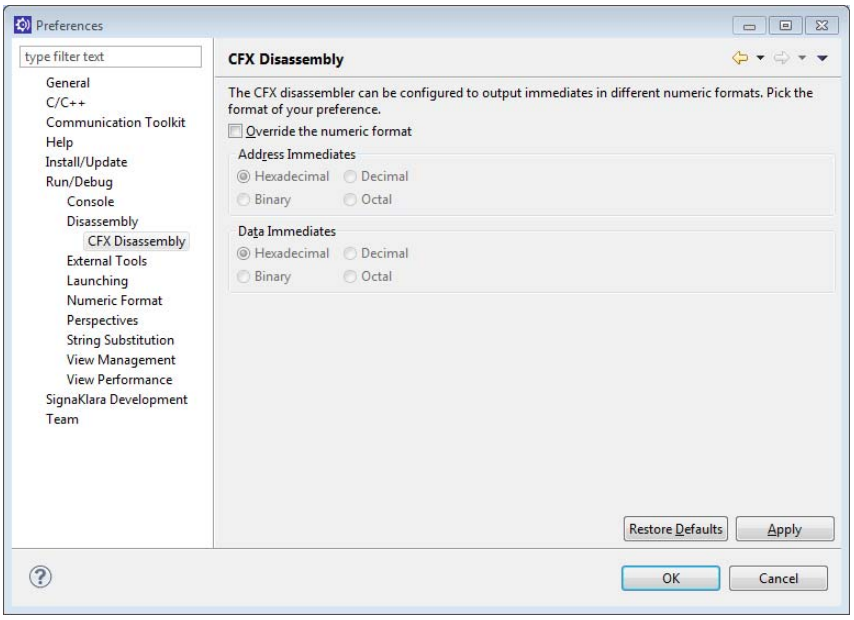


Figure 66. Preferences - CFX Disassembly Tab

By default, the disassembler uses the global setting, which is also used to control, for instance, how register values are displayed in the register view. You can override this default behavior, and specify your own preferences for addresses and data, by checking the **Override the numeric format** box.

3.3.3.3.2 ARM Cortex-M3 Core Disassembly

You can control what is displayed and the address display format in the ARM Cortex-M3 core disassembler by choosing **C/C++ > Debug > Disassembly** as shown in Figure 67.

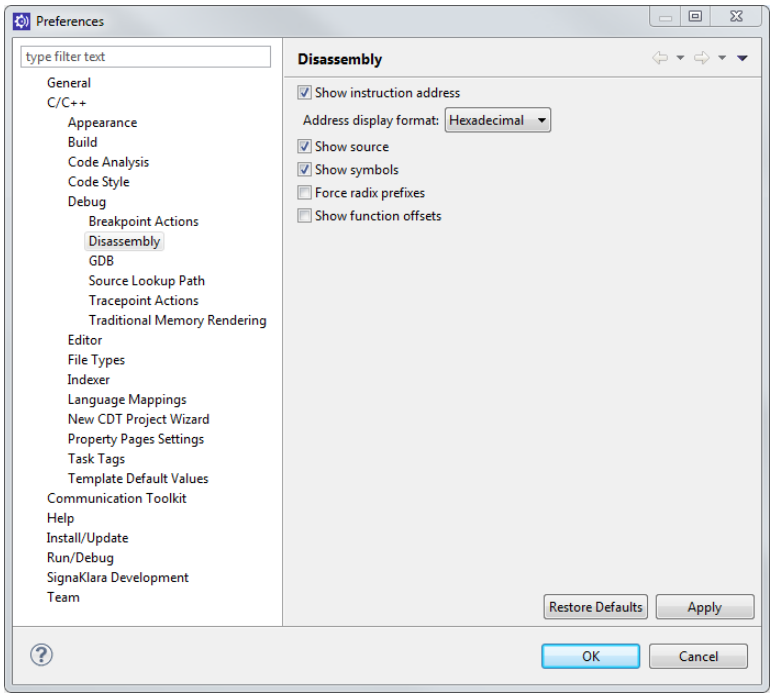


Figure 67. Preferences - ARM Cortex-M3 Core Disassembly

3.3.3.4 External Tools

There are preferences for working with external tool project builders, such as Ant. The external tools preferences are shown in Figure 68.

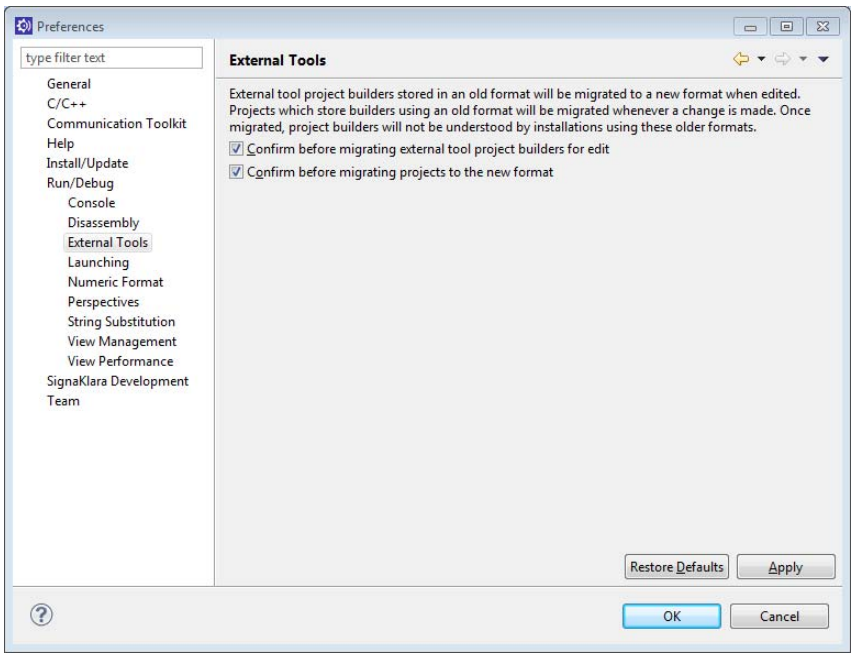


Figure 68. Preferences - External Tools Tab

You can control when you will be asked to confirm updates to project files.

3.3.3.5 Launching

The Launching tab offers some control over how debug sessions are launched, as shown in Figure 69.

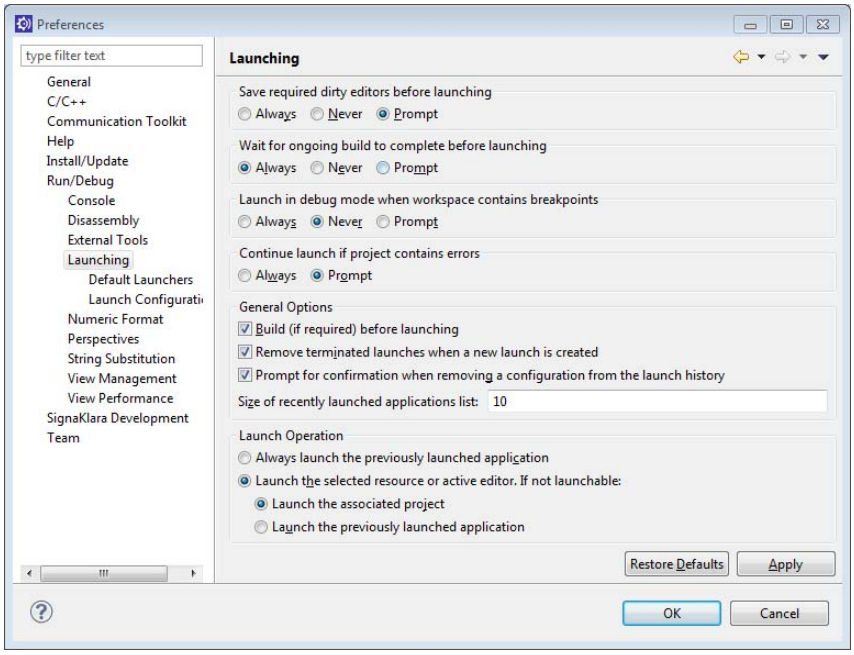


Figure 69. Preferences - Launching Tab

Several options are available to control how launching interacts with building:

- Whether a build should be done before a debug session is launched
- Whether edited files should be saved before launching
- Whether builds that are in progress must complete before a session is launched

There is an option specifying whether to remove terminated sessions from the Debug view when a new session is launched.

You can also choose:

- Whether, in the presence of breakpoints, only debug sessions should be launched, even if the user chooses to download and run the program
- Whether a launch should continue if there are errors in a project

You can also specify how many items appear in the list of debug configurations in the debug toolbar.

3.3.3.6 Numeric Format

You can control how numeric values are displayed in various Debug views, using the Numeric Format tab shown in Figure 70.

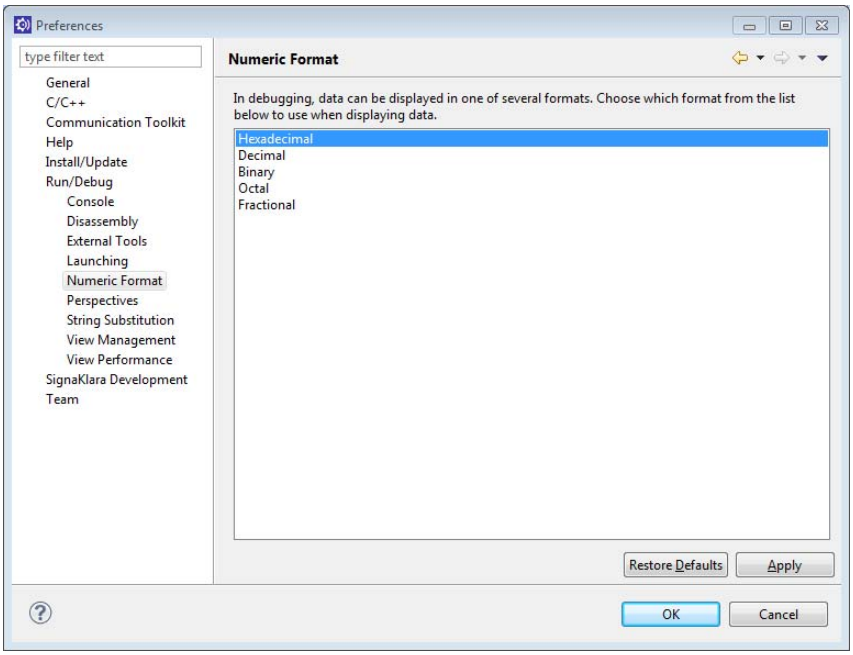


Figure 70. Preferences - Numeric Format Tab

You can choose how numeric values are displayed in the Expression and Register views: hexadecimal, decimal, binary, octal or fractional.

3.3.3.7 Perspectives

These settings control perspectives for running and debugging.

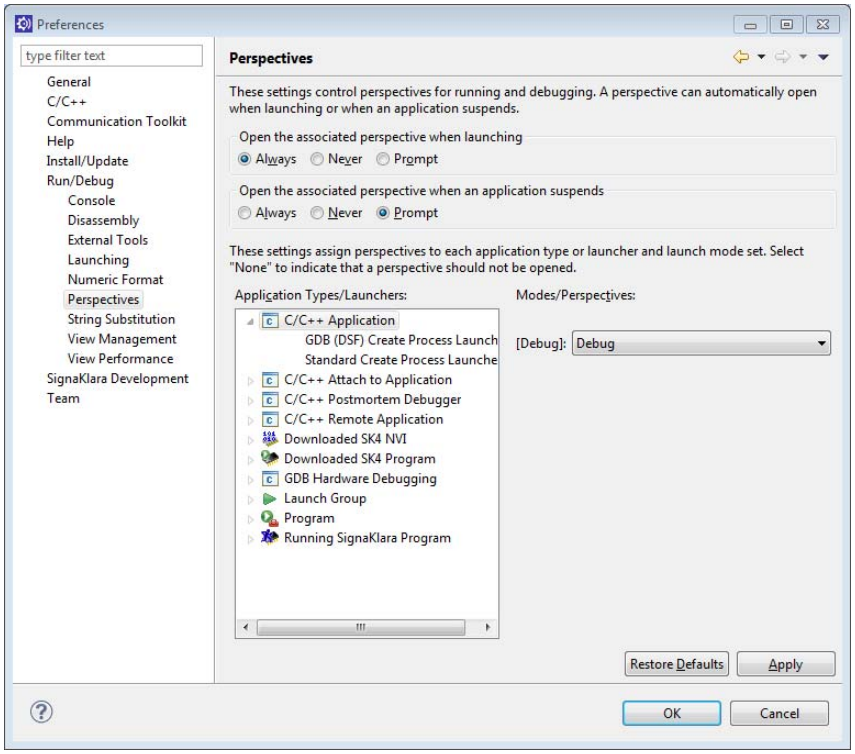


Figure 71. Preferences - Perspectives Tab

Based on the application type or launcher, it is possible to assign an associated perspective. Select an application type/launcher from the list on the left side. Then on the right side, using the **Modes/Perspectives** drop-down list box, select the perspective to associate.

3.4 JYTHON COMMAND CONSOLE

The IDE includes an embedded command interpreter that is accessible through the debug console as described in Section 3.1.2.8, “Debug Consoles” on page 53. The language that the interpreter understands is Jython, a Java-based implementation of the popular scripting language Python.

Jython is a full-featured object-oriented programming language with variables, functions, control flow, complex data types and a huge library. As such, teaching you about this language is beyond the scope of this guide. To learn Jython, see the reference material in Section 3.4.5, “Further Information” on page 89.

NOTE: The Jython Command Console is only available when debugging applications on the CFX core.

3.4.1 An Introduction to the Interpreter

You can invoke commands by calling methods on an object named `ddb`. For example, to single step the processor, type `ddb.step()`. To run a program from its entry point, type `ddb.run()`. To get a list of all the commands available, type `ddb.help()`.

The methods that you can call on the `ddb` object are listed and described in Appendix A, “DDB Command Reference” on page 107. This section provides a few examples.

NOTE: Lines beginning with >>> indicate user input. Other lines are interpreter output.

We have already seen how to single step:

```
>>>ddb.step()
32769L
```

You can also step over a line:

```
>>>ddb.next()
32771L
```

To continue from the current point of execution without blocking (i.e. without locking up the user interface while it waits for a breakpoint to be hit; this is the equivalent of pressing the run button in the Debug view toolbar), enter:

```
>>>ddb.contnb()
```

To determine the current point of execution:

```
>>>ddb.where()
..\MyProject\source.cfs:13
32772L
```

Commands can also take parameters. For instance, to evaluate an expression, pass in a string containing the expression to evaluate:

```
>>>ddb.eval("main")
32768L
```

You can write registers (in this example, a0 is the register name):

```
>>>ddb.put("a0", 0x20)
```

And read them:

```
>>>ddb.get("a0")
32L
```

You can write memory:

```
>>>ddb.write(1, 0x0100, [0, 1, 2, 3, 4, 5, 6, 7])
8
```

And read it:

```
>>>ddb.read(1, 0x0100, 8)
array([0L, 1L, 2L, 3L, 4L, 5L, 6L, 7L], long)
```

The first parameter in both the previous examples is a numeric identifier for the memory space, a standard parameter in commands that require an address. On CFX platforms, EEPROM, program, X data, Y data and XY data have the identifiers -1, 0, 1, 2 and 3, respectively.

If you would rather specify the address as an expression, there is a whole family of methods, whose names are prefixed with 'x', that accept addresses as expressions. For example:

```
>>>ddb.xread("0x0100{XMEM}", 8)
array([0L, 1L, 2L, 3L, 4L, 5L, 6L, 7L], long)
```

You can disassemble code in memory, too. An example of CFX-disassembled code is:

```
>>>ddb.xdisassemble("main", 4)
0x8000 (0x3A000003) load a0, x[sp + 3]
0x8001 (0x3A000003) load a0, x[sp + 3]
0x8002 (0x3880102A) load a0, y[4138]
0x8003 (0x3CD80A00) breakpoint
'0x8000 main:\n0x8000 (0x3A000003) load a0, x[sp + 3]\n0x8001 (0x3A000003) load a0, x[sp
   + 3]\n0x8002 (0x3880102A) load a0, y[4138]\n0x8003 (0x3CD80A00) breakpoint\n'
```

It is even possible to set breakpoints:

```
>>>ddb.breakAt("/MyProject/source.cfs", 12)
array([2], int)
```

And remove them:

```
>>>ddb.rmBreak(2)
```

3.4.2 Defining Functions

To automate more complicated actions, you need a way to compose the basic commands into larger abstractions. Functions and flow control serve this purpose.

For example, suppose you would like a function that performs a set number of step operations, waiting for a brief amount of time between each step so you can watch the execution progress in the editor. Here is a definition for such a function:

```
import time

def stepN(n, sleepSecs=0.5):
  for i in range(n):
    ddb.step()
    time.sleep(sleepSecs)
```

There are several new elements in this example. The first is the `import` statement, which initializes the `time` module from Jython's standard library, preparing it for use later in the function.

The `def` line defines the `stepN` function with two parameters:

`n` The number of steps to make.

`sleepSecs` The time to wait between each step in seconds. This parameter has a default value, so including it is optional when calling the function.

The next line is indented from the definition line. Jython uses whitespace to structure code.

The `for` statement iterates through any indented lines of code that follow `n` times. There are two function calls in the loop. The first, `ddb.step()` executes one instruction on the processor. The second, `time.sleep(sleepSecs)` waits `sleepSecs` seconds before continuing.

If you were to copy the above definition to the scripting tab of a debug configuration (see Section 3.3.1.5, "Scripting Tab" on page 75), and launch a debug session, you could run `stepN` from the debug console like this:

```
>>>stepN(100)
```

This would step the processor 100 times with each step separated by half a second. You would be able to follow the progress of the program in the editor as each step is taken.

3.4.3 Storing Functions in Modules

Once a function is written, you might find that you can use it elsewhere. Although it is simple enough to copy the definition from one scripting tab to another in the **Debug Configurations** dialog, you might find it inconvenient to update all the copies if you ever find that the definition must be changed. It would be better to keep a single definition that you can use from any debug configuration.

Jython allows you to package your function definitions in a module so that other scripts can use them via the `import` statement.

A module is a text file containing function definitions. There are only two requirements on the text file:

1. It must be located in either a folder named *scripts* in your project or a folder named *scripts* in your user home directory (typically *C:\Documents and Settings**username**\scripts* or *C:\Users**username**\scripts*), because this is where Jython searches for modules.
2. It must end with the extension *.py*, because that is how Jython recognizes modules.

For example, if you were to bundle the `stepN()` function of the previous section in a module, you might create a file named *stepping.py* containing the definition.

You could then replace the function definition with an `import` statement in the scripting tab of any debug configurations that use `stepN`:

```
import stepping
```

From the debug console, you could then call the function:

```
>>>stepping.stepN(100)
```

3.4.4 Launching the Standalone Debugger

The CFX debugger's command interpreter can also be run outside of the IDE as a separate process. Its interface is textual, not graphical, so it offers an excellent way to run debugging scripts, such as automated batch tests, from the Windows command prompt.

You can launch the standalone command-line debugger from the command prompt with the `ddb` command, which takes the following form:

```
ddb [options] [file | -] [args]
```

The valid options are described in Table 2.

Table 2. Options for the DDB Command

Option	Description
`--config=CONFIG`	Specifies the name of a CTK communication configuration to use for debugging. If none is specified, DDB uses the first one in the list of available configurations. For this reason, we recommend that you always specify this option. You can see the list of available configurations (and add new ones) using the CTK Configuration Manager.
`--sourcepath=PATH`	Specifies a source code lookup path as described in Section 3.3.1.3, “Source Path Tab” on page 72. Path names in `PATH` must be delimited with a semicolon.
`-h` or `--help`	Prints a usage message and terminates.

You can supply a script for DDB to run with the `[file | -]` argument. If you supply a file name (or a dash to use standard input), the command interpreter reads the entire contents of the file as a script and runs it, terminating when the script completes. If no script file is specified, DDB launches in interactive mode, allowing you to enter commands individually and see their results. Any arguments that follow the file name are passed on as arguments to the script file.

3.4.5 Further Information

For more information about Jython and Python, refer to the following:

- Jython website: http://www.jython.org/
- Python website: http://python.org/
- Lutz, Mark, and Ascher, David. *Learning Python*. 2nd ed. (O’Reilly, 2003)
- Lutz, Mark. *Programming Python*. 2nd ed. (O’Reilly, 2001)
- Pedroni, Samuele, and Rappin, Noel. *Jython Essentials*. 1st ed. (O’Reilly, 2002)

3.5 ARM Cortex-M3 Core Command Console

In addition to the commands available in the standard GDB console, the ARM Cortex-M3 core debug console supports some commands specific to the ARM Cortex-M3 core. Interrupt control, hardware breakpoints, and watchpoints for the ARM Cortex-M3 core can be managed using commands entered directly in the debug console. Information about these can be obtained with query commands in the GDB console as well.

When you are debugging an ARM Cortex-M3 processor program, you can tell that the GDB console is active because **[GDB Hardware Debugging] gdb** appears in the title bar of the Console view as shown in Figure 72. If it is not there, click the **gdb** line in the Debug view or use the Display Selected Console dropdown list on the Console view,

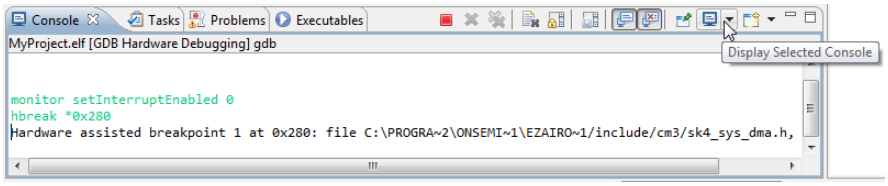


Figure 72. Display Selected Console Dropdown List

3.5.1 Interrupt Control on the ARM Cortex-M3 Core

Interrupts on the ARM Cortex-M3 core can be enabled and disabled by using the monitor command `monitor setInterruptEnabled <state>`, where **`state`** is `0` or `1`,indicating false or true respectively. Interrupts are enabled by default.

To disable interrupts, enter the command `monitor setInterruptEnabled 0`, as shown in Figure 73.

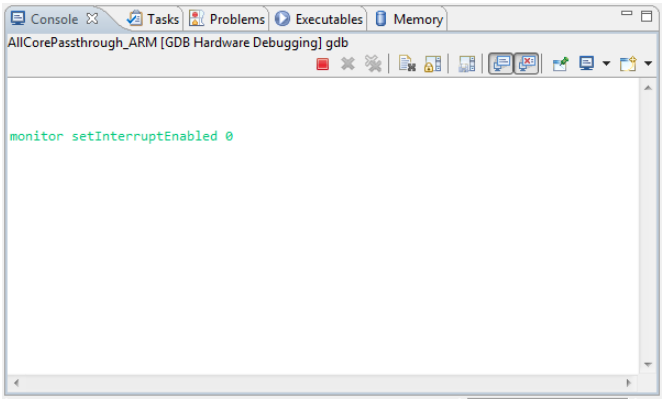


Figure 73. Disabling ARM Cortex-M3 Core Interrupts

Disabling interrupts will clear any of the 255 interrupts enabled in the NVIC. It will also clear all pending interrupts. Interrupts which are currently being serviced (active) are not affected.

To enable interrupts, enter the command `monitor setInterruptEnabled 1`. This command only enables the interrupts which were enabled before interrupts were disabled.

3.5.2 Creating ARM Cortex-M3 Core Hardware Breakpoints and Watchpoints

Hardware breakpoints use the ARM Cortex-M3 core controller hardware instead of substituting the line of code with a break instruction. ARM Cortex-M3 core hardware breakpoints can be set using the `hbreak <addr>` command in the console, where `addr` is the address of an instruction in memory.

To place a hardware breakpoint at address 0x0280, type `hbreak *0x280` in the GDB console. This acts as a normal breakpoint, such that execution of the program is suspended at that address.

Watchpoints can suspend the execution of a program when a certain variable is accessed. The type of access can be specified as well, so that the program suspends only when the variable is read, only when the variable is written, or when the variable is either read or written. To set a watchpoint on the ARM Cortex-M3 core, type one of the following in the console:

- `rwatch` **`<variable>`**, to halt on a read access
- `watch` **`<variable>`**, to halt on a write access
- `awatch` **`<variable>`**, to halt on either a read or write access

3.5.2.1 Managing ARM Cortex-M3 Core Hardware Breakpoints and Watchpoints

When a hardware breakpoint or watchpoint is created, it is assigned a number. This number is used with the `enable` and `disable` commands. The numbers start with 1 and ascend by 1 for each new hardware breakpoint or

watchpoint. So the first one created will be assigned the number 1, the second 2, and so on. Information about which number is assigned to a given hardware breakpoint or watchpoint can be obtained using the `info breakpoints` command. This command also provides information about the type (hardware breakpoint or watchpoint), enablement, and address or variable name as applicable.

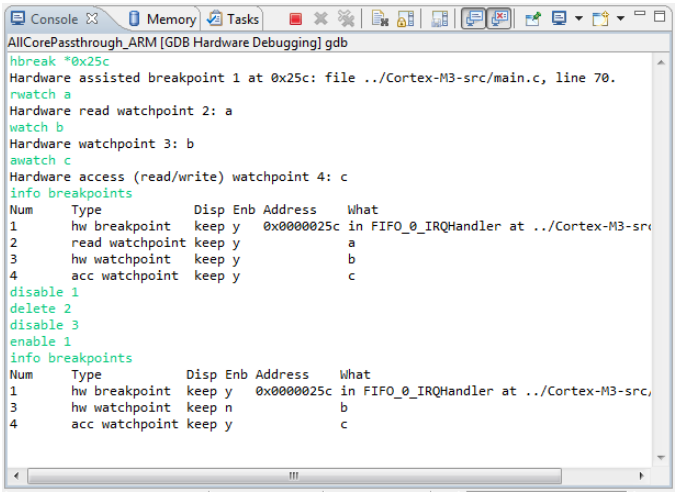


Figure 74. ARM Cortex-M3 Core Console Commands

To disable and enable a hardware breakpoint or a watchpoint, use the commands `disable` **`<n>`** and `enable` **`<n>`** respectively, where **`n`** is the number of the hardware breakpoint or watchpoint to disable or enable.

To delete a hardware breakpoint or watchpoint use the command `delete` **`<n>`** where **`n`** is the number of the hardware breakpoint or watchpoint to delete.

See Figure 74 for some examples of these commands.

3.6 Expression Language

NOTE: This section applies only to the CFX core. The expression language for ARM Cortex-M3 processors is publicly available with other GDB documentation.

The main purpose of having an expression language is to look up data on a target processor from a symbolic description, which can include label and register names. This saves a programmer from having to remember the addresses of important data that are likely to shift in memory when a program is changed and recompiled.

The language consists of C-style expressions. As in C, the expressions recognized by this expression language consist of literals, identifiers and operators. If you have programmed in the C programming language you will find the expression syntax quite familiar. Most of C's operators are supported in the language, as well as some additional operators for specifying memory spaces.

Expressions are used in the following places when you are debugging:

- The Expressions view
- For certain commands in the debug console
- The upload and download dialog boxes

- Addresses in the Memory view
- To change register values
- To change memory values

3.6.1 Literals and Identifiers

Literals express constant values and generally follow the C notation. They can be expressed in decimal (for example, `16`), octal (`020`) indicated by the leading zero, hexadecimal (`0x10`), binary (`0b1101`), or fractional (`0.5231r`).

A width for fractional literals can be specified in two ways. The first is with a memory bank specifier (described in more detail below). For example, the expression `0.5r{pmem}` is interpreted as a value with the same width as program memory. The second way to specify the width is with an optional suffix on the literal. For example,`0.5r16` is treated as a 16-bit integer that evaluates to the value `0x4000`. As a further example, `0.5r32`evaluates to `0x40000000`.

Fractional literals are evaluated as integer values. Consequently, multiplying two fractional literals together is equivalent to multiplying their corresponding integer values. For example,`0.5r16 * 0.5r16` is equivalent to `0x4000 * 0x4000`. Both result in `0x10000000` (or `4096.0r16` as a 16-bit fractional value).

Identifiers are names. They refer to labels, DSP registers, analog control registers, digital control and configuration registers, or memory banks. When an identifier names a label, the resulting value is the address of that label. When it names a register, it results in the value stored in the register.

In assembly language, symbols are allowed to contain the dot (.) character. This creates an ambiguity with the member access operator, which also makes use of the character. To eliminate this ambiguity, you must always escape dot characters in identifiers by preceding them with a backslash (\). To refer to a symbol with the name `name.containing.dots`, for example, type `name\.containing\.dots`.

Memory bank identifiers are built-in names corresponding to the various memory spaces available. They are case-insensitive identifiers that have the form `<name>MEM`, where `<name>` is a short name (often a single letter) denoting the memory bank. Memory bank identifiers evaluate to the integer index of the banks they represent and are most useful when specifying a bank with the { } operator.

For example, on the CFX DSP:

- `RMEM` or `-1` identifies non-volatile EEPROM
- `PMEM` or `0` identifies program memory
- `XMEM` or `1` identifies X data memory
- `YMEM` or `2` identifies Y data memory
- `XYMEM` or `3` identifies XY data memory

3.6.2 List of Expression Operators

The DDB expression language uses many of the same operators found in the C language, and in the same order of precedence. They include the C operators for arithmetic, comparison, bit manipulation, conditions and array indexing. Address referencing and dereferencing are also supported. The following table provides a complete list of the operators in order of precedence from highest to lowest. Operators in the same group have the same precedence and are evaluated from left to right, except prefix operators (precedence level 3), which are evaluated from right to left.

Table 3. Operator Precedence

Precedence	Operator	Description
1 (highest)	@"*file*":*line*	address of source location
	(*expr*)	parenthesized expression
2	*expr* [*expr*]	array indexing
	expr { *expr* }	memory bank specification
	expr -> *expr*	pointer dereference
	expr . *expr*	member access
3	* *expr*	memory dereferencing
	& *expr*	address of expression
	+ *expr*	has no effect
	- *expr*	negation
	! *expr*	logical NOT
	~ *expr*	bitwise complement
4	*expr* * *expr*	multiplication
	expr / *expr*	division
	expr % *expr*	remainder
5	*expr* + *expr*	addition
	expr - *expr*	subtraction
6	*expr* << *expr*	shift left
	expr >> *expr*	shift right
7	*expr* > *expr*	greater than
	expr >= *expr*	greater than or equal to
	expr < *expr*	less than
	expr <= *expr*	less than or equal to
8	*expr* == *expr*	equal to
	expr != *expr*	not equal to
9	*expr* & *expr*	bitwise AND
10	*expr* ^ *expr*	bitwise XOR
11	*expr* | *expr*	bitwise OR
12	*expr* && *expr*	logical AND
13	*expr* || *expr*	logical OR
14 (lowest)	*expr* ? *expr* : *expr*	conditional

All but two of the operators listed above will be familiar to C programmers. The exceptions are:

source location operator (@"file":line)

This operator evaluates to the address of the first instruction generated for a given line of source code. This extension was added to make it easier to add line breakpoints from the command interpreter. It has little practical value in the IDE where line breakpoints are supported directly through the user interface.

memory bank specifier operator (expr { expr })

This operator allows you to target data in a particular memory space. When an expression is evaluated, the result consists of a value part and a memory bank part. Both parts are represented by signed integers. The value part is the number resulting from evaluating the expression. For example, for the expression `10{1}` the value part of the expression is `10`. The memory bank part is `1`. If no memory bank is specified with the `{}` operator, the memory part is assumed to be `0`.

The memory bank part is used when addresses are dereferenced. For example, the expression `*0x1000{1}` results in the value stored in address `0x1000` of memory bank `1`, or XRAM, on the DSP.

In binary (or two-operand) operations, like multiplication and addition, the memory bank part of the result is assumed from the memory bank part of the left-hand subexpression. For example, the memory bank part of `0x1000{1} + 4{2}` is `1`.

For label names, the memory part is the bank in which the label resides. For register names, the memory bank depends on the register. On the CFX DSP, for example, the X-pointer registers, `XP0-XP7`, have a memory bank part of `1` corresponding to X data memory, the Y-pointer registers, `YP0-YP3`, a bank of `2` for Y data memory, and the rest default to `0`.

The member access operator (`.`) and pointer dereference operator (`->`) can be used to access bit fields in core registers and memory-mapped control registers.

CHAPTER 4
The Communications Toolkit (CTK) Configuration Manager

4.1 Introduction

The Communications Toolkit (CTK) Configuration Manager creates and manages communication configurations that other applications (such as the debuggers and EEPROM Manager) use to connect your computer to the Evaluation and Development Board or other chips in the SignaKlara technology family. With an EDK installed, the CTK Configuration Manager is seamlessly integrated into the IDE, so that most of the time you will not need to run it separately.

A configuration consists of the following information:

- **The name of the configuration.**
- **The communication interface to be used.** A variety of communication interfaces are supported, including the Total Phase™ Promira™ Serial Platform, the ON Semiconductor Communication Accelerator Adaptor, the ON Semiconductor Level Translator (as a separate board or integrated into an Evaluation and Development Board), and various industry-specific communication interfaces.
- **The communication protocol to be used.** For EDK tools communicating with Ezairo 7100 hardware, the I^2C protocol is used.
- **Parameters required to configure the communication interface.** Depending on the communication interface, these might include the COM port to which the communication interface is attached, or the “ear” on the interface that is used to connect to the hardware.
- **Parameters required to configure the communication protocol.** These include the address, communication speed, multi-master (on or off), and the maximum transaction size.

Configuration names can contain letters and numbers. We recommend that you use configuration names that are meaningful to you, especially if you will use more than one configuration on a regular basis.

The EDK tools store a reference to the name of a CTK configuration to use. If a configuration is changed using the CTK Configuration Manager, then the change is reflected in every application that uses that configuration. Therefore the two default configurations that are included with the EDK (CFX_CFG and CM3_CFG) should not be changed. If you require different settings, we recommend creating a copy of a default configuration and modifying it, or creating your own configuration.

4.2 Launching the CTK Configuration Manager

Each EDK tool that communicates with hardware provides an interface for selecting the configuration to be used for communication, and a method of launching the CTK Configuration Manager. You can also launch the CTK Configuration Manager from outside the IDE.

4.2.1 From the Start Menu

You can launch the CTK Configuration Manager from **ON Semiconductor > CTK Configuration Manager** in the Windows **Start** menu.

4.2.2 From the IDE

Each launch configuration in the IDE maintains the name of the communication configuration used to communicate with the hardware when running or debugging the program. To launch the CTK Configuration Manager

from the IDE, click the **Connection** tab in a **Run Configurations...** or **Debug Configurations...** configuration window, or click **Window > Preferences...** and select **Communication Toolkit**. (See Figure 75.)

4.3 Using the CTK Configuration Manager

When you launch the CTK Configuration Manager from the IDE, you see the main window shown in Figure 75.

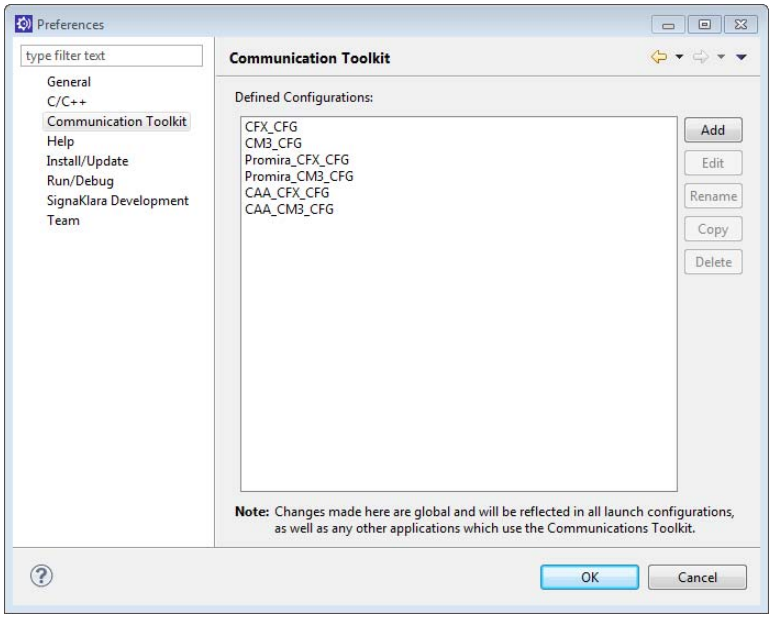


Figure 75. The Configuration Manager Main Window from the IDE

If you launch the CTK Configuration Manager from the **Start** menu, it looks like Figure 76.

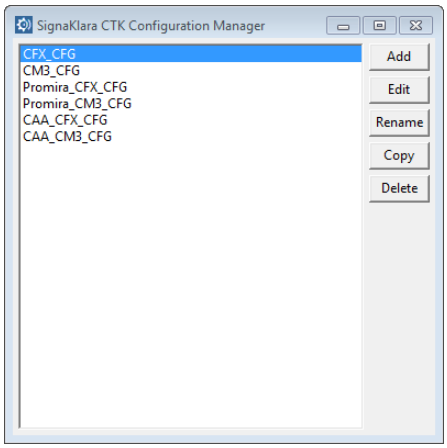


Figure 76. The Configuration Manager Main Window from the Start Menu

The central section of the window lists existing Defined Configurations. The buttons on the right allow you to add, edit or delete configurations.

To create a new configuration, click **Add**. You are prompted to enter a name for the new configuration. The Configuration Editor window allows you to fill in the configuration details.

To edit a configuration, select the configuration from the Defined Configurations list and click **Edit**. This opens the Configuration Editor window; a completed one is shown in Figure 77.

To delete a configuration, select the configuration from the Defined Configurations list and click **Delete**.

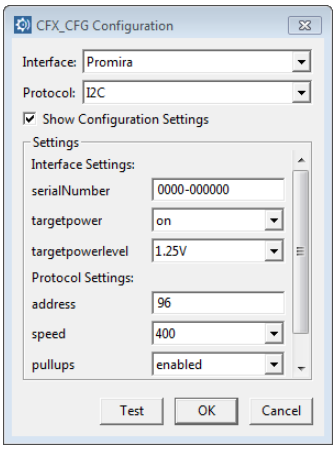


Figure 77. The Configuration Editor Window

The Configuration Editor window is divided into two regions, as shown in Figure 77.

At the top, you can select the communication interface and protocol for this configuration. Because some communication interfaces do not support all protocols, the choices in the Protocol drop-down list change when the Interface is changed.

The settings for the communication interface and protocol appear in the middle of the Configuration Editor window. The list of settings is different for each communication interface and protocol. Some settings can only take a fixed number of different values, and are presented as a drop-down list. Other settings can take arbitrary numeric or text values and have text-entry boxes.

In all cases, you can see a short description of the setting by hovering the mouse pointer over the setting. Section 4.4, “Configuration Examples” contains configuration examples and a brief description of common settings.

To check that the communication interface and protocol can be initialized with the current settings, click **Test**. To save the new or updated configuration, click **OK**. To discard your changes, click **Cancel**.

4.4 Configuration Examples

4.4.1 Promira Serial Platform Examples

Here are some common communication configuration examples for connecting to an Evaluation and Development Board using the Promira Serial Platform. For more information about the Promira adapter, see the *Promira Serial Platform Quick Start Guide*.

NOTE: To determine the serial number to put in the **serialNumber** field below, look at the bottom of the Promira adapter. If you do not enter a serial number, the CTK uses the first Promira adapter that it finds.

To communicate with the CFX core:

- **Interface:** Promira
- **Protocol:** I2C
- **Interface Settings:**
  - **serialNumber:** 2416-599650 *(see note above)*
  - **targetpower**: on
  - **targetpowerlevel**: 1.25V

- **Protocol Settings:**
  - **address:** 96
  - **speed:** 400
  - **pullups:** enabled
  - **maxTransactionSize:** 1024

To communicate with the ARM Cortex-M3 core:

- **Interface:** Promira
- **Protocol:** I2C
- **Interface Settings:**
  - **host:** 2416-599650 *(see note above)*
  - **targetpower**: on
  - **targetpowerlevel**: 1.25V

- **Protocol Settings:**
  - **address:** 64
  - **speed:** 400
  - **pullups:** enabled
  - **maxTransactionSize:** 1024

4.4.2 Communication Accelerator Adaptor Examples

Here are some common communication configuration examples for connecting to an Evaluation and Development Board using the Communication Accelerator Adaptor and a 6-pin mini DIN cable.

To communicate with the CFX core:

- **Interface:** Communication Accelerator Adaptor
- **Protocol:** I2C
- **Interface Settings:**
  - **iovoltage:** Vsense
  - **batteryvoltage:** 1.25V
  - **index:** 0

- **Protocol Settings:**
  - **address:** 96
  - **speed:** 400

    - **multi-master:** off
    - **maxTransactionSize:** 1024

To communicate with the ARM Cortex-M3 core:

- **Interface:** Communication Accelerator Adaptor
- **Protocol:** I2C
- **Interface Settings:**
    - **iovoltage:** Vsense
    - **batteryvoltage:** 1.25V
    - **index:** 0

- **Protocol Settings:**
    - **address:** 64
    - **speed:** 400
    - **multi-master:** off
    - **maxTransactionSize:** 1024

CHAPTER 5
Using the Documentation

5.1 Introduction

The following sections describe how to access the EDK documentation effectively. EDK documentation is available from the IDE in the following formats:

Welcome Screen

This is the screen you see by default when you start the IDE for the first time. You can view this screen at any time by selecting **Help > Welcome.** The Welcome Screen information provides a quick way to learn the basics, find out about what has changed from the last EDK release and any known issues, and get contact information.

Cheat Sheets

These are interactive tutorials to help you get started using the EDK. Step by step, they describe how to connect the board and download, run and debug a sample application. To access the cheat sheets, select **Help > Cheat Sheets**.

Online Help

The complete set of manuals for your product are available in searchable online help files, which can also be called up in context. For more information, see the following Section 5.2, “Using Online Help”.

NOTE: If you view the online help in a browser other than Internet Explorer, some pages are displayed with a default style instead of the customized style. For the best view of all the documentation, either use Internet Explorer or the internal help browser.

Printable Format

The manuals are also available as PDF files in the online help. See Section 5.3, “Printing the Documentation” on page 106 for details.

Stand-Alone Documentation

You can access the manuals from outside the IDE. See Section 5.4, “Stand-Alone Documentation” on page 106 for details.

5.2 Using Online Help

This section provides some basic information about using the full-text search feature and the context-sensitive dynamic help. Online help can be displayed in two ways:

- *External help* appears in a separate help window. To see external help, select **Help > Help Contents**, or click the **Show in External Window** button as seen in Figure 81 on page 103 in the Integrated Help view.
- *Integrated help* appears in a view in your current perspective as shown in Figure 83 on page 105. To see an integrated help view, press **F1** or select **Help > Search** or **Help > Dynamic Help**.

You can access help via a full-text search or via context-sensitive dynamic help.

For complete information about using help, see the *Workbench User Guide* under **Help > Help Contents**.

5.2.1 Full-Text Search with External Help

To access the searching functionality from the external help window:

1. Choose **Help > Help Contents**. This displays the list of online documentation sets.

NOTE: You might see a message that says, “To help protect your computer, Windows Firewall has blocked some features of this program.” To avoid seeing this message each time, choose “Unblock”.

2. At the top left of the window, there is a search field. By default, all the documentation sets in the **Contents** panel are searched. See Section 5.2.1.1, “Narrowing the Search Scope in External Help” for how to narrow the search scope.
3. The search results appear in the left panel. When you select a specific result, its file appears in the right panel and the search term is highlighted as shown in Figure 79 on page 102. You might have to scroll through the file to find all the highlighted terms.

5.2.1.1 Narrowing the Search Scope in External Help

1. You can narrow the scope of the search by clicking the **Scope** link beside the Search field. This allows you to choose a particular documentation set or a particular manual to limit the scope of your search.
2. Click **New** to see a list of documentation you can select. You will see a dialog box like the one in Figure 78. Expand the topic lists, select the documents, and give your defined search scope a name.
3. When you click **OK**, you will see your defined search list in the Select Scope dialog.
4. Click **OK**, and you will see that the Scope has changed to your customized one. Perform a search as usual.

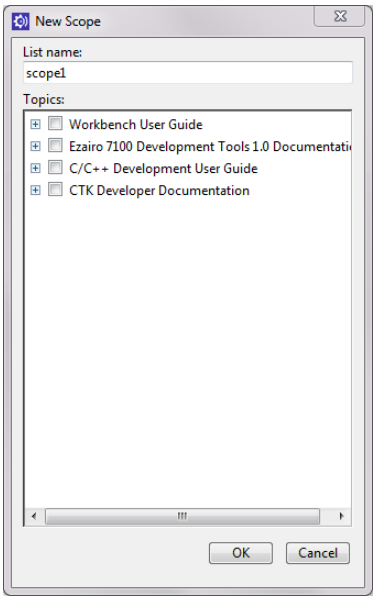


Figure 78. Narrowing the Scope of your Search in External Help

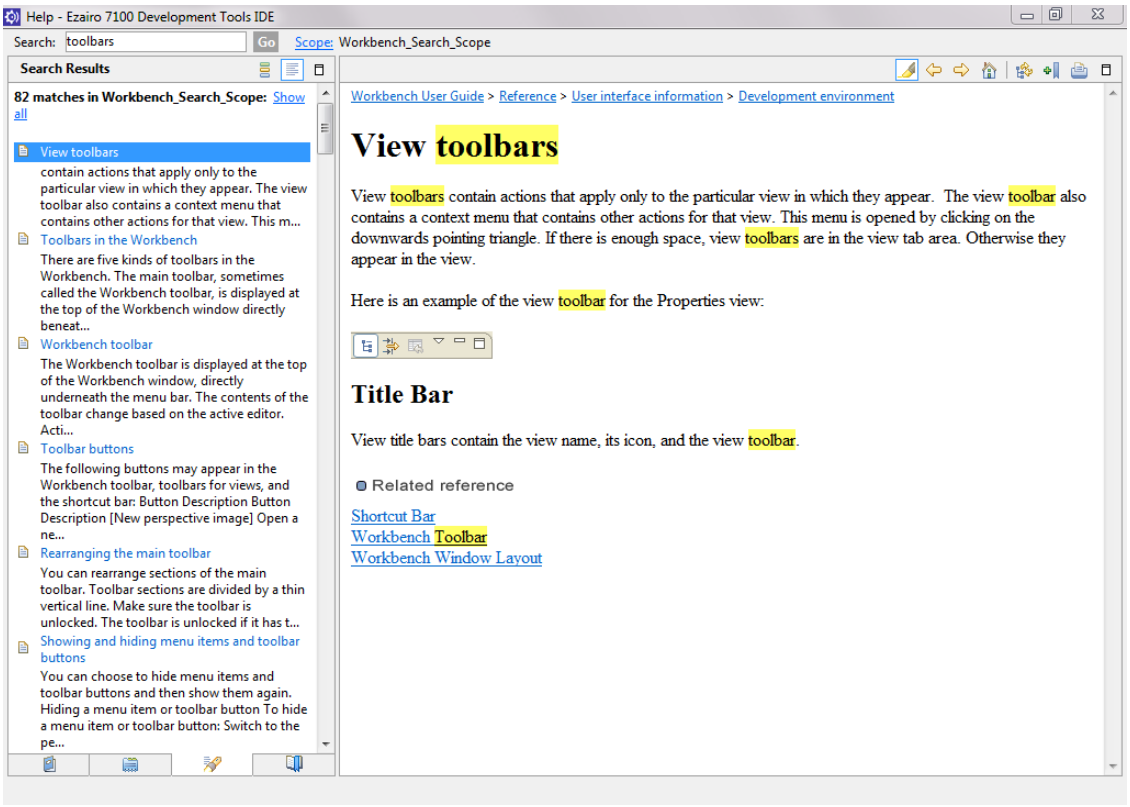


Figure 79. Results from a Full-Text Search with a Defined Search Scope

5.2.2 Full-Text Search with Integrated Help

To access full-text search from the integrated help:

1. Click **Help > Search**. This displays a help view with a Search field. By default, all of the local help files are searched.
2. If you would like to narrow the scope of the search, or broaden it to include the web, see Section 5.2.2.1, "Narrowing or Expanding the Search Scope in Integrated Help" on page 104.
3. Enter a search expression and click **Go** to carry out the search. All the search results are displayed in a list, grouped by search scope, below the search field as shown in Figure 80 on page 103. The figure is also an example of including Web Search in your scope. You can see the result of the Web Search by scrolling to the bottom of the search results.

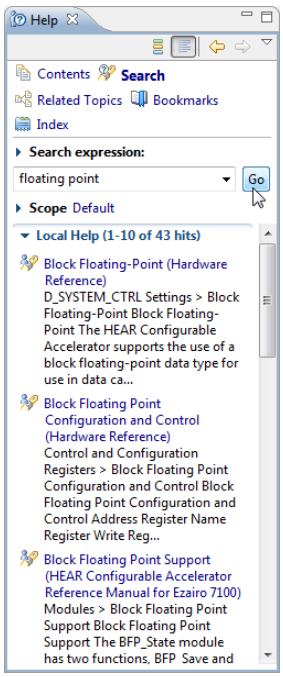


Figure 80. Search Results from Doing a Full-Text Search in Integrated Help, Including Searching the Web

4. If you click on a link to any of the local help files or the Web Search, they are displayed in the Help view, which might be too small. To get a bigger view of the help file, click on the **Show in external window** icon (see Figure 81 on page 103) and it will open a new window to display the help file.

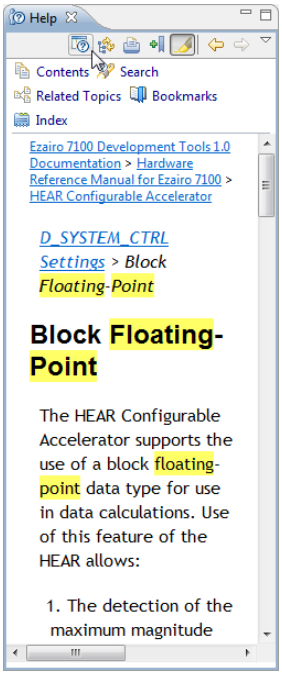


Figure 81. Integrated Help View Showing the Icon to Display the Help in an External Window

5.2.2.1 Narrowing or Expanding the Search Scope in Integrated Help

1. At the top of the integrated help tab, click the **Search** link. Then click the triangle next to **Scope**, then click **Advanced Settings**. This lets you define a default search scope. The **Scope - Default** dialog box appears.
2. Local Help is already there by default and includes all the local help files. You can edit this to change its scope of local searches:
   a. Select **Local Help**.
   b. Ensure that **Enable search engine** is selected.
   c. Click **Include only the following topics**.
   d. Expand the lists and select documents from the **Working set content** list.

3. To add another type of search, you can define a new search engine by clicking the **New...** button, and selecting a search engine depending on where you want to look for information.
   a. If you choose Local Help, click either **Search all topics** or **Search only the following topics**. If you choose the latter, you can select, via check boxes, the subset of the local documentation that you wish to search.

NOTE: You might want to have two versions of Local Help: one with its scope set to all documents, and a second one with its scope narrowed to the documents you use most frequently.

   b. If you choose **Web Search**, you will need a URL template:
      i. Open a browser.
      ii. Using your favorite browser search engine, search for something on the web.
      iii. Copy the web query from the browser address field to the **URL template** field in the Web Search panel.
      iv. Replace the search word or phrase with `{expression}`. See Figure 82 for an example.
      v. Click **Apply**.

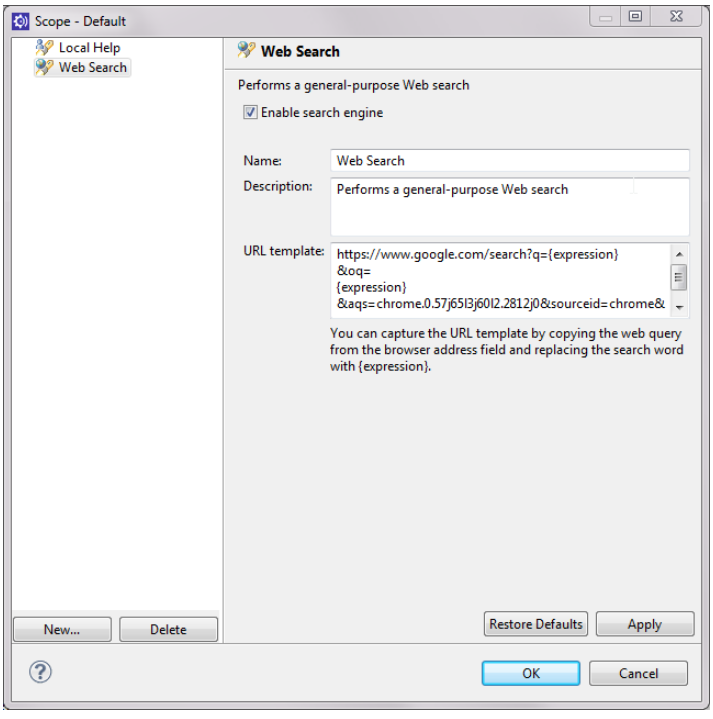


Figure 82. Defining a Search Scope in Integrated Help: Web Search

4. When you are finished defining new search engines and search scopes, click **OK** to leave the Advanced Settings.
5. Under the Search Scope list, you will see all your defined searches. Select or deselect them as needed, enter an expression in the search field and click **Go**.

5.2.3 Context-Sensitive Dynamic Help

The IDE also has context-sensitive dynamic help. This help works as follows:

1. Click on a term in any view and press **F1**. In the example shown in Figure 83 on page 105, `load` was highlighted when **F1** was pressed. An integrated help view appears on the right side of the perspective.

NOTE: If you click **Search** at the top of this view, you get the same full-text search feature that is described in Section 5.2.2, “Full-Text Search with Integrated Help” on page 102. If you click **Contents**, you get a table of contents of the local help files.

2. The Related Topics are grouped into **About** (the type of window you were in) and **Dynamic Help**. For example, if you are in the Source Editor, the first group of topics is about the editor. Under **Dynamic Help**, you find a list of results that contain the term you are seeking. The search scope is whatever you defined as your default.

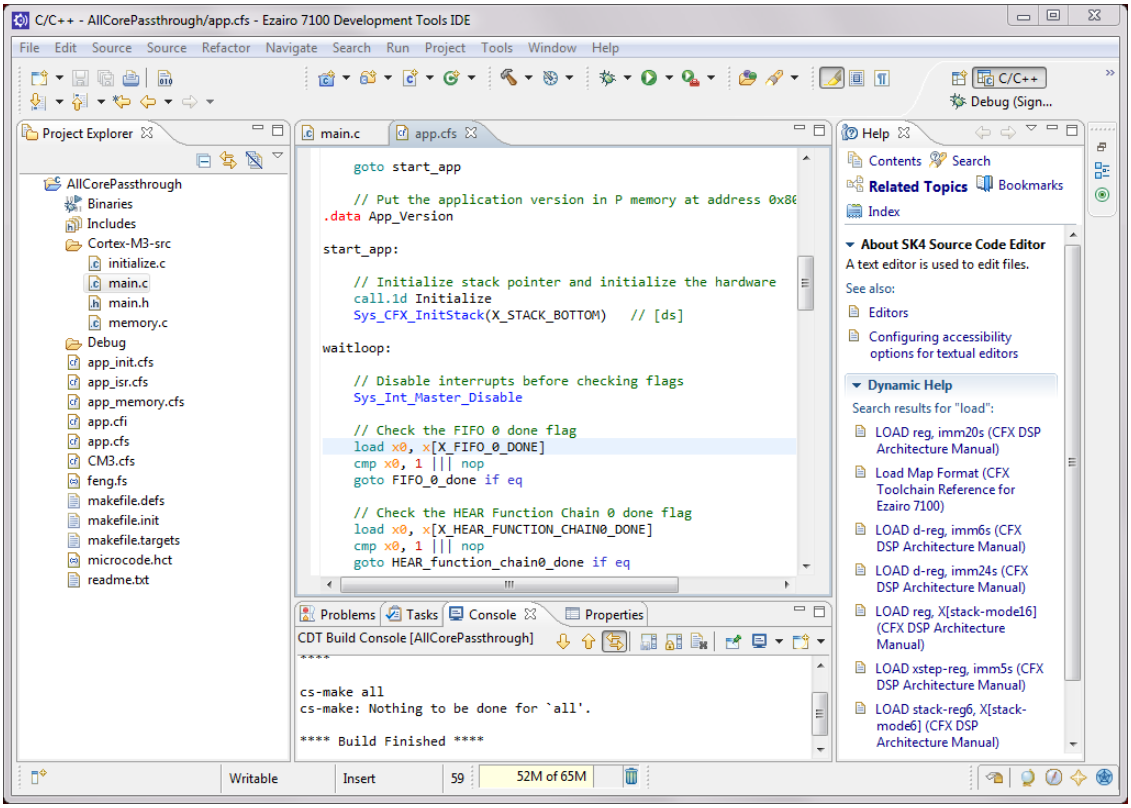


Figure 83. Results from Using Context-Sensitive Dynamic help

3. As you type in the Editor view, or click on other terms, the list of items in the Dynamic Help changes to match the search results relevant to your current term.

4. If you select one of these search results, the relevant help file is displayed in the Help view. To expand the help file, click the **Show in external window** button as shown in Figure 81 on page 103.

5.3 Printing the Documentation

At the top of the table of contents for each manual, there is a **Print topics** icon. Alternatively, you can access the PDF versions of the manuals as described in Section 5.4, "Stand-Alone Documentation".

To print a document:

1. Select the document you wish to print.
2. Click the **Print topics** icon.
3. Click either **Print selected topic** or **Print selected topic and all subtopics**.

The manual is displayed in a separate window and the Print dialog box appears. Select your desired printer and click **Print**.

If you want to view the PDF in Adobe Reader or Acrobat instead of the browser, you can configure Adobe Acrobat or Reader to open PDF files outside of the browser. To do this:

1. Open Acrobat or Reader.
2. Click **Edit > Preferences > Internet**.
3. Clear the **Display PDF in browser** check box.

NOTE: This change affects all PDF documents that you view through the browser.

5.4 Stand-Alone Documentation

You can view all the manuals as PDF files without starting the IDE. They are accessible from the following places:

- They are installed with the EDK in **<EDK_HOME>**/*documentation*. **<EDK_HOME>** is the folder where the EDK is installed. See the *Ezairo 7100 Installation Guide* for more information about the default installation location.
- They are available on the Windows Start menu. Click **ON Semiconductor > EDK 1 Ezairo 7100 Documentation**.

APPENDIX A
DDB Command Reference

A.1 INTRODUCTION

This appendix describes the commander object that is used on the CFX debug console (actually a Jython interpreter) and lists its methods. Each method description includes its return value, parameters, and the exceptions it can throw.

For an example of using the Jython console, see Section 3.4, “Jython Command Console” on page 85.

A.2 COMMANDER

Declaration

```
public interface Commander
```

All Known Implementing Classes:

```
DefaultCommander
```

Description

A high-level interface to DDB intended for use by interactive command languages

Table 4. Field Summary

Type	Name
int	CRAM_LSB Operate on the least significant bits in cram extension or truncation
int	CRAM_MSB Operate on the most significant bits in cram extension or truncation
int	CRAM_ONES Extend with one (high) bits when cramming
int	CRAM_SIGNED Extend with signed bits when cramming, that is, with the same value as the bit adjacent to the end in which the value is extended
int	CRAM_ZEROS Extend with zero (low) bits when cramming

Table 5. Member Summary

Return Type	Signature of the Method
long	***assign(java.lang.String lvalue, java.lang.String rvalue)*** Assign an expression the value of another expression
int[]	***breakAt(java.lang.String fileName, int lineNum)*** Set a breakpoint on a source line
int	***breakOn(int bank, long addr)*** Set a breakpoint on a memory location
void	***clear(java.lang.String fileName)*** Remove a file from the symbolic information database
void	***clearAll()*** Clear the entire symbolic information database
long	***cont()*** Continue execution
long	***contFrom(long addr)*** Continue execution from a specified address
void	***contnb()*** Continue from current program counter, without blocking (in other words, without locking up the user interface while it waits for a breakpoint to be hit)
void	***contnbFrom(long addr)*** Continue from a specified address, without blocking (in other words, without locking up the user interface while it waits for a breakpoint to be hit)
int	***copy(int dstBank, long dstAddr, int srcBank, long srcAddr, int len)*** Copy a block of data from one part of memory to another
int	***cram(int bank, long addr, java.lang.String fileName)*** Cram data into memory from a file in relative format
int	***cramAbs(java.lang.String fileName)*** Cram data into memory from a file in absolute format

Table 5. Member Summary (Continued)

Return Type	Signature of the Method
int	***cramAbsEx(java.lang.String fileName, int truncEnd, int extendEnd, int extendWith)*** An extended version of cramAbs() that has options for truncating and extending mis-sized values
int	***cramEx(int bank, long addr, java.lang.String fileName, int truncEnd, int extendEnd, int extendWith)*** An extended version of cram() that has options for truncating and extending mis-sized values
java.lang.String	***disassemble(int bank, long addr, int length)*** Disassemble a block of code
int	***display(java.lang.String expr)*** Create a display expression that is displayed whenever execution halts.
int	***dump(int bank, long addr, int len, java.lang.String fileName)*** Dump the contents of memory to a file in relative format
int	***dumpAbs(int bank, long addr, int len, java.lang.String fileName)*** Dump the contents of memory to a file in absolute format
boolean	***extBreak(int index, boolean enabled, int mode, int[] options)*** Configure an external breakpoint
long	***eval(java.lang.String expr)*** Evaluate a simple expression
int	***fill(int bank, long addr, int len, long val)*** Fill a range of memory with a value
long	***get(java.lang.String regName)*** Get the value of a register
com.signaklara.ddb.session.Session	***getSession()*** Returns the underlying session object
long	***halt()*** Halt execution of the target
void	***help()*** Print out a helpful message

Table 5. Member Summary (Continued)

Return Type	Signature of the Method
`java.lang.Object`	***info(java.lang.String field)*** Get information related to the debug session
`boolean`	***isInterruptEnabled()*** Print a message to the output stream to indicate whether interrupts are enabled on the target
`boolean`	***isRunning()*** Print a message to the output stream to indicate whether the target is executing
`long`	***istep()*** Step one instruction - deprecated
`void`	***load(java.lang.String fileName)*** Load a program into memory
`long`	***next()*** Step over one line
`long`	***nexti()*** Step over one instruction
`void`	***nextinb()*** Step over one instruction without blocking
`boolean`	***ping()*** Synchronize communication, then send and receive test data
`void`	***put(java.lang.String regName, long val)*** Put a value in a register
`long[]`	***read(int bank, long addr, int len)*** Read a block of memory, printing the data to the output stream
`void`	***rmBreak(int bkptId)*** Remove a breakpoint
`void`	***rmDisplay(int id)*** Remove one or all of the display expressions

Table 5. Member Summary (Continued)

Return Type	Signature of the Method
`long`	***run()*** Run the loaded program from its entry point
`void`	***setInterruptEnabled(boolean value)*** Set whether interrupts are enabled on the target
`void`	***setPC(long addr)*** Set the program counter to a new value
`long`	***step()*** Step one line of code
`long`	***stepi()*** Step one instruction
`void`	***symbols(java.lang.String fileName)*** Read symbolic information from a file without loading it
`java.lang.String`	***version()*** Get a version string
`long`	***where()*** Returns the value of the program counter
`int`	***write(int bank, long addr, long[] data)*** Write a block of memory
`int`	***xbreakOn(java.lang.String expr)*** Like `breakOn`, but the address is given as an expression
`long`	***xcontFrom(java.lang.String expr)*** Continue execution from a specified address
`void`	***xcontnbFrom(java.lang.String expr)*** Continue from a specified address, without blocking (in other words, without locking up the user interface while it waits for a breakpoint to be hit)
`int`	***xcopy(java.lang.String dstExpr, java.lang.String srcExpr, int len)*** Like `copy()`, but evaluates expressions to determine the source and destination locations

Table 5. Member Summary (Continued)

Return Type	Signature of the Method
`java.lang.String`	***xdisassemble(java.lang.String addrExpr, int length)*** Like `disassemble()`, but evaluates an expression to determine the start location
`int`	***xdump(java.lang.String expr, int len, java.lang.String fileName)*** Like `dump()`, but evaluates an expression to determine the start location
`int`	***xfill(java.lang.String expr, int len, java.lang.String val)*** Like `fill()`, but evaluates expressions to determine the start location and the fill value
`long[]`	***xread(java.lang.String expr, int len)*** Like `read()`, but evaluates an expression to determine the start location
`void`	***xsetPC(java.lang.String expr)*** Set the program counter using an expression
`int`	***xwrite(java.lang.String expr, long[] data)*** Like `write()`, but evaluates an expression to determine the start location

A.2.1 Methods

assign(String, String)

```
public long assign(java.lang.String lvalue, java.lang.String rvalue)
    throws EvalError, CommException
```

Assign an expression the value of another expression

Parameters:

`lvalue` - The expression to be assigned

`rvalue` - The value to assign to `lvalue`

Returns:

The new value

Throws:

`com.signaklara.ddb.expr.EvalError`

`com.signaklara.ddb.util.CommException`

breakAt(String, int)

```
public int[] breakAt(java.lang.String fileName, int lineNum) throws CommException
```

Set a breakpoint on a source line

Parameters:

`fileName` - The name of the source file

`lineNum` - The line number

Returns:

An array of breakpoint identifiers. Several breakpoints may be created by calling this breakpoint because macros can have several invocations.

Throws:

`com.signaklara.ddb.util.CommException`

breakOn(int, long)

```
public int breakOn(int bank, long addr) throws CommException, EvalError
```

Set a breakpoint on a memory location

Parameters:

`bank` - The memory bank

`addr` - The address

Returns:

A unique identifier for the new breakpoint

Throws:

`com.signaklara.ddb.util.CommException`

`com.signaklara.ddb.expr.EvalError`

clear(String)

```
public void clear(java.lang.String fileName)
```

Remove a file from the symbolic information database

Parameters:

`fileName` - The file to remove

clearAll()

```
public void clearAll()
```

Clear the entire symbolic information database

cont()

```
public long cont() throws CommException, NotificationError
```

Continue execution

Returns:

The value of the program counter when the target suspends

Throws:

`com.signaklara.ddb.util.CommException` - on communication error

`com.signaklara.ddb.util.NotificationError`

contFrom(long)

```
public long contFrom(long addr) throws CommException, NotificationError
```

Continue execution from a specified address

Parameters:

`addr` - The address to continue from

Returns:

The value of the program counter when the target suspends

Throws:

`com.signaklara.ddb.util.CommException` - on communication error

`com.signaklara.ddb.util.NotificationError`

contnb()

```
public void contnb() throws CommException, NotificationError
```

Continue from current program counter, without blocking (in other words, without locking up the user interface while it waits for a breakpoint to be hit)

Throws:

`com.signaklara.ddb.util.CommException`

`com.signaklara.ddb.util.NotificationError`

contnbFrom(long)

```
public void contnbFrom(long addr) throws CommException, NotificationError
```

Continue from a specified address, without blocking (in other words, without locking up the user interface while it waits for a breakpoint to be hit)

Parameters:

`addr` - The address to continue from

Throws:

`com.signaklara.ddb.util.CommException`

`com.signaklara.ddb.util.NotificationError`

copy(int, long, int, long, int)

```
public int copy(int dstBank, long dstAddr, int srcBank, long srcAddr, int len)
    throws CommException
```

Copy a block of data from one part of memory to another

Parameters:

`dstBank` - The destination bank

`dstAddr` - The destination address

`srcBank` - The source bank

`srcAddr` - The source address

`len` - The length of the block

Returns:

The number of words copied

Throws:

`com.signaklara.ddb.util.CommException` - on failure

cram(int, long, String)

```
public int cram(int bank, long addr, java.lang.String fileName)
    throws CommException, IOException
```

Cram data into memory from a file in relative format. See the *Communication Toolkit Programmer's Guide* for more information about this file format. This method is equivalent to `cramEx(bank, addr, fileName, CRAM_MSB, CRAM_MSB, CRAM_SIGNED)`.

To write data to memory from an array, see the `write` method.

Parameters:

`bank` - The memory bank

`addr` - The address to write the first word

`fileName` - The file to read

Returns:

The number of words crammed into memory

Throws:

`java.io.IOException`

`com.signaklara.ddb.util.CommException`

cramAbs(String)

```
public int cramAbs(java.lang.String fileName) throws CommException, IOException
```

Cram data into memory from a file in absolute format. See the *Communication Toolkit Programmer's Guide* for more information about this file format. This method is equivalent to `cramAbsEx(fileName, CRAM_MSB, CRAM_MSB, CRAM_SIGNED)`.

To write data to memory from an array, see the `write` method.

Parameters:

`fileName` - The file to read

Returns:

The number of words crammed into memory

Throws:

`java.io.IOException`

`com.signaklara.ddb.util.CommException`

cramAbsEx(String, int, int, int)

```
public int cramAbsEx(java.lang.String fileName, int truncEnd, int extendEnd, int
   extendWith)
throws CommException, IOException;
```

An extended version of cramAbs() that has options for truncating and extending mis-sized values.

Parameters:

`fileName` - The file to read

`truncEnd` - Which end of long values to truncate. Either `CRAM_MSB` for most significant bits, or `CRAM_LSB` for least.

`extendEnd` - Which end of short values to extend. Either `CRAM_MSB` for most significant bits, or `CRAM_LSB` for least.

`extendWith` - What value to use to extend short values. Either `CRAM_ONES`, `CRAM_ZEROS`, or `CRAM_SIGNED`, which extends using the value of the bit nearest the extension. For example, the value 0x8 is extended to 0xF8 when

extendEnd is CRAM_MSB, and 0x80 for CRAM_LSB. On the other hand, 0x1 is extended to 0x01 when extendEnd is CRAM_MSB, and 0x1F for CRAM_LSB.

Returns:

The number of words crammed into memory

Throws:

java.io.IOException

com.signaklara.ddb.util.CommException

cramEx(int, long, String, int, int, int)

```
public int cramEx(int bank, long addr, java.lang.String fileName, int truncEnd, int
    extendEnd, int extendWith)
throws CommException, IOException;
```

An extended version of cram() that has options for truncating and extending mis-sized values.

Parameters:

bank - The memory bank

addr - The address to write the first word

fileName - The file to read

truncEnd - Which end of long values to truncate. Either CRAM_MSB for most significant bits, or CRAM_LSB for least.

extendEnd - Which end of short values to extend. Either CRAM_MSB for most significant bits, or CRAM_LSB for least.

extendWith - What value to use to extend short values. Either CRAM_ONES, CRAM_ZEROS, or CRAM_SIGNED, which extends using the value of the bit nearest the extension. For example, the value 0x8 is extended to 0xF8 when extendEnd is CRAM_MSB, and 0x80 for CRAM_LSB. On the other hand, 0x1 is extended to 0x01 when extendEnd is CRAM_MSB, and 0x1F for CRAM_LSB.

Returns:

The number of words crammed into memory

Throws:

java.io.IOException

com.signaklara.ddb.util.CommException

disassemble(int, long, int)

```
public java.lang.String disassemble(int bank, long addr, int length) throws CommException
```

Disassemble a block of code

Parameters:

`bank` - The bank in which the code resides

`addr` - The address of the code block

`length` - The length of the code block

Returns:

The assembly code of the block of memory

Throws:

`com.signaklara.ddb.util.CommException`

display(String)

```
public int display(java.lang.String expr)
```

Create a display expression that is displayed whenever execution halts

Parameters:

`expr` - The expression to evaluate

Returns:

The ID of the expression, which can be used in the remove method

dump(int, long, int, String)

```
public int dump(int bank, long addr, int len, java.lang.String fileName)
    throws CommException, IOException
```

Dump the contents of memory to a file in relative format

Parameters:

`bank` - Bank of memory to read data from

`addr` - Address of first word of memory

`len` - Length of block to dump

`fileName` - Name of file to write

Returns:

The number of words written to the file

Throws:

`com.signaklara.ddb.util.CommException`

java.io.IOException

dumpAbs(int, long, int, String)

```
public int dumpAbs(int bank, long addr, int len, java.lang.String fileName)
    throws CommException, IOException
```

Dump the contents of memory to a file in absolute format

Parameters:

bank - Bank of memory to read data from

addr - Address of first word of memory

len - Length of block to dump

fileName - Name of file to write

Returns:

The number of words written to the file

Throws:

com.signaklara.ddb.util.CommException

java.io.IOException

extBreak(int, boolean, int, int[])

```
public boolean extBreak(int index, boolean enabled, int mode, int[] options)
    throws CommException
```

Configure an external breakpoint

Parameters:

index - The index of the external breakpoint to configure

enabled - True to enable the breakpoint; false to disable it

mode - The desired external breakpoint mode

options - The options for the external breakpoint

Returns:

True if the operation succeeds; false otherwise

Throws:

com.signaklara.ddb.util.CommException

eval(String)

```
public long eval(java.lang.String expr) throws EvalError, CommException
```

Evaluate a simple expression

Parameters:

`expr` - The expression to evaluate

Returns:

The result of evaluating `expr`

Throws:

`com.signaklara.ddb.expr.EvalError`

`com.signaklara.ddb.util.CommException`

fill(int, long, int, long)

```
public int fill(int bank, long addr, int len, long val) throws CommException
```

Fill a range of memory with a value

Parameters:

`bank` - The data bank to fill

`addr` - The starting address of the region

`len` - The length of the region

`val` - The value to write to each word in the region

Returns:

The number of words filled

Throws:

`com.signaklara.ddb.util.CommException`

get(String)

```
public long get(java.lang.String regName) throws CommException
```

Get the value of a register

Parameters:

`regName` - The name of the register to get

Returns:

The value of the register

Throws:

`com.signaklara.ddb.util.CommException` - on failure

getSession()

```
public com.signaklara.ddb.session.Session getSession()
```

Returns the underlying session object

All of the debugger's lower level functionality is available through the `Session` class, such as access to the debug information, target communications and breakpoints. For more information about the `Session` class, contact ON Semiconductor Customer Support.

Returns:

The underlying session object

halt()

```
public long halt() throws CommException
```

Halt execution of the target

Returns:

The value of the program counter

Throws:

`com.signaklara.ddb.util.CommException` - on communication error

help()

```
public void help()
```

Print out a helpful message

info(String)

```
public java.lang.Object info(java.lang.String field) throws CommException
```

Get information related to the debug session

Parameters:

`field` - The key of the information

Returns:

The information

Throws:

`com.signaklara.ddb.util.CommException`

isInterruptEnabled()

```
public boolean isInterruptEnabled() throws CommException
```

Print a message to the output stream to indicate whether interrupts are enabled on the target

Returns:

True if interrupts are enabled; false otherwise

Throws:

`com.signaklara.ddb.util.CommException` - on communication error

isRunning()

```
public boolean isRunning() throws CommException
```

Print a message to the output stream to indicate whether the target is executing

Returns:

True if running; false otherwise.

Throws:

`com.signaklara.ddb.util.CommException` - on communication error

istep()

```
public long istep() throws CommException, NotificationError
```

Step one instruction. This method is deprecated. Use "stepi()" on page 126 instead.

Returns:

The terminating address

Throws:

`com.signaklara.ddb.util.CommException` - on failure

`com.signaklara.ddb.util.NotificationError`

load(String)

```
public void load(java.lang.String fileName) throws CommException, IOException
```

Load a program into memory

Parameters:

`fileName` - The name of the file to load

Throws:

`com.signaklara.ddb.util.CommException` - on communication error

`java.io.IOException` - on read error

next()

```
public long next() throws CommException, NotificationError
```

Step over one line.

Returns:

The terminating address

Throws:

`com.signaklara.ddb.util.CommException` - on failure

`com.signaklara.ddb.util.NotificationError`

nexti()

```
public long nexti() throws CommException, NotificationError
```

Step over one instruction.

Returns:

The terminating address

Throws:

`com.signaklara.ddb.util.CommException` - on failure

`com.signaklara.ddb.util.NotificationError`

nextinb()

```
public void nextinb() throws CommException, NotificationError
```

Step over one instruction without blocking. In other words, it allows the program to run without waiting for the step to complete.

Throws:

`com.signaklara.ddb.util.CommException` - on failure

`com.signaklara.ddb.util.NotificationError`

ping()

```
public boolean ping()
```

Synchronize communication, then send and receive test data

Returns:

True if the ping succeeded; false otherwise

put(String, long)

```
public void put(java.lang.String regName, long val) throws CommException
```

Put a value in a register

Parameters:

`regName` - The name of the register to assign

`val` - The value to put in the register

Throws:

`com.signaklara.ddb.util.CommException` - on failure

read(int, long, int)

```
public long[] read(int bank, long addr, int len) throws CommException
```

Read a block of memory, printing the data to the output stream

Parameters:

`bank` - Bank of memory to read

`addr` - Address of first word

`len` - Number of words to read

Returns:

The data

Throws:

`com.signaklara.ddb.util.CommException`

rmBreak(int)

```
public void rmBreak(int bkptId)
```

Remove a breakpoint

Parameters:

`bkptId` - The identifier of the breakpoint to remove. If 0, all breakpoints will be removed.

rmDisplay(int)

```
public void rmDisplay(int id)
```

Remove one or all of the display expressions

Parameters:

`id` - The ID of the expression to remove. If 0, all display expressions are removed

run()

```
public long run() throws CommException, NoEntryException, NotificationError
```

Run the loaded program from its entry point

Returns:

The value of the program counter when the target suspends

Throws:

`com.signaklara.ddb.util.CommException` - on communication error

`com.signaklara.ddb.util.NoEntryException` - if no entry point is found

`com.signaklara.ddb.util.NotificationError`

setInterruptEnabled(boolean)

```
public void setInterruptEnabled(boolean addr) throws CommException
```

Set whether interrupts are enabled on the target

Parameters:

`value` - True to enable interrupts; false to disable them

Throws:

`com.signaklara.ddb.util.CommException`

setPC(long)

```
public void setPC(long addr) throws CommException
```

Set the program counter to a new value

Parameters:

`addr` - The new value for the program counter

Throws:

`com.signaklara.ddb.util.CommException`

step()

```
public long step() throws CommException, NotificationError
```

Step one line of code

Returns:

The terminating address

Throws:

`com.signaklara.ddb.util.CommException` - on failure

`com.signaklara.ddb.util.NotificationError`

stepi()

```
public long stepi() throws CommException, NotificationError
```

Step one instruction

Returns:

The terminating address

Throws:

`com.signaklara.ddb.util.CommException` - on failure

`com.signaklara.ddb.util.NotificationError`

symbols(String)

```
public void symbols(java.lang.String fileName) throws IOException
```

Read symbolic information from a file without loading it

Parameters:

`fileName` - The name of the file to read

Throws:

`java.io.IOException` - on read error

version()

```
public java.lang.String version()
```

Get a version string

Returns:

The version string

where()

```
public long where() throws CommException
```

Returns the value of the program counter

Returns:

The value of the program counter

Throws:

`com.signaklara.ddb.util.CommException` - on communication error

write(int, long, long[])

```
public int write(int bank, long addr, long[] data) throws CommException
```

Write a block of memory from an array. See the `cram` methods for writing to memory from a file.

Parameters:

`bank` - Bank of memory to write

`addr` - Address of first word

`data` - Data to write

Returns:

Number of words written

Throws:

`com.signaklara.ddb.util.CommException`

xbreakOn(String)

```
public int xbreakOn(java.lang.String expr) throws CommException, EvalError
```

Like `breakOn`, but the address is given as an expression

Parameters:

`expr` - The address, specified as an expression

Returns:

A unique identifier for the breakpoint

Throws:

`com.signaklara.ddb.expr.EvalError`

`com.signaklara.ddb.util.CommException`

xcontFrom(String)

```
public long xcontFrom(java.lang.String expr) throws CommException, EvalError,
    NotificationError
```

Continue execution from a specified address

Parameters:

`expr` - The address to continue from as an expression

Returns:

The value of the program counter when the target suspends

Throws:

`com.signaklara.ddb.util.CommException` - on communication error

`com.signaklara.ddb.expr.EvalError`

`com.signaklara.ddb.expr.NotificationError`

xcontnbFrom(String)

```
public void xcontnbFrom(java.lang.String expr) throws CommException, EvalError,
    NotificationError
```

Continue from a specified address, without blocking (in other words, without locking up the user interface while it waits for a breakpoint to be hit)

Parameters:

`expr` - The address to continue from as an expression

Throws:

`com.signaklara.ddb.util.CommException`

`com.signaklara.ddb.expr.EvalError`

`com.signaklara.ddb.expr.NotificationError`

xcopy(String, String, int)

```
public int xcopy(java.lang.String dstExpr, java.lang.String srcExpr, int len)
    throws CommException, EvalError
```

Like `copy()`, but evaluates expressions to determine the source and destination locations

Parameters:

`dstExpr` - An expression that evaluates to the destination address

`srcExpr` - An expression that evaluates to the source address

`len` - The length of the block

Returns:

The number of words copied

Throws:

`com.signaklara.ddb.expr.EvalError`

com.signaklara.ddb.util.CommException

xdisassemble(String, int)

```
public java.lang.String xdisassemble(java.lang.String addrExpr, int length)
    throws CommException, EvalError
```

Like `disassemble()`, but evaluates an expression to determine the start location

Parameters:

`addrExpr` - An expression that evaluates to the start address

`length` - The length of the code block

Returns:

The assembly code of the block of memory

Throws:

`com.signaklara.ddb.expr.EvalError`

`com.signaklara.ddb.util.CommException`

xdump(String, int, String)

```
public int xdump(java.lang.String expr, int len, java.lang.String fileName)
    throws CommException, IOException, EvalError
```

Like `dump()`, but evaluates an expression to determine the start location

Parameters:

`expr` - An expression that evaluates to the source address

`len` - Length of block to dump

`fileName` - Name of file to write

Returns:

The number of words written to the file

Throws:

`com.signaklara.ddb.util.CommException`

`java.io.IOException`

`com.signaklara.ddb.expr.EvalError`

xfill(String, int, String)

```
public int xfill(java.lang.String expr, int len, java.lang.String val)
    throws CommException, EvalError
```

Like `fill()`, but evaluates expressions to determine the start location and the fill value

Parameters:

`expr` - An expression that evaluates to the destination address

`len` - The length of the region

`val` - An expression that evaluates to the value to write to each word in the region

Returns:

The number of words filled

Throws:

`com.signaklara.ddb.util.CommException`

`com.signaklara.ddb.expr.EvalError`

xread(String, int)

```
public long[] xread(java.lang.String expr, int len) throws CommException, EvalError
```

Like `read()`, but evaluates an expression to determine the start location

Parameters:

`expr` - An expression that evaluates to the source address

`len` - Number of words to read

Returns:

The data

Throws:

`com.signaklara.ddb.util.CommException`

`com.signaklara.ddb.expr.EvalError`

xsetPC(String)

```
public void xsetPC(java.lang.String expr) throws CommException, EvalError
```

Set the program counter using an expression

Parameters:

`expr` - The new value for the program counter

Throws:

`com.signaklara.ddb.util.CommException`

com.signaklara.ddb.expr.EvalError

xwrite(String, long[])

```
public int xwrite(java.lang.String expr, long[] data) throws CommException, EvalError
```

Like `write()`, but evaluates an expression to determine the start location

Parameters:

`expr` - An expression that evaluates to the destination address

`data` - The data to write

Returns:

Number of words written

Throws:

com.signaklara.ddb.util.CommException

com.signaklara.ddb.expr.EvalError

ON Semiconductor and ON are registered trademarks of Semiconductor Components Industries, LLC (SCILLC). Ezairo is either a trademark or registered trademark of SCILLC. SCILLC owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of SCILLC's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. SCILLC reserves the right to make changes without further notice to any products herein. SCILLC makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does SCILLC assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in SCILLC data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. SCILLC does not convey any license under its patent rights nor the rights of others. SCILLC products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SCILLC product could create a situation where personal injury or death may occur. Should Buyer purchase or use SCILLC products for any such unintended or unauthorized application, Buyer shall indemnify and hold SCILLC and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that SCILLC was negligent regarding the design or manufacture of the part. SCILLC is an Equal Opportunity/Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:
Literature Distribution Center for ON Semiconductor
P.O. Box 5163, Denver, Colorado 80217 USA
Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada
Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada
Email: orderlit@onsemi.com

N. American Technical Support: 800-282-9855 Toll Free USA/Canada
Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910
Japan Customer Focus Center: Phone: 81-3-5817-1050

ON Semiconductor Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative

M-20805-010